

islr-ch6

Rohit Goswami

```
+++ title = "ISLR :: Linear Model Selection and Regularization" author = ["Rohit Goswami"] date = 2020-02-19T07:00:00+00:00 tags = ["solutions", "R", "ISLR"] categories = ["programming"] draft = false toc = true comments = true +++
```

Chapter VI - Linear Model Selection and Regularization

All the questions are as per the ISL seventh printing of the First edition^[^fn:1].

Common

Instead of using the standard functions, we will leverage the `mlr3` package^[^fn:2].

```
#install.packages("mlr3","data.table","mlr3viz","mlr3learners")
```

Actually for R version 3.6.2, the steps to get it working were a bit more involved.

Load ISLR and other libraries.

```
libsUsed<-c("dplyr","ggplot2","tidyverse",
            "ISLR","caret","MASS","gridExtra",
            "pls","latex2exp","data.table")
invisible(lapply(libsUsed, library, character.only = TRUE))
```

Question 6.8 - Page 262

In this exercise, we will generate simulated data, and will then use this data to perform best subset selection.

(a) Use the `=rnorm()`=function to generate a predictor X of length $n = 100$, as well as a noise vector η of length $n = 100$.

(b) Generate a response vector Y of length $n = 100$ according to the model

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \eta$$

, where β_0 , β_1 , β_2 , and β_3 are constants of your choice.

(c) Use the `regsubsets()` function to perform best subset selection in order to choose the best model containing the predictors X , X^2 , ..., X^{10} . What is the best model obtained according to C_p , BIC, and adjusted R^2 ? Show some plots to provide evidence for your answer, and report the coefficients of the best model obtained. Note you will need to use the `data.frame()` function to create a single data set containing both X and Y .

(d) Repeat (c), using forward stepwise selection and also using backwards stepwise selection. How does your answer compare to the results in (c)?

(e) Now fit a lasso model to the simulated data, again using X , X^2 , ..., X^{10} as predictors. Use cross-validation to select the optimal value of λ . Create plots of the cross-validation error as a function of λ . Report the resulting coefficient estimates, and discuss the results obtained.

(f) Now generate a response vector Y according to the model

$$Y = \beta_0 + \beta_7 X^7 + \eta,$$

and perform best subset selection and the lasso. Discuss the results obtained.

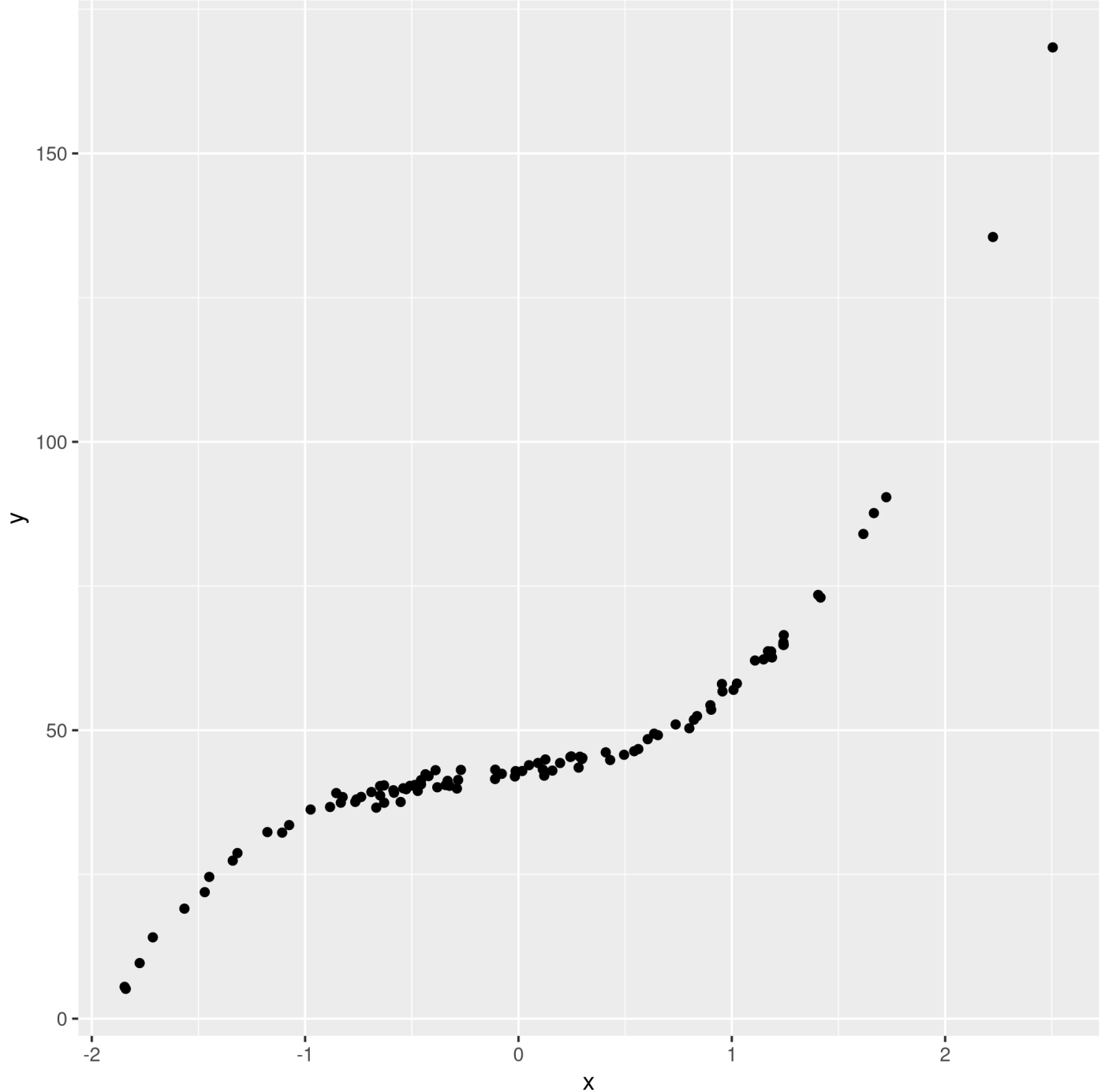
Answer

a) Generate model

```
set.seed(1984)
x<-rnorm(100)
noise<-rnorm(100)
```

b) Response vector

```
beta=c(43,5,3,6)
y<-beta[1] + beta[2]*x + beta[3]*x^2 + beta[4]*x^3 + noise
qplot(x,y)
```



c) Subset selection

Since the question requires it, we will be using the `leaps` libraries.

```
library(leaps)
df<-data.frame(y=y,x=x)
sets=regsubsets(y~poly(x,10,raw=T),data=df,nvmax=10)
sets %>% summary
```

```

## Subset selection object
## Call: regsubsets.formula(y ~ poly(x, 10, raw = T), data = df, nvmax = 10)
## 10 Variables (and intercept)
##
##              Forced in Forced out
## poly(x, 10, raw = T)1      FALSE      FALSE
## poly(x, 10, raw = T)2      FALSE      FALSE
## poly(x, 10, raw = T)3      FALSE      FALSE
## poly(x, 10, raw = T)4      FALSE      FALSE
## poly(x, 10, raw = T)5      FALSE      FALSE
## poly(x, 10, raw = T)6      FALSE      FALSE
## poly(x, 10, raw = T)7      FALSE      FALSE
## poly(x, 10, raw = T)8      FALSE      FALSE
## poly(x, 10, raw = T)9      FALSE      FALSE
## poly(x, 10, raw = T)10     FALSE      FALSE
## 1 subsets of each size up to 10
## Selection Algorithm: exhaustive
##
##      poly(x, 10, raw = T)1 poly(x, 10, raw = T)2 poly(x, 10, raw = T)3
## 1 ( 1 ) " " " " "*"
## 2 ( 1 ) " " "*" "*"
## 3 ( 1 ) "*" "*" "*"
## 4 ( 1 ) "*" "*" "*"
## 5 ( 1 ) "*" "*" "*"
## 6 ( 1 ) "*" " " "*"
## 7 ( 1 ) "*" "*" "*"
## 8 ( 1 ) "*" "*" "*"
## 9 ( 1 ) "*" "*" "*"
## 10 ( 1 ) "*" "*" "*"
##
##      poly(x, 10, raw = T)4 poly(x, 10, raw = T)5 poly(x, 10, raw = T)6
## 1 ( 1 ) " " " " "
## 2 ( 1 ) " " " " "
## 3 ( 1 ) " " " " "
## 4 ( 1 ) "*" " " "
## 5 ( 1 ) " " " " "
## 6 ( 1 ) "*" " " "*"
## 7 ( 1 ) "*" " " "*"
## 8 ( 1 ) "*" "*" "*"
## 9 ( 1 ) "*" " " "*"
## 10 ( 1 ) "*" "*" "*"
##
##      poly(x, 10, raw = T)7 poly(x, 10, raw = T)8 poly(x, 10, raw = T)9
## 1 ( 1 ) " " " " "
## 2 ( 1 ) " " " " "
## 3 ( 1 ) " " " " "
## 4 ( 1 ) " " " " "
## 5 ( 1 ) " " " " "*"
## 6 ( 1 ) " " "*" " "
## 7 ( 1 ) " " "*" " "
## 8 ( 1 ) " " "*" " "
## 9 ( 1 ) "*" "*" "*"
## 10 ( 1 ) "*" "*" "*"
##
##      poly(x, 10, raw = T)10
## 1 ( 1 ) " "
## 2 ( 1 ) " "
## 3 ( 1 ) " "
## 4 ( 1 ) " "
## 5 ( 1 ) "*"
## 6 ( 1 ) "*"
## 7 ( 1 ) "*"
## 8 ( 1 ) "*"
## 9 ( 1 ) "*"
## 10 ( 1 ) "*"

```

We also want the best parameters.

```
summarySet<-summary(sets)
which.min(summarySet$cp) %>% print
```

```
## [1] 3
```

```
which.min(summarySet$bic) %>% print
```

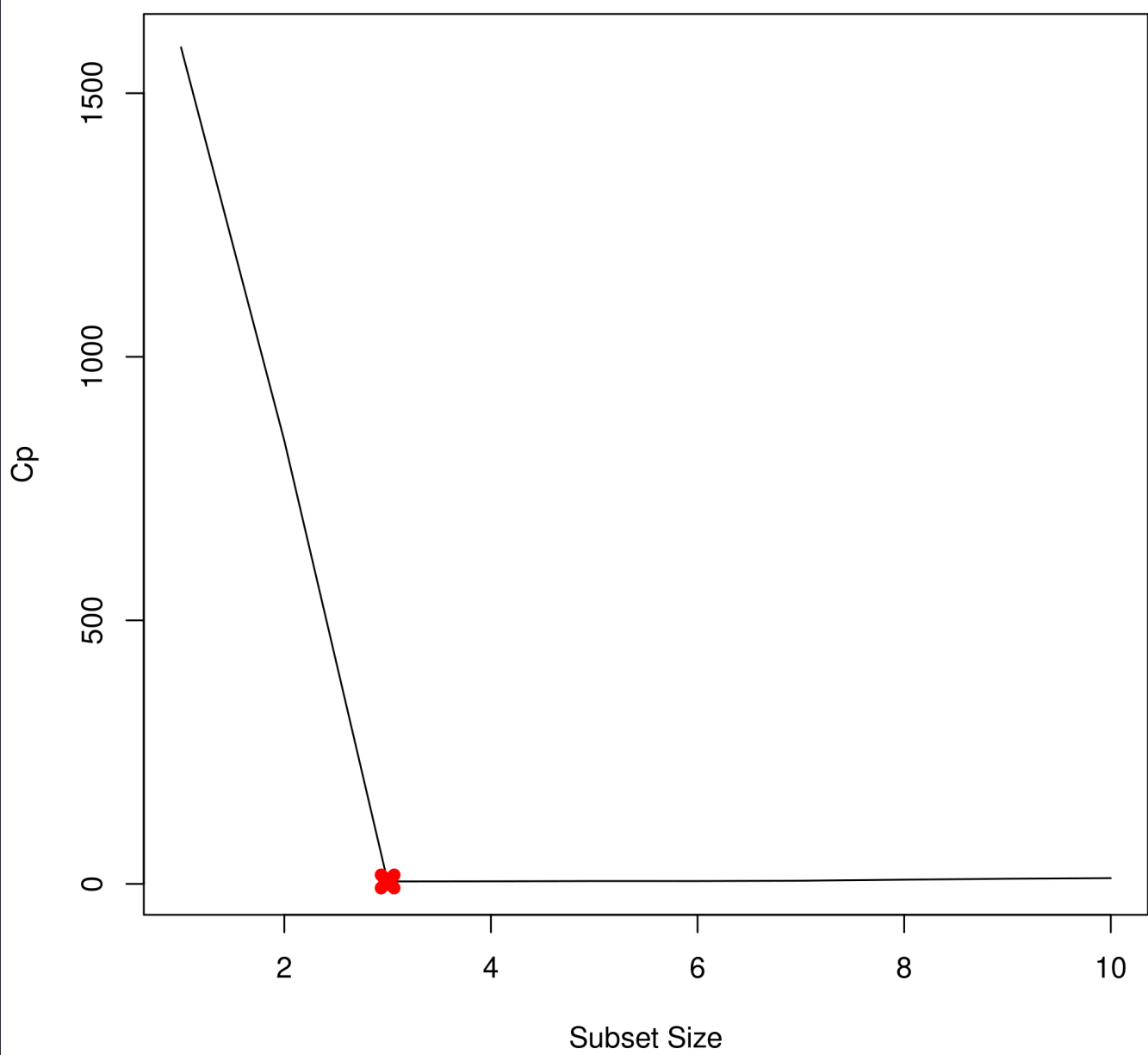
```
## [1] 3
```

```
which.max(summarySet$adjr2) %>% print
```

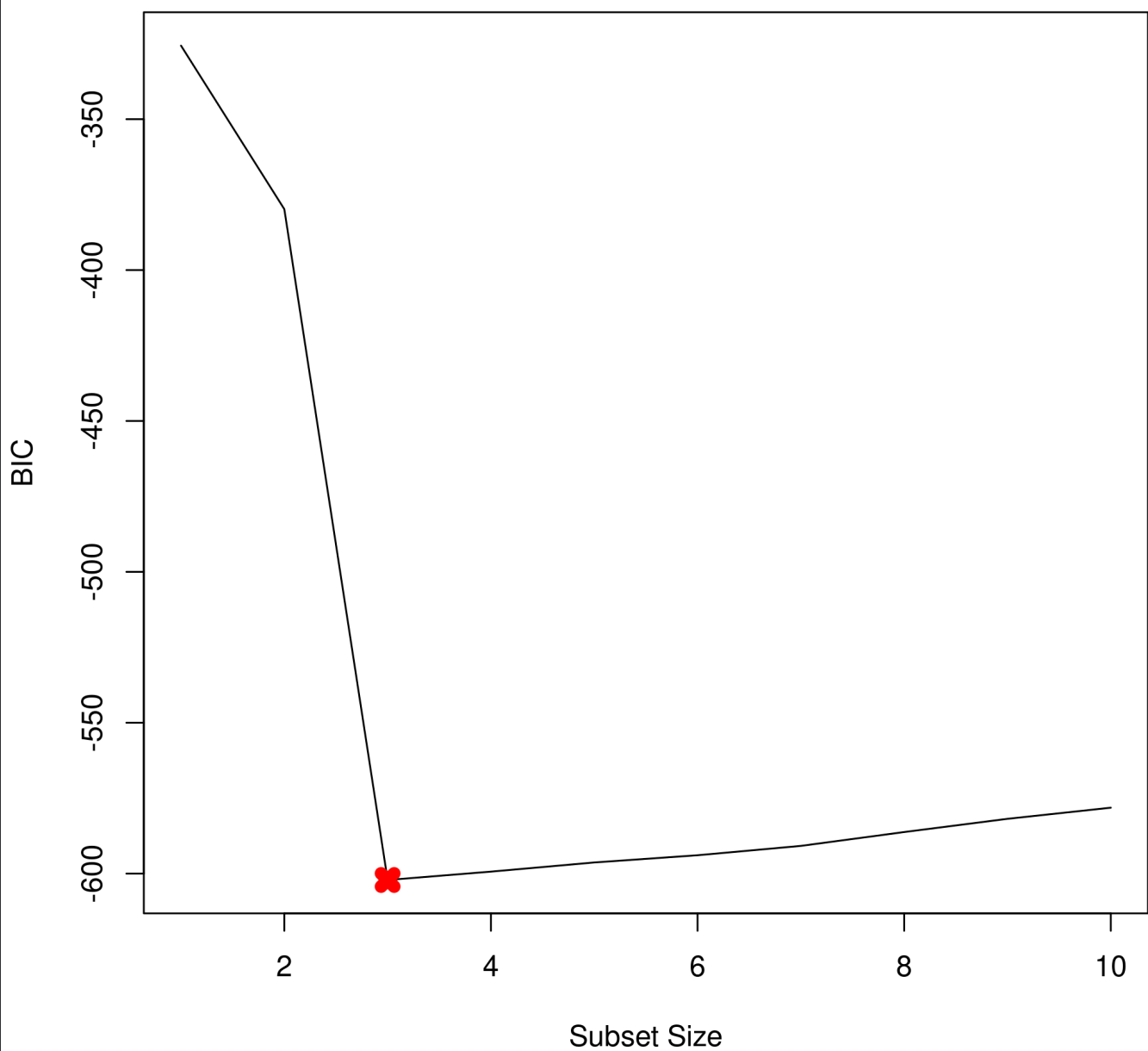
```
## [1] 7
```

We might want to see this as a plot.

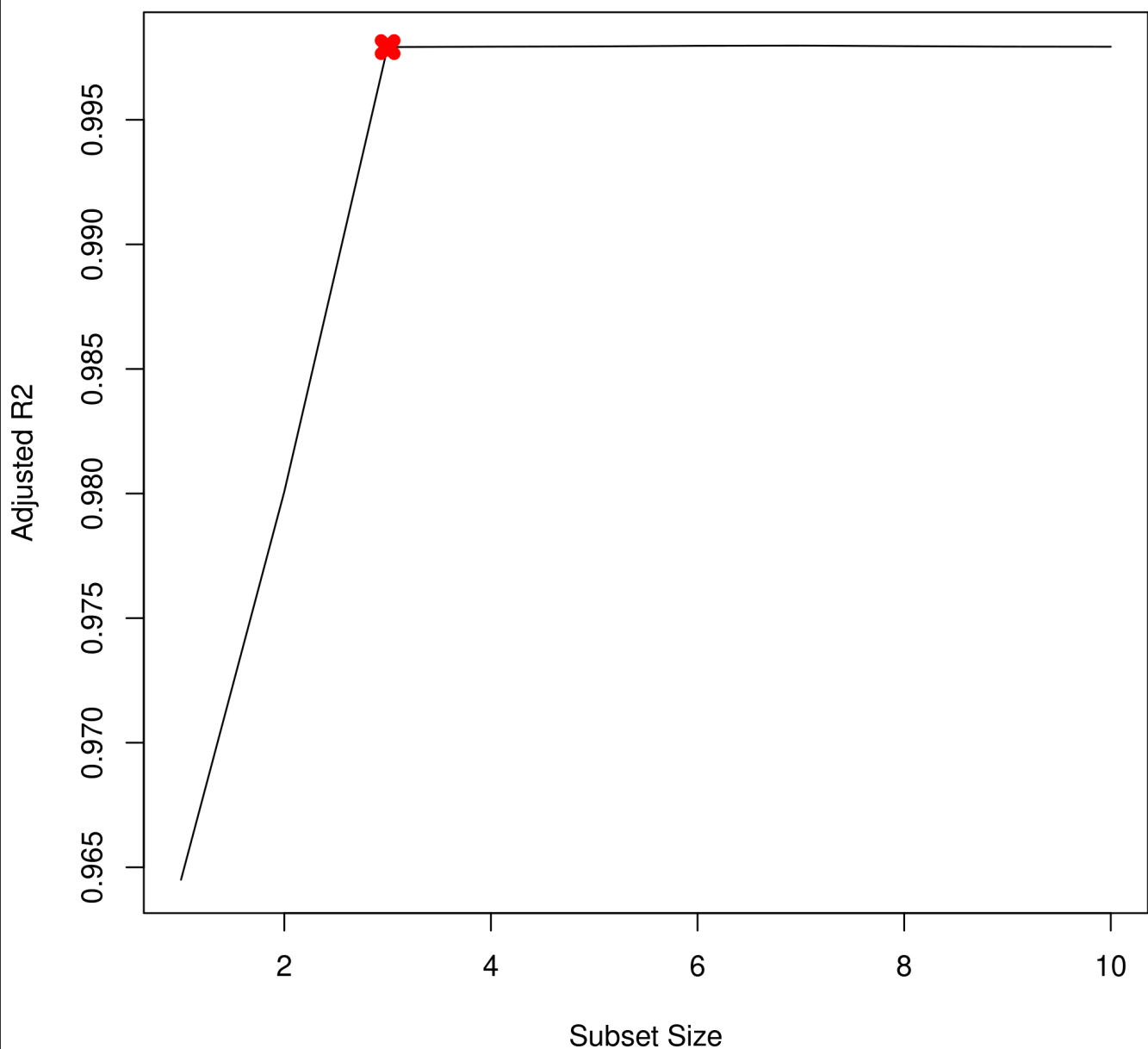
```
plot(summarySet$cp, xlab = "Subset Size", ylab = "Cp", pch = 20, type = "l")
points(3,summarySet$cp[3],pch=4,col='red',lwd=7)
```



```
plot(summarySet$bic, xlab = "Subset Size", ylab = "BIC", pch = 20, type = "l")
points(3,summarySet$bic[3],pch=4,col='red',lwd=7)
```



```
plot(summarySet$adjr2, xlab = "Subset Size", ylab = "Adjusted R2", pch = 20, type = "l")
points(3,summarySet$adjr2[3],pch=4,col='red',lwd=7)
```



Lets check the coefficients.

```
coefficients(sets,id=3) %>% print
```

```
##      (Intercept) poly(x, 10, raw = T)1 poly(x, 10, raw = T)2
##      42.895657      5.108094      3.034408
## poly(x, 10, raw = T)3
##      5.989367
```

```
beta %>% print
```

```
## [1] 43  5  3  6
```

We see that we actually have a pretty good set of coefficients.

d) Forward and backward stepwise models

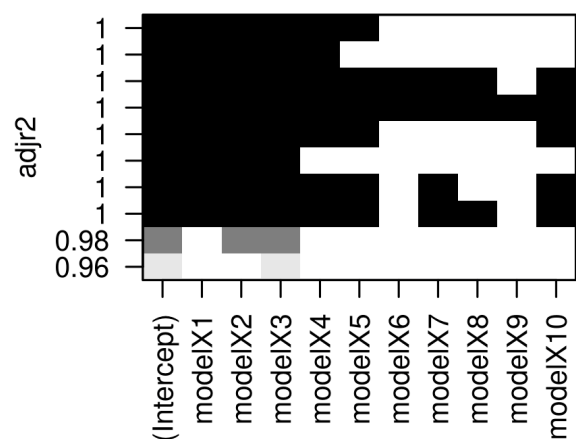
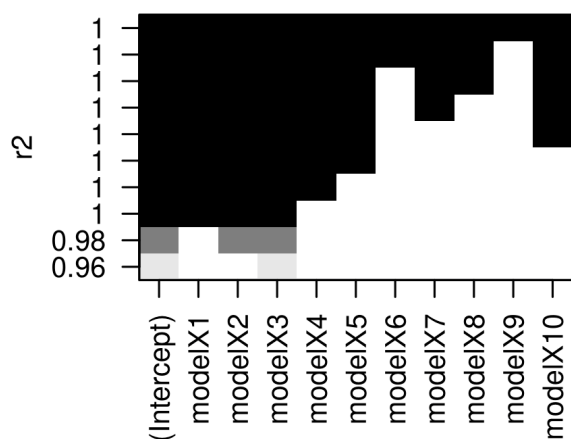
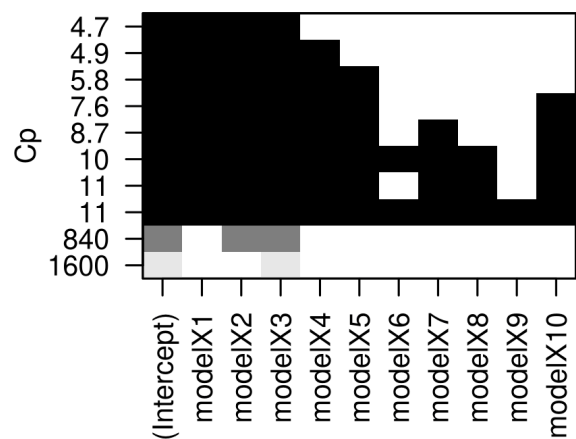
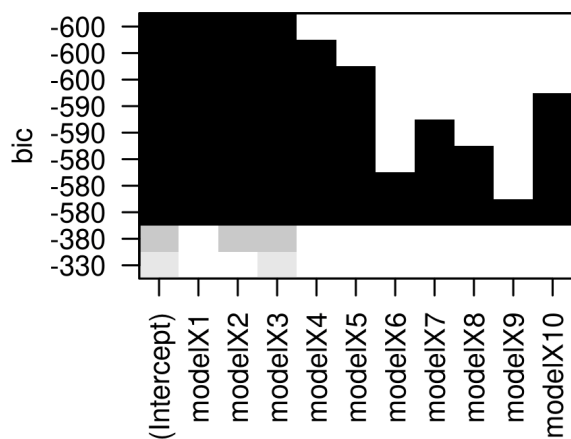

```
modelX<-poly(x,10,raw=T)
forwardFit<-regsubsets(y~modelX,data=df,nvmax=10,method="forward")
forwardFit %>% summary %>% print
```

```
## Subset selection object
## Call: regsubsets.formula(y ~ modelX, data = df, nvmax = 10, method = "forward")
## 10 Variables (and intercept)
##           Forced in Forced out
## modelX1      FALSE      FALSE
## modelX2      FALSE      FALSE
## modelX3      FALSE      FALSE
## modelX4      FALSE      FALSE
## modelX5      FALSE      FALSE
## modelX6      FALSE      FALSE
## modelX7      FALSE      FALSE
## modelX8      FALSE      FALSE
## modelX9      FALSE      FALSE
## modelX10     FALSE      FALSE
## 1 subsets of each size up to 10
## Selection Algorithm: forward
##           modelX1 modelX2 modelX3 modelX4 modelX5 modelX6 modelX7 modelX8
## 1 ( 1 ) " "      " "      "*"      " "      " "      " "      " "      " "
## 2 ( 1 ) " "      "*"      "*"      " "      " "      " "      " "      " "
## 3 ( 1 ) "*"      "*"      "*"      " "      " "      " "      " "      " "
## 4 ( 1 ) "*"      "*"      "*"      "*"      " "      " "      " "      " "
## 5 ( 1 ) "*"      "*"      "*"      "*"      "*"      " "      " "      " "
## 6 ( 1 ) "*"      "*"      "*"      "*"      "*"      "*"      " "      " "
## 7 ( 1 ) "*"      "*"      "*"      "*"      "*"      "*"      " "      "*"
## 8 ( 1 ) "*"      "*"      "*"      "*"      "*"      "*"      " "      "*"
## 9 ( 1 ) "*"      "*"      "*"      "*"      "*"      "*"      "*"      "*"
## 10 ( 1 ) "*"      "*"      "*"      "*"      "*"      "*"      "*"      "*"
##           modelX9 modelX10
## 1 ( 1 ) " "      " "
## 2 ( 1 ) " "      " "
## 3 ( 1 ) " "      " "
## 4 ( 1 ) " "      " "
## 5 ( 1 ) " "      " "
## 6 ( 1 ) " "      "*"
## 7 ( 1 ) " "      "*"
## 8 ( 1 ) " "      "*"
## 9 ( 1 ) " "      "*"
## 10 ( 1 ) "*"      "*"

```

We might want to take a look at these.

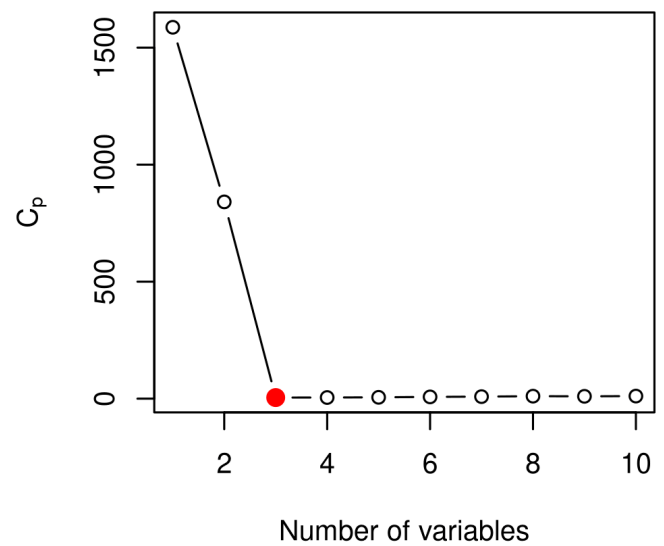
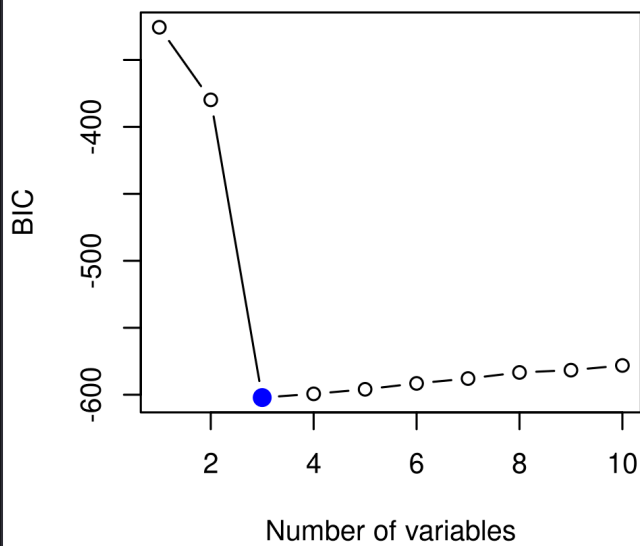
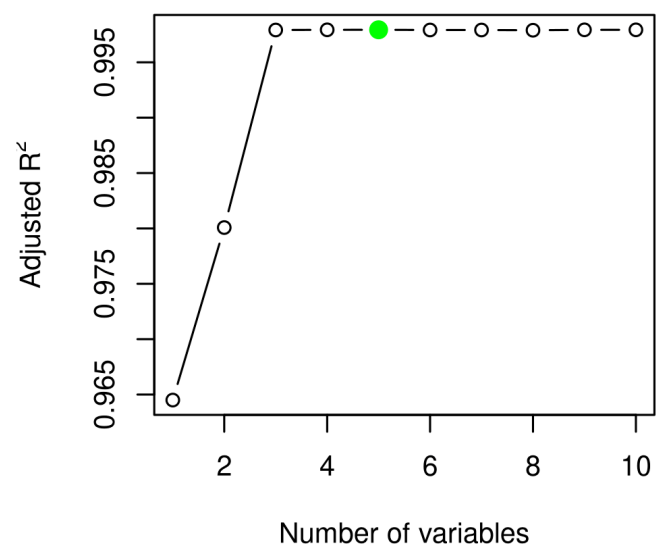
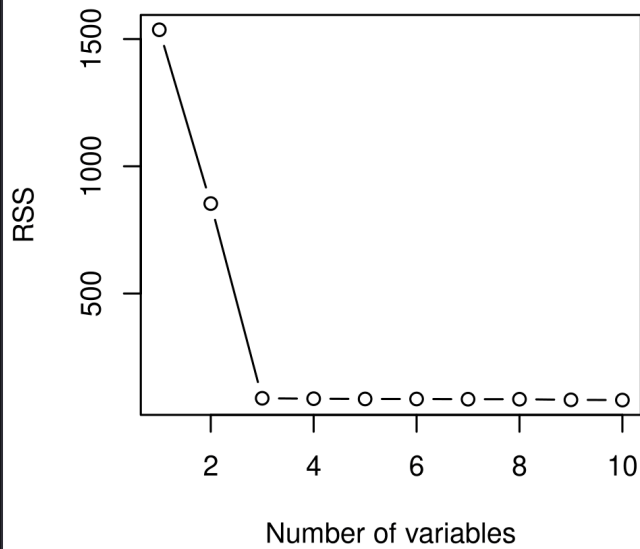
```
par(mfrow=c(2,2))
plot(forwardFit)
plot(forwardFit,scale='Cp')
plot(forwardFit,scale='r2')
plot(forwardFit,scale='adjr2')
```



I find these not as fun to look at, so we will do better.

```
plotLEAP=function(leapObj){
  par(mfrow = c(2,2))
  bar2=which.max(leapObj$adjr2)
  bbic=which.min(leapObj$bic)
  bcp=which.min(leapObj$cp)
  plot(leapObj$rss,xlab="Number of variables",ylab="RSS",type="b")
  plot(leapObj$adjr2,xlab="Number of variables",ylab=TeX("Adjusted R^2"),type="b")
  points(bar2,leapObj$adjr2[bar2],col="green",cex=2,pch=20)
  plot(leapObj$bic,xlab="Number of variables",ylab=TeX("BIC"),type="b")
  points(bbic,leapObj$bic[bbic],col="blue",cex=2,pch=20)
  plot(leapObj$cp,xlab="Number of variables",ylab=TeX("C_p"),type="b")
  points(bcp,leapObj$cp[bcp],col="red",cex=2,pch=20)
}
```

```
plotLEAP(forwardFit %>% summary)
```



Lets check the backward selection as well.

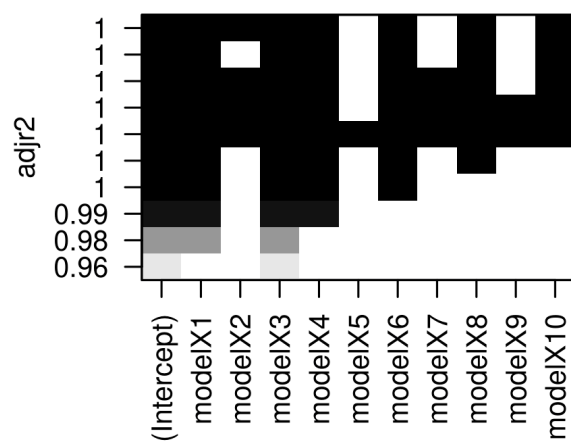
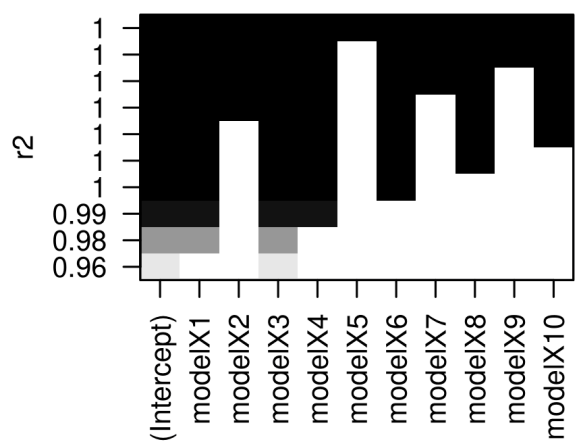
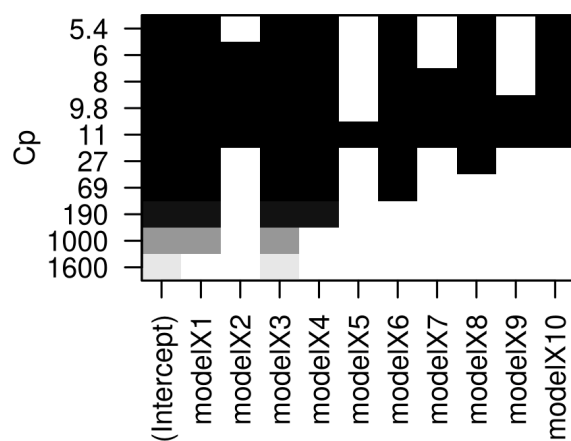
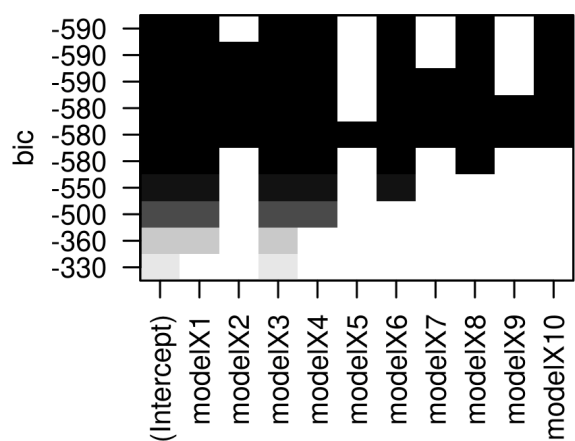
```
modelX<-poly(x,10,raw=T)
backwardFit<-regsubsets(y~modelX,data=df,nvmax=10,method="backward")
backwardFit %>% summary %>% print
```

```
## Subset selection object
## Call: regsubsets.formula(y ~ modelX, data = df, nvmax = 10, method = "backward")
## 10 Variables (and intercept)
##           Forced in Forced out
## modelX1      FALSE      FALSE
## modelX2      FALSE      FALSE
## modelX3      FALSE      FALSE
## modelX4      FALSE      FALSE
## modelX5      FALSE      FALSE
## modelX6      FALSE      FALSE
## modelX7      FALSE      FALSE
## modelX8      FALSE      FALSE
## modelX9      FALSE      FALSE
## modelX10     FALSE      FALSE
## 1 subsets of each size up to 10
## Selection Algorithm: backward
##           modelX1 modelX2 modelX3 modelX4 modelX5 modelX6 modelX7 modelX8
## 1 ( 1 ) " " " " "*" " " " " " " " "
## 2 ( 1 ) "*" " " "*" " " " " " " " "
## 3 ( 1 ) "*" " " "*" "*" " " " " " "
## 4 ( 1 ) "*" " " "*" "*" "*" " " " " "
## 5 ( 1 ) "*" " " "*" "*" "*" " " "*" " "
## 6 ( 1 ) "*" " " "*" "*" "*" " " "*" "*"
## 7 ( 1 ) "*" "*" "*" "*" "*" " " "*" "*"
## 8 ( 1 ) "*" "*" "*" "*" "*" " " "*" "*"
## 9 ( 1 ) "*" "*" "*" "*" "*" " " "*" "*"
## 10 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*" "*"
##           modelX9 modelX10
## 1 ( 1 ) " " " "
## 2 ( 1 ) " " " "
## 3 ( 1 ) " " " "
## 4 ( 1 ) " " " "
## 5 ( 1 ) " " " "
## 6 ( 1 ) " " "*"
## 7 ( 1 ) " " "*"
## 8 ( 1 ) " " "*"
## 9 ( 1 ) "*" "*"
## 10 ( 1 ) "*" "*"

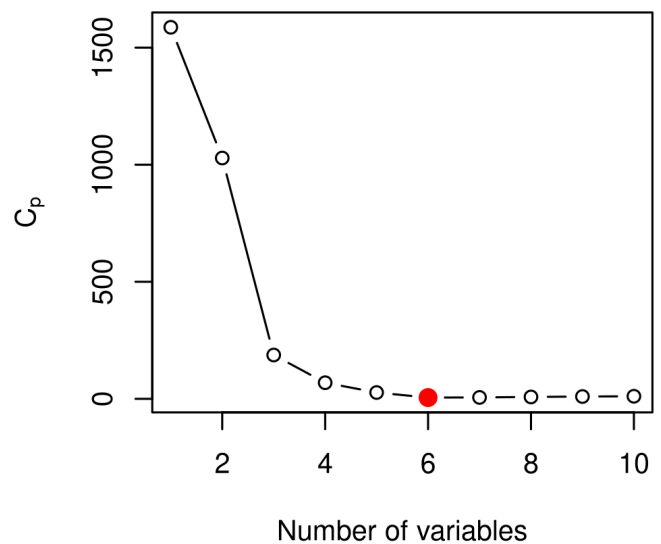
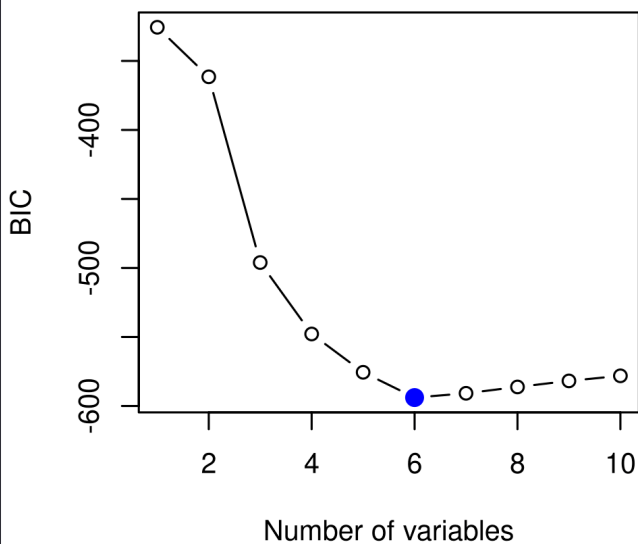
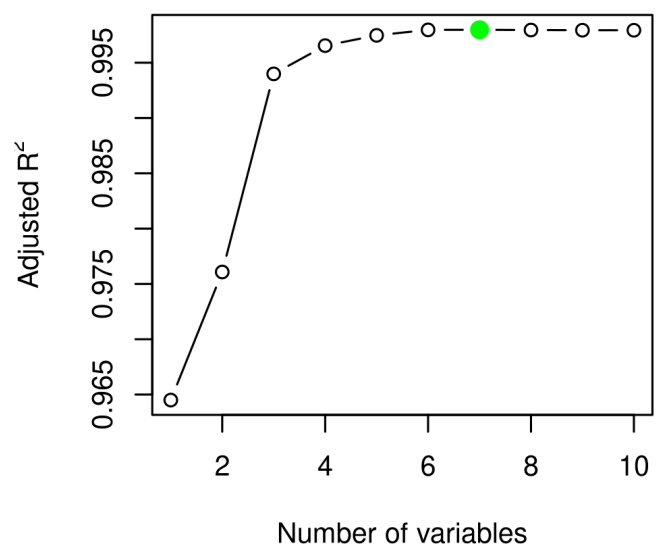
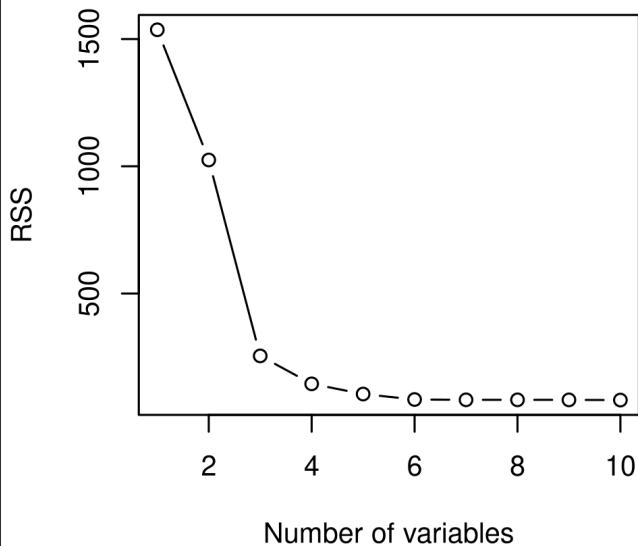
```

We might want to take a look at these.

```
par(mfrow=c(2,2))
plot(backwardFit)
plot(backwardFit,scale='Cp')
plot(backwardFit,scale='r2')
plot(backwardFit,scale='adjr2')
```



```
plotLEAP(backwardFit %>% summary)
```



In spite of some slight variations, overall all methods converge to the same **best** set of parameters, that of the third model.

e) LASSO and Cross Validation

For this, instead of using `glmnet` directly, we will use `caret`.

```
df<-df %>% mutate(x2=x^2,x3=x^3,
                  x4=x^4,x5=x^5,
                  x6=x^6,x7=x^7,
                  x8=x^8,x9=x^9,
                  x10=x^10)
```

```
lambda<-10^seq(-3, 3, length = 100)
lassoCaret= train(y~.,data=df,method="glmnet",tuneGrid=expand.grid(alpha=1,lambda=lambda))
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```



```
## glmnet
##
## 100 samples
## 10 predictor
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 100, 100, 100, 100, 100, 100, ...
## Resampling results across tuning parameters:
##
##  lambda      RMSE      Rsquared    MAE
##  1.000000e-03  1.009696  0.9965632  0.8051425
##  1.149757e-03  1.009696  0.9965632  0.8051425
##  1.321941e-03  1.009696  0.9965632  0.8051425
##  1.519911e-03  1.009696  0.9965632  0.8051425
##  1.747528e-03  1.009696  0.9965632  0.8051425
##  2.009233e-03  1.009696  0.9965632  0.8051425
##  2.310130e-03  1.009696  0.9965632  0.8051425
##  2.656088e-03  1.009696  0.9965632  0.8051425
##  3.053856e-03  1.009696  0.9965632  0.8051425
##  3.511192e-03  1.009696  0.9965632  0.8051425
##  4.037017e-03  1.009696  0.9965632  0.8051425
##  4.641589e-03  1.009696  0.9965632  0.8051425
##  5.336699e-03  1.009696  0.9965632  0.8051425
##  6.135907e-03  1.009696  0.9965632  0.8051425
##  7.054802e-03  1.009696  0.9965632  0.8051425
##  8.111308e-03  1.009696  0.9965632  0.8051425
##  9.326033e-03  1.009696  0.9965632  0.8051425
##  1.072267e-02  1.009696  0.9965632  0.8051425
##  1.232847e-02  1.009696  0.9965632  0.8051425
##  1.417474e-02  1.009696  0.9965632  0.8051425
##  1.629751e-02  1.009696  0.9965632  0.8051425
##  1.873817e-02  1.009696  0.9965632  0.8051425
##  2.154435e-02  1.009696  0.9965632  0.8051425
##  2.477076e-02  1.009696  0.9965632  0.8051425
##  2.848036e-02  1.009696  0.9965632  0.8051425
##  3.274549e-02  1.009696  0.9965632  0.8051425
##  3.764936e-02  1.009696  0.9965632  0.8051425
##  4.328761e-02  1.009696  0.9965632  0.8051425
##  4.977024e-02  1.009696  0.9965632  0.8051425
##  5.722368e-02  1.009696  0.9965632  0.8051425
##  6.579332e-02  1.009696  0.9965632  0.8051425
##  7.564633e-02  1.009637  0.9965632  0.8050666
##  8.697490e-02  1.009216  0.9965637  0.8047862
##  1.000000e-01  1.008901  0.9965636  0.8046468
##  1.149757e-01  1.009470  0.9965616  0.8054790
##  1.321941e-01  1.011206  0.9965561  0.8074253
##  1.519911e-01  1.014475  0.9965476  0.8104930
##  1.747528e-01  1.019202  0.9965383  0.8147296
##  2.009233e-01  1.025943  0.9965259  0.8203974
##  2.310130e-01  1.035374  0.9965094  0.8284187
##  2.656088e-01  1.048294  0.9964878  0.8393282
##  3.053856e-01  1.065717  0.9964592  0.8530952
##  3.511192e-01  1.088903  0.9964215  0.8701072
##  4.037017e-01  1.119433  0.9963715  0.8918217
##  4.641589e-01  1.158919  0.9963053  0.9193677
##  5.336699e-01  1.209841  0.9962136  0.9532842
##  6.135907e-01  1.275467  0.9960778  0.9957151
##  7.054802e-01  1.357247  0.9958966  1.0471169
##  8.111308e-01  1.457886  0.9956561  1.1087362
##  9.326033e-01  1.580743  0.9953362  1.1818188
##  1.072267e+00  1.729330  0.9949070  1.2696235
##  1.232847e+00  1.907599  0.9943306  1.3758463
##  1.417474e+00  2.120178  0.9935518  1.5059031
##  1.629751e+00  2.369642  0.9924954  1.6673393
```

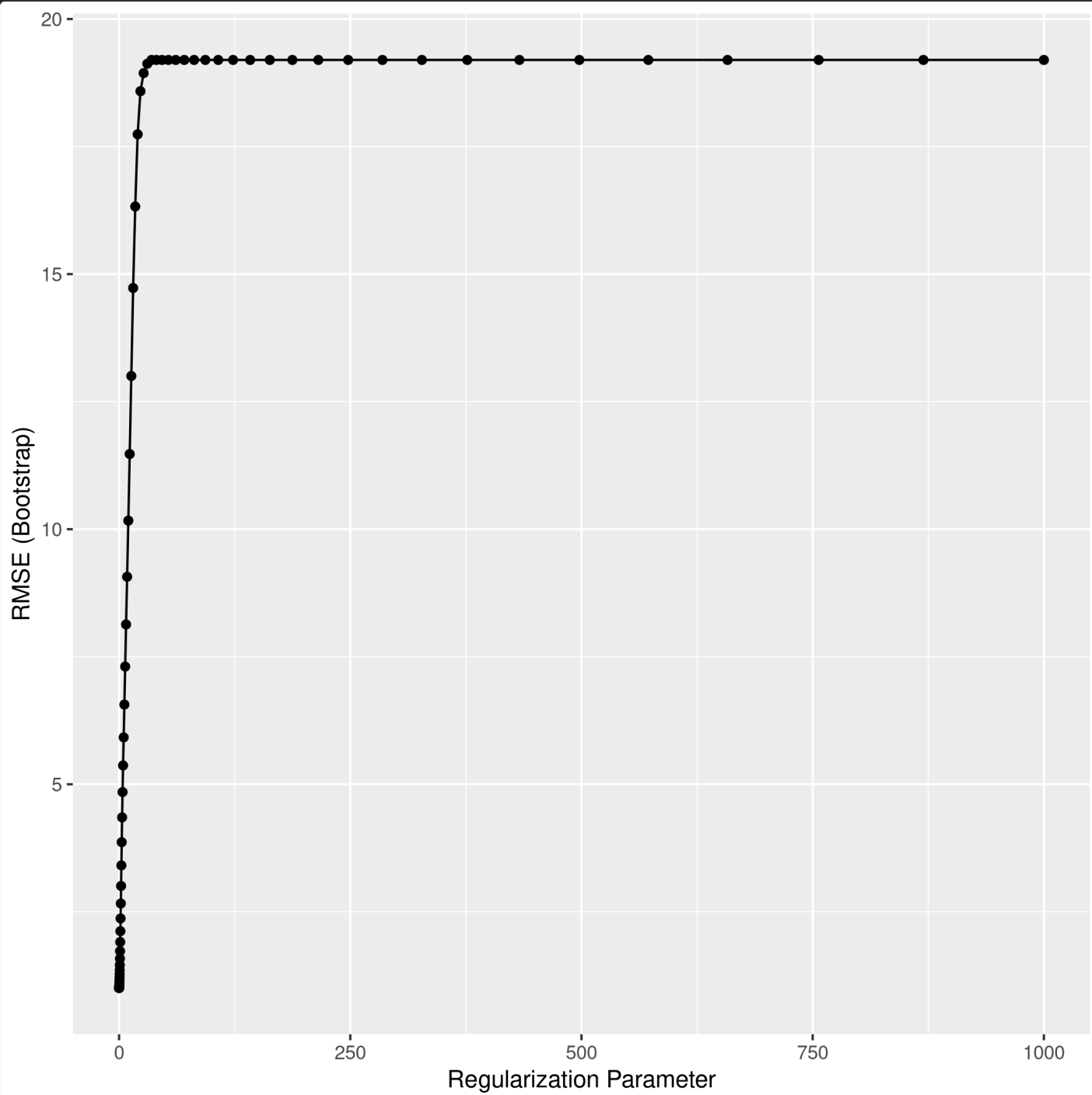


```

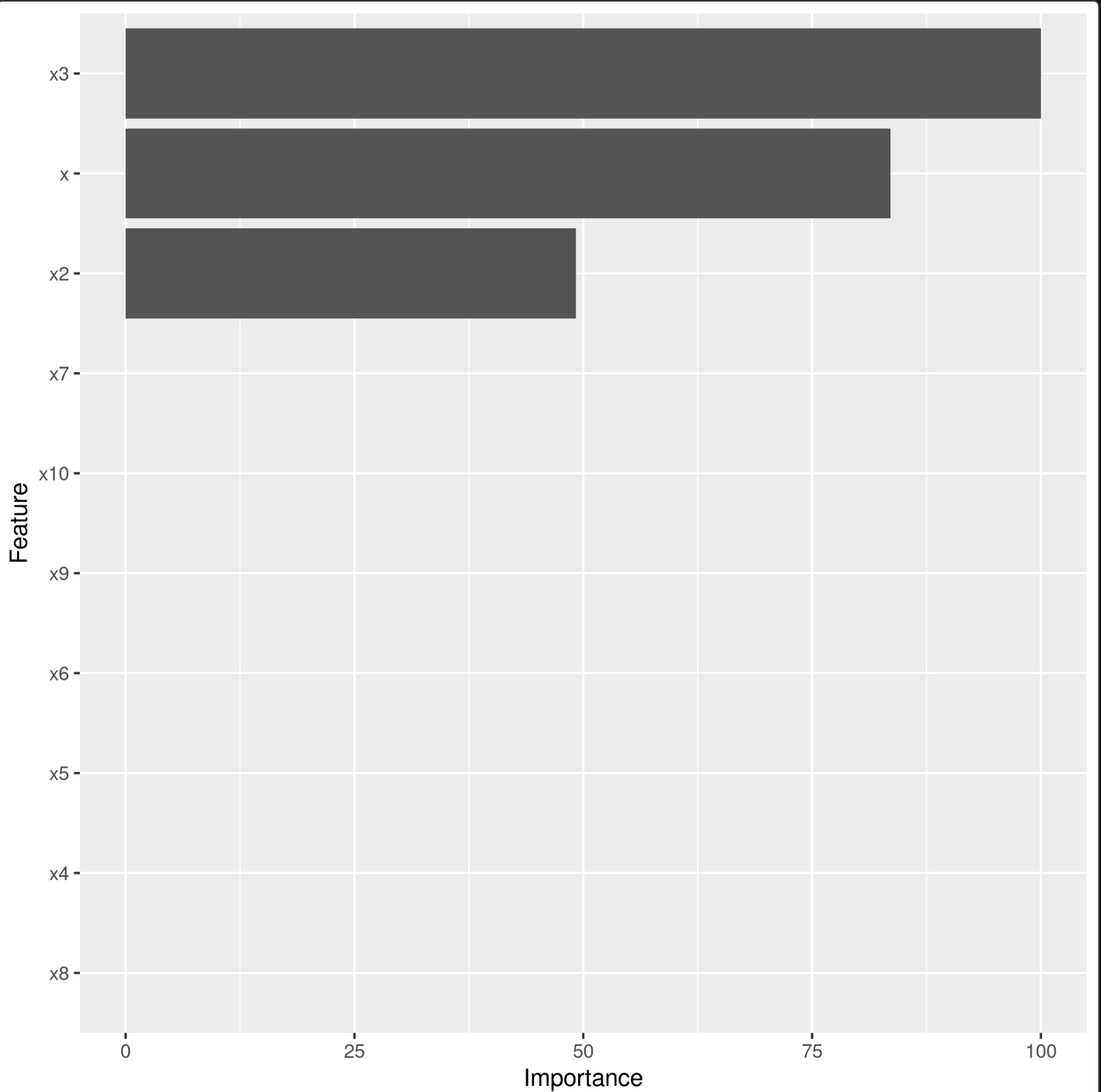
## 1.873817e+00 2.662906 0.9910539 1.8621728
## 2.154435e+00 3.007271 0.9890638 2.0978907
## 2.477076e+00 3.409377 0.9863097 2.3788439
## 2.848036e+00 3.864727 0.9825900 2.7053428
## 3.274549e+00 4.350785 0.9778541 3.0659309
## 3.764936e+00 4.847045 0.9724311 3.4403210
## 4.328761e+00 5.369017 0.9668240 3.8351441
## 4.977024e+00 5.919492 0.9626812 4.2512694
## 5.722368e+00 6.562134 0.9580843 4.7389049
## 6.579332e+00 7.307112 0.9534537 5.2945905
## 7.564633e+00 8.132296 0.9500300 5.8774541
## 8.697490e+00 9.067321 0.9486589 6.4760997
## 1.000000e+01 10.167822 0.9483195 7.1226569
## 1.149757e+01 11.473284 0.9482975 7.8556639
## 1.321941e+01 13.002703 0.9482975 8.6990451
## 1.519911e+01 14.727852 0.9454119 9.6414650
## 1.747528e+01 16.325210 0.9426796 10.5303097
## 2.009233e+01 17.740599 0.9357286 11.3560865
## 2.310130e+01 18.585795 0.9227167 11.8799668
## 2.656088e+01 18.939596 0.9080584 12.1336575
## 3.053856e+01 19.123568 0.9109065 12.2733471
## 3.511192e+01 19.197966 NaN 12.3308613
## 4.037017e+01 19.197966 NaN 12.3308613
## 4.641589e+01 19.197966 NaN 12.3308613
## 5.336699e+01 19.197966 NaN 12.3308613
## 6.135907e+01 19.197966 NaN 12.3308613
## 7.054802e+01 19.197966 NaN 12.3308613
## 8.111308e+01 19.197966 NaN 12.3308613
## 9.326033e+01 19.197966 NaN 12.3308613
## 1.072267e+02 19.197966 NaN 12.3308613
## 1.232847e+02 19.197966 NaN 12.3308613
## 1.417474e+02 19.197966 NaN 12.3308613
## 1.629751e+02 19.197966 NaN 12.3308613
## 1.873817e+02 19.197966 NaN 12.3308613
## 2.154435e+02 19.197966 NaN 12.3308613
## 2.477076e+02 19.197966 NaN 12.3308613
## 2.848036e+02 19.197966 NaN 12.3308613
## 3.274549e+02 19.197966 NaN 12.3308613
## 3.764936e+02 19.197966 NaN 12.3308613
## 4.328761e+02 19.197966 NaN 12.3308613
## 4.977024e+02 19.197966 NaN 12.3308613
## 5.722368e+02 19.197966 NaN 12.3308613
## 6.579332e+02 19.197966 NaN 12.3308613
## 7.564633e+02 19.197966 NaN 12.3308613
## 8.697490e+02 19.197966 NaN 12.3308613
## 1.000000e+03 19.197966 NaN 12.3308613
##
## Tuning parameter 'alpha' was held constant at a value of 1
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 1 and lambda = 0.1.

```

```
lassoCaret %>% ggplot
```



```
lassoCaret %>% varImp %>% ggplot
```



```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##  
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':  
##  
##   expand, pack, unpack
```

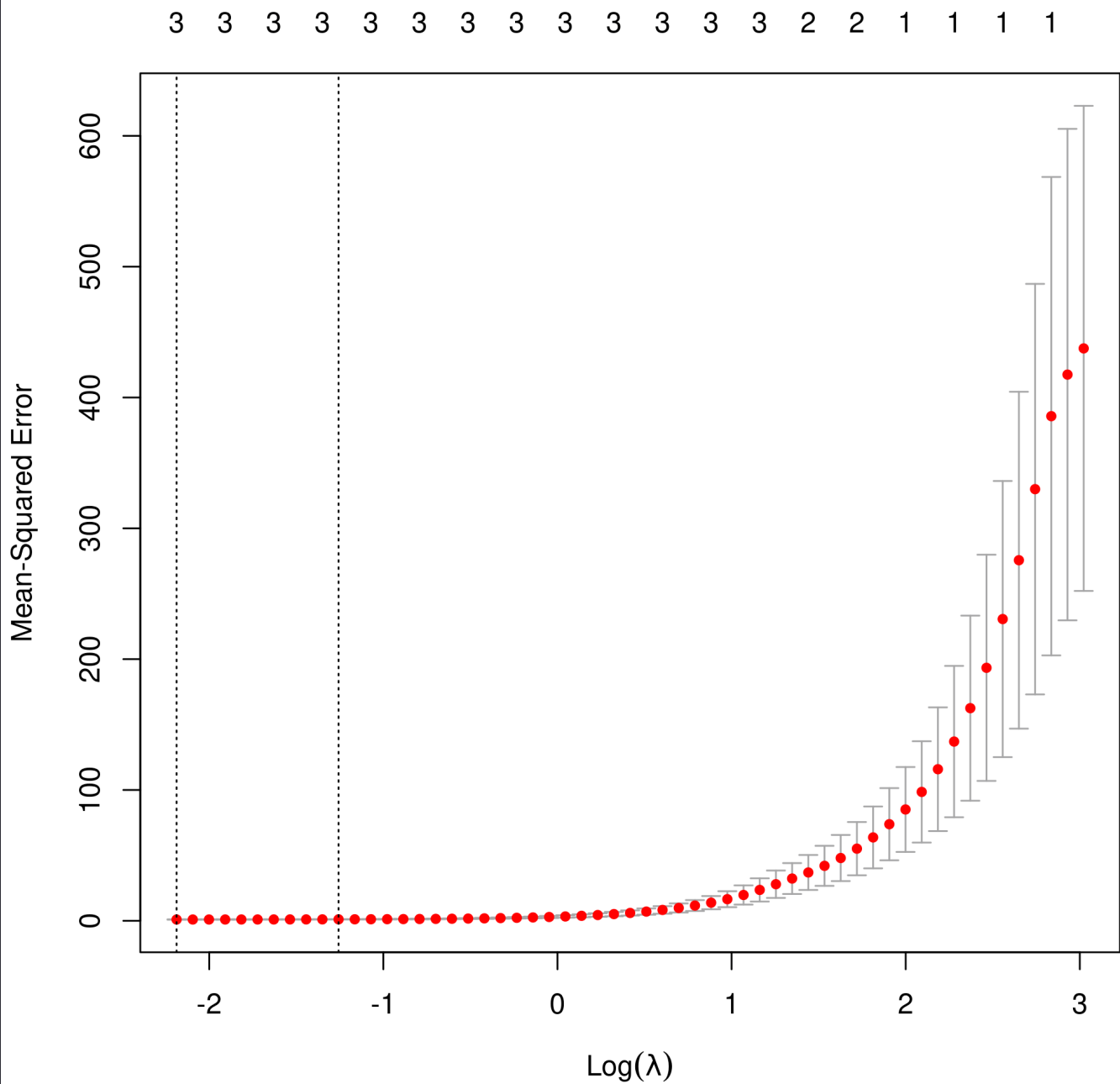
```
## Loaded glmnet 3.0-2
```

```
library(boot)
```

```
##  
## Attaching package: 'boot'
```

```
## The following object is masked from 'package:lattice':  
##  
##      melanoma
```

```
lasso.mod <- cv.glmnet(as.matrix(df[-1]), y, alpha=1)  
lambda <- lasso.mod$lambda.min  
plot(lasso.mod)
```



```
predict(lasso.mod, s=lambda, type="coefficients")
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) 42.975240
## x           5.005023
## x2          2.947540
## x3          5.989105
## x4           .
## x5           .
## x6           .
## x7           .
## x8           .
## x9           .
## x10          .
```

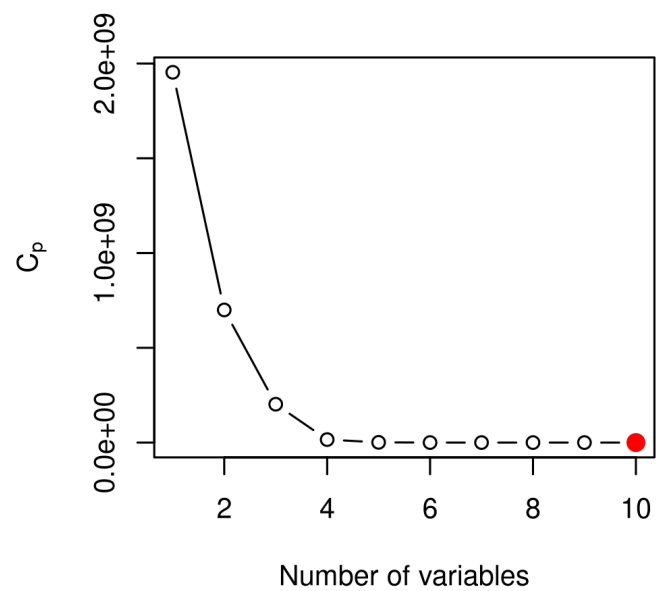
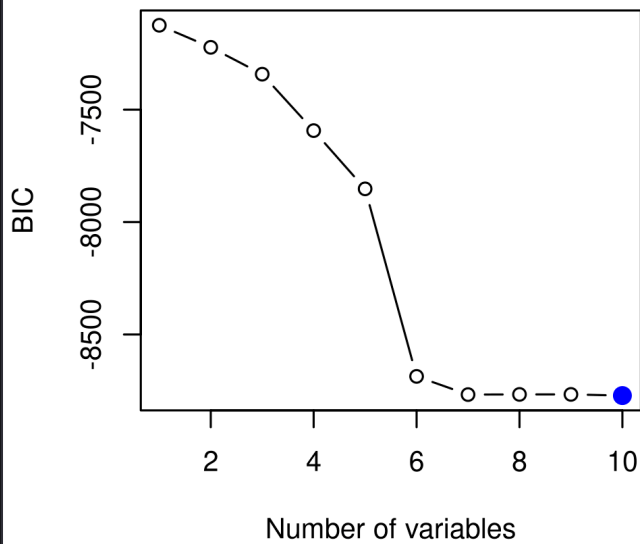
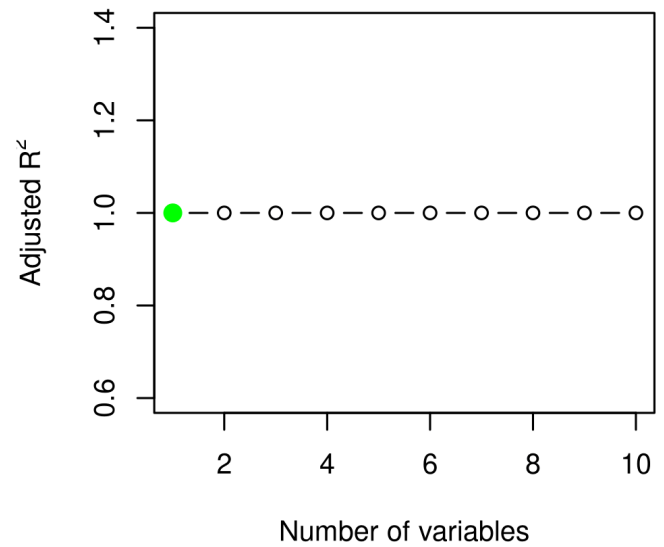
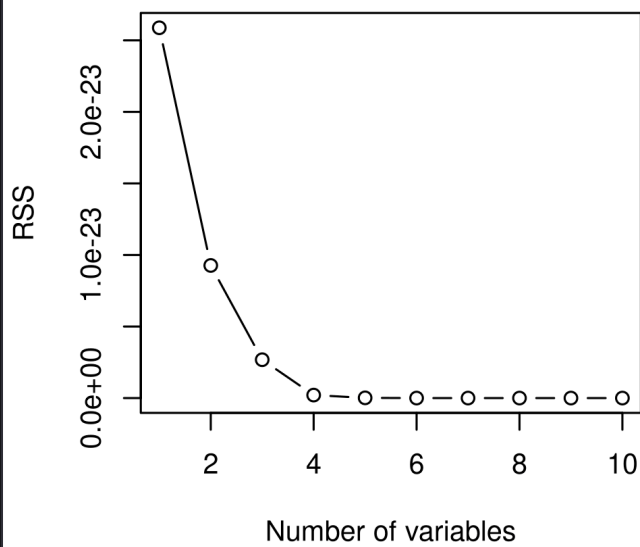
Clearly, the only important variables are x , x^2 and x^3 .

f) New model

Our new model requires a newly expanded set of betas as well.

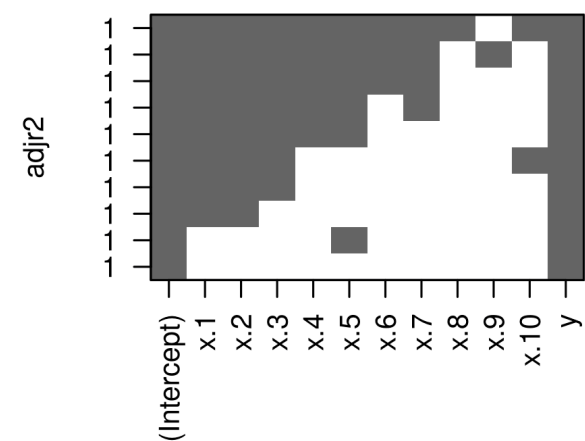
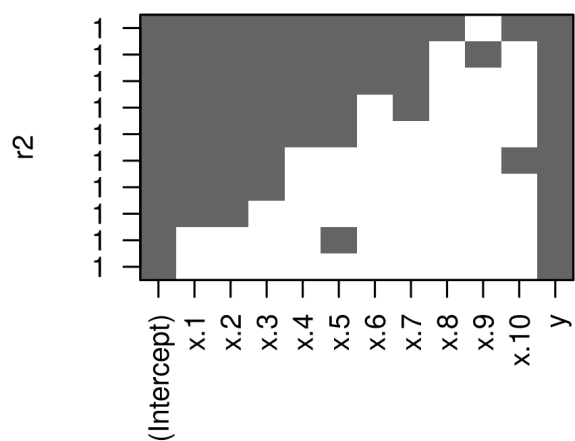
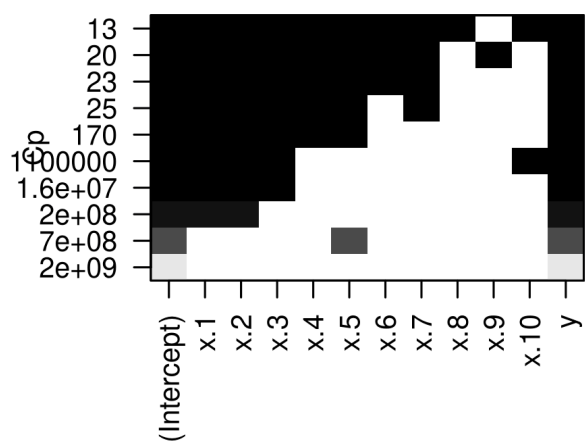
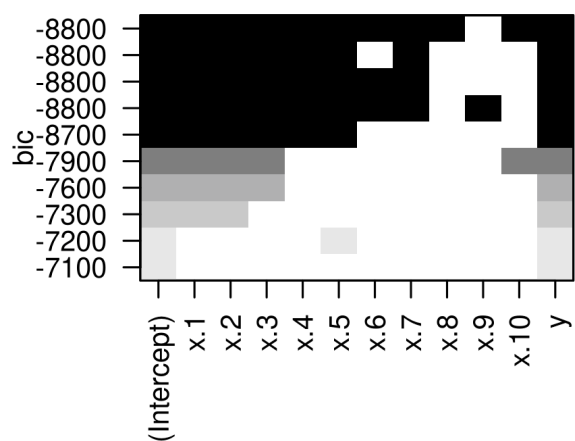
```
modelX<-poly(x,10,raw=T)
newDF<-data.frame(x=as.matrix(modelX),y=y2)
newSub<-regsubsets(y2~.,data=newDF,nvmax=10)
newSub %>% summary
```

```
## Subset selection object
## Call: regsubsets.formula(y2 ~ ., data = newDF, nvmax = 10)
## 11 Variables (and intercept)
##      Forced in Forced out
## x.1      FALSE      FALSE
## x.2      FALSE      FALSE
## x.3      FALSE      FALSE
## x.4      FALSE      FALSE
## x.5      FALSE      FALSE
## x.6      FALSE      FALSE
## x.7      FALSE      FALSE
## x.8      FALSE      FALSE
## x.9      FALSE      FALSE
## x.10     FALSE      FALSE
## y        FALSE      FALSE
## 1 subsets of each size up to 10
## Selection Algorithm: exhaustive
##      x.1 x.2 x.3 x.4 x.5 x.6 x.7 x.8 x.9 x.10 y
## 1 ( 1 ) " " " " " " " " " " " " " " " " "*"
## 2 ( 1 ) " " " " " " " "*" " " " " " " " " " "*"
## 3 ( 1 ) "*" "*" " " " " " " " " " " " " " "*"
## 4 ( 1 ) "*" "*" "*" " " " " " " " " " " " "*"
## 5 ( 1 ) "*" "*" "*" " " " " " " " " " " "*" "*"
## 6 ( 1 ) "*" "*" "*" "*" "*" " " " " " " " " "*"
## 7 ( 1 ) "*" "*" "*" "*" "*" " "*" " " " " " " "*"
## 8 ( 1 ) "*" "*" "*" "*" "*" "*" "*" " " " " " "*"
## 9 ( 1 ) "*" "*" "*" "*" "*" "*" "*" " " "*" " "*"
## 10 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*" " " "*" *
```

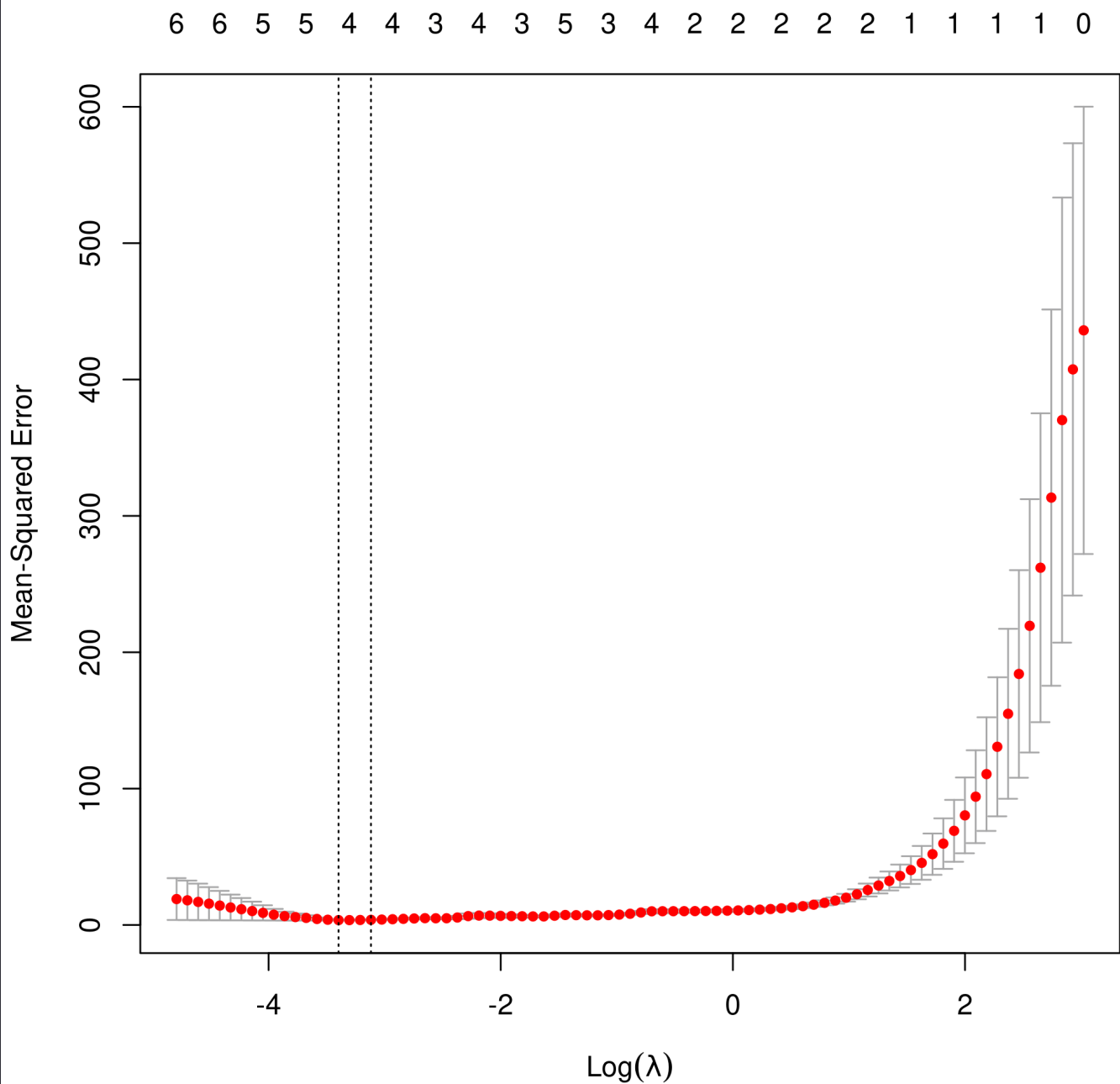


Or in its more native look,

```
par(mfrow=c(2,2))
plot(newSub)
plot(newSub,scale='Cp')
plot(newSub,scale='r2')
plot(newSub,scale='adjr2')
```



```
library(glmnet)
library(boot)
lasso.mod2 <- cv.glmnet(as.matrix(newDF[-1]), y, alpha=1)
lambda2 <- lasso.mod2$lambda.min
plot(lasso.mod2)
```



```
predict(lasso.mod2, s=lambda, type="coefficients")
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) 42.67982691
## x.2         3.22521396
## x.3         8.56699146
## x.4         .
## x.5        -0.10229572
## x.6         .
## x.7        -0.03184905
## x.8         .
## x.9         .
## x.10        .
## y          .
```

```
lambda<-10^seq(-3, 3, length = 100)
lassocaret2= train(y~.,data=newDF,method="glmnet",tuneGrid=expand.grid(alpha=1,lambda=lambda))
```



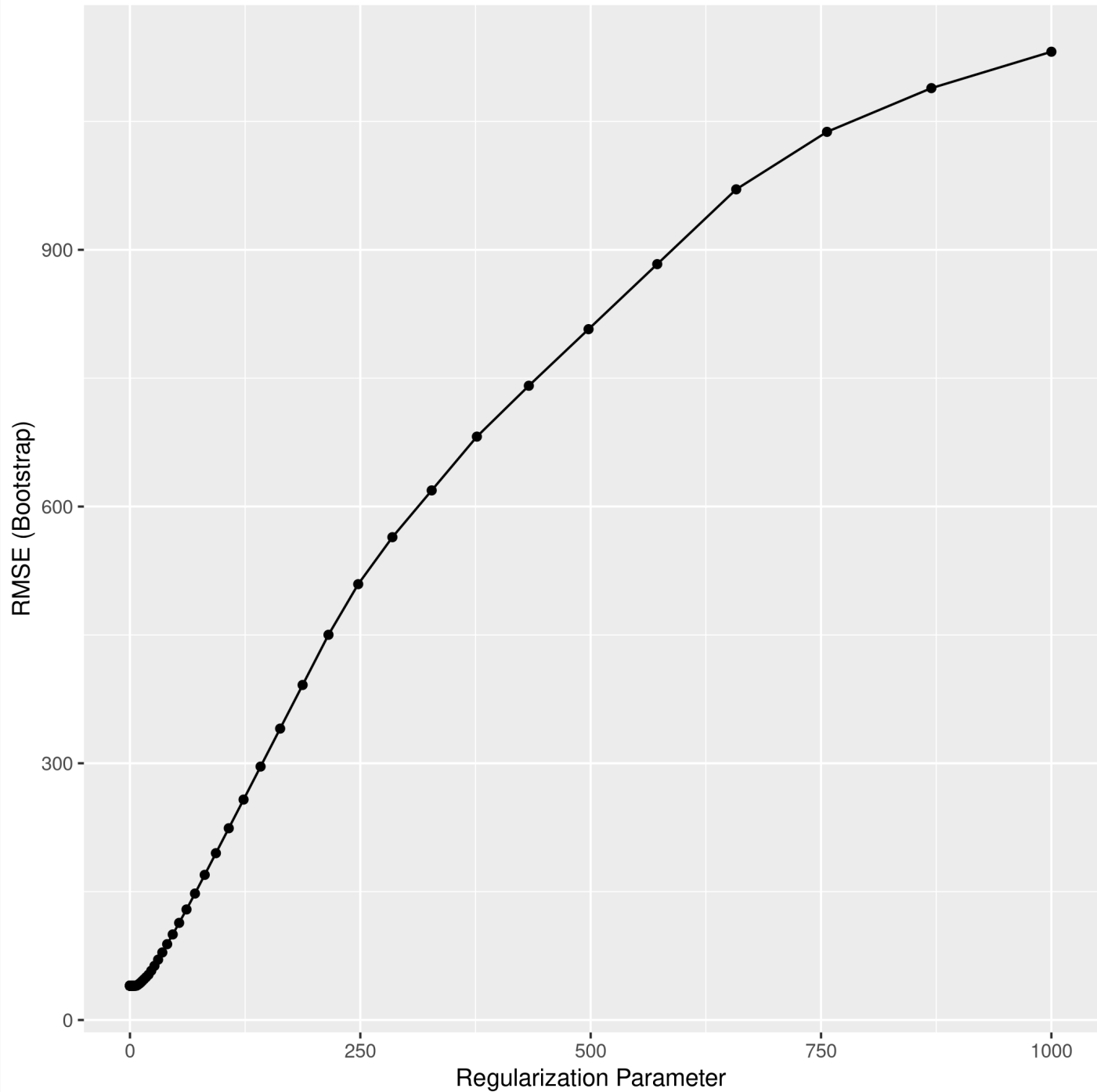
```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :  
## There were missing values in resampled performance measures.
```

```
lassocaret2 %>% print
```

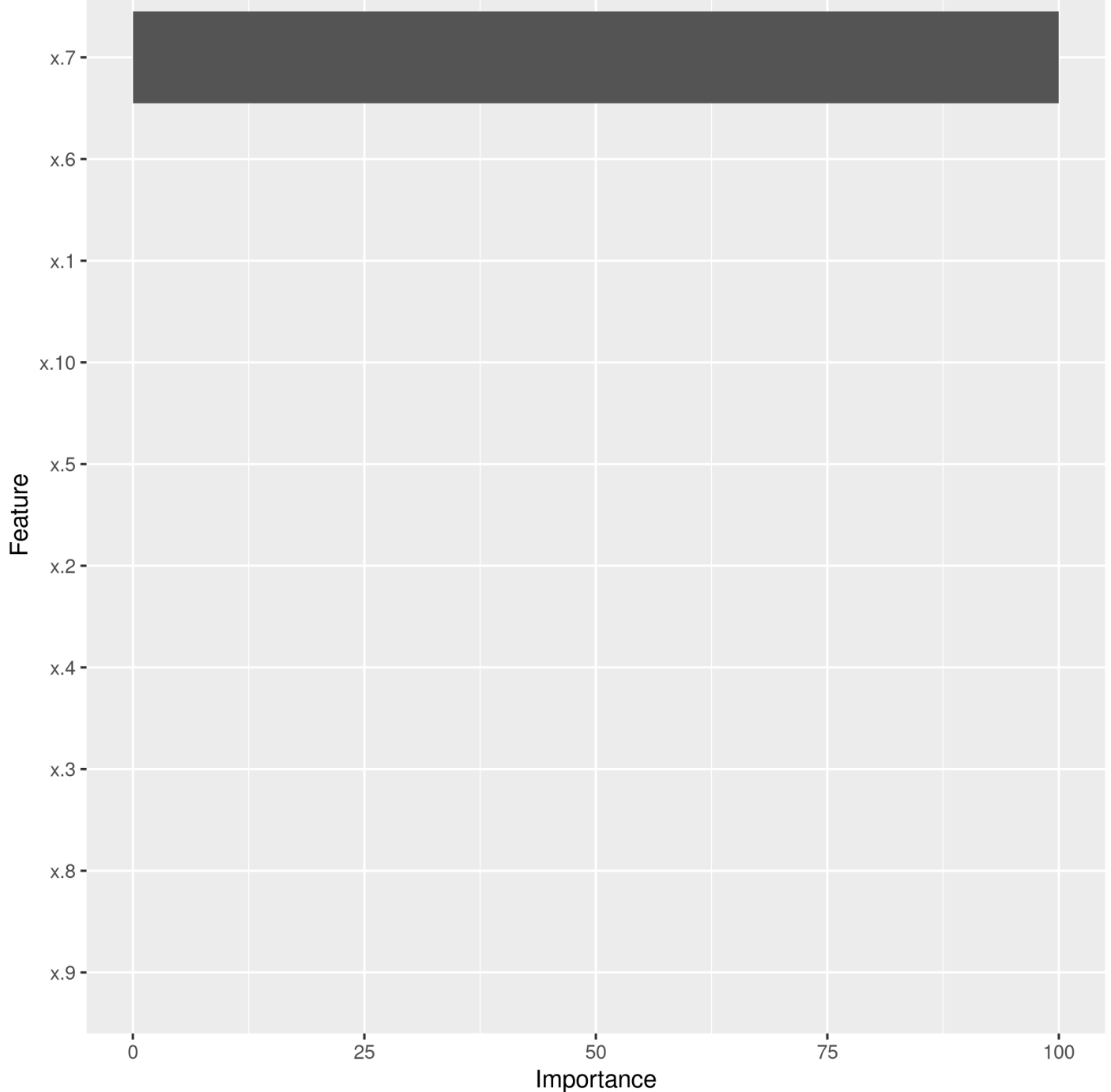
```
## glmnet
##
## 100 samples
## 10 predictor
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 100, 100, 100, 100, 100, 100, ...
## Resampling results across tuning parameters:
##
##  lambda      RMSE      Rsquared    MAE
##  1.000000e-03  40.03231  0.9999955  14.48774
##  1.149757e-03  40.03231  0.9999955  14.48774
##  1.321941e-03  40.03231  0.9999955  14.48774
##  1.519911e-03  40.03231  0.9999955  14.48774
##  1.747528e-03  40.03231  0.9999955  14.48774
##  2.009233e-03  40.03231  0.9999955  14.48774
##  2.310130e-03  40.03231  0.9999955  14.48774
##  2.656088e-03  40.03231  0.9999955  14.48774
##  3.053856e-03  40.03231  0.9999955  14.48774
##  3.511192e-03  40.03231  0.9999955  14.48774
##  4.037017e-03  40.03231  0.9999955  14.48774
##  4.641589e-03  40.03231  0.9999955  14.48774
##  5.336699e-03  40.03231  0.9999955  14.48774
##  6.135907e-03  40.03231  0.9999955  14.48774
##  7.054802e-03  40.03231  0.9999955  14.48774
##  8.111308e-03  40.03231  0.9999955  14.48774
##  9.326033e-03  40.03231  0.9999955  14.48774
##  1.072267e-02  40.03231  0.9999955  14.48774
##  1.232847e-02  40.03231  0.9999955  14.48774
##  1.417474e-02  40.03231  0.9999955  14.48774
##  1.629751e-02  40.03231  0.9999955  14.48774
##  1.873817e-02  40.03231  0.9999955  14.48774
##  2.154435e-02  40.03231  0.9999955  14.48774
##  2.477076e-02  40.03231  0.9999955  14.48774
##  2.848036e-02  40.03231  0.9999955  14.48774
##  3.274549e-02  40.03231  0.9999955  14.48774
##  3.764936e-02  40.03231  0.9999955  14.48774
##  4.328761e-02  40.03231  0.9999955  14.48774
##  4.977024e-02  40.03231  0.9999955  14.48774
##  5.722368e-02  40.03231  0.9999955  14.48774
##  6.579332e-02  40.03231  0.9999955  14.48774
##  7.564633e-02  40.03231  0.9999955  14.48774
##  8.697490e-02  40.03231  0.9999955  14.48774
##  1.000000e-01  40.03231  0.9999955  14.48774
##  1.149757e-01  40.03231  0.9999955  14.48774
##  1.321941e-01  40.03231  0.9999955  14.48774
##  1.519911e-01  40.03231  0.9999955  14.48774
##  1.747528e-01  40.03231  0.9999955  14.48774
##  2.009233e-01  40.03231  0.9999955  14.48774
##  2.310130e-01  40.03231  0.9999955  14.48774
##  2.656088e-01  40.03231  0.9999955  14.48774
##  3.053856e-01  40.03231  0.9999955  14.48774
##  3.511192e-01  40.03231  0.9999955  14.48774
##  4.037017e-01  40.03231  0.9999955  14.48774
##  4.641589e-01  40.03231  0.9999955  14.48774
##  5.336699e-01  40.03231  0.9999955  14.48774
##  6.135907e-01  40.03231  0.9999955  14.48774
##  7.054802e-01  40.03231  0.9999955  14.48774
##  8.111308e-01  40.03231  0.9999955  14.48774
##  9.326033e-01  40.03231  0.9999955  14.48774
##  1.072267e+00  40.03231  0.9999955  14.48774
##  1.232847e+00  40.03231  0.9999955  14.48774
##  1.417474e+00  40.03231  0.9999955  14.48774
##  1.629751e+00  40.03231  0.9999955  14.48774
```

```
## 1.873817e+00 40.03231 0.9999955 14.48774
## 2.154435e+00 40.03231 0.9999955 14.48774
## 2.477076e+00 40.03231 0.9999955 14.48774
## 2.848036e+00 40.03231 0.9999955 14.48774
## 3.274549e+00 40.03231 0.9999955 14.48774
## 3.764936e+00 40.03231 0.9999955 14.48774
## 4.328761e+00 40.03231 0.9999955 14.48774
## 4.977024e+00 40.03231 0.9999955 14.48774
## 5.722368e+00 40.03231 0.9999955 14.48774
## 6.579332e+00 40.03231 0.9999955 14.48774
## 7.564633e+00 40.43005 0.9999955 14.59881
## 8.697490e+00 41.25214 0.9999955 14.81913
## 1.000000e+01 42.30446 0.9999955 15.09937
## 1.149757e+01 43.59429 0.9999955 15.44307
## 1.321941e+01 45.43633 0.9999955 15.93255
## 1.519911e+01 47.55425 0.9999955 16.49605
## 1.747528e+01 49.98935 0.9999955 17.14447
## 2.009233e+01 52.90533 0.9999955 17.91650
## 2.310130e+01 57.57589 0.9999955 19.10125
## 2.656088e+01 63.25484 0.9999955 20.53147
## 3.053856e+01 70.51580 0.9999955 22.36400
## 3.511192e+01 78.93391 0.9999955 24.49105
## 4.037017e+01 88.61274 0.9999955 26.93830
## 4.641589e+01 99.97831 0.9999955 29.83601
## 5.336699e+01 113.48225 0.9999955 33.39320
## 6.135907e+01 129.17536 0.9999955 37.58303
## 7.054802e+01 147.76452 0.9999957 42.74333
## 8.111308e+01 169.60027 0.9999961 48.98043
## 9.326033e+01 194.94266 0.9999965 56.29001
## 1.072267e+02 224.07631 0.9999969 64.70026
## 1.232847e+02 257.56092 0.9999971 74.36989
## 1.417474e+02 296.13382 0.9999971 85.51504
## 1.629751e+02 340.49129 0.9999971 98.33212
## 1.873817e+02 391.49185 0.9999971 113.06864
## 2.154435e+02 450.13031 0.9999971 130.01206
## 2.477076e+02 509.28329 0.9999970 147.15405
## 2.848036e+02 564.17558 0.9999969 163.34475
## 3.274549e+02 618.84080 0.9999969 179.85589
## 3.764936e+02 681.69265 0.9999969 198.83969
## 4.328761e+02 741.14452 0.9999967 217.28049
## 4.977024e+02 807.25385 0.9999967 237.88938
## 5.722368e+02 883.26360 0.9999967 261.58461
## 6.579332e+02 970.65640 0.9999967 288.82836
## 7.564633e+02 1037.84801 0.9999960 312.54099
## 8.697490e+02 1088.92551 0.9999960 334.04769
## 1.000000e+03 1131.46176 0.9999955 354.62317
##
## Tuning parameter 'alpha' was held constant at a value of 1
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 1 and lambda = 6.579332.
```

```
lassocaret2 %>% ggplot
```



```
lassocaret2 %>% varImp %>% ggplot
```



Clearly, the LASSO model has correctly reduced the model down to the correct single variable form, though best subset seems to suggest using more predictors, their coefficients are low enough to recognize that they are noise.

Question 6.9 - Page 263

In this exercise, we will predict the number of applications received using the other variables in the `College` data set.

- (a) Split the data set into a training set and a test set.
- (b) Fit a linear model using least squares on the training set, and report the test error obtained.
- (c) Fit a ridge regression model on the training set, with λ chosen by cross-validation. Report the test error obtained.
- (d) Fit a lasso model on the training set, with λ chosen by crossvalidation. Report the test error obtained, along with the number of non-zero coefficient estimates.

(e) Fit a PCR model on the training set, with M chosen by crossvalidation. Report the test error obtained, along with the value of M selected by cross-validation.

(f) Fit a PLS model on the training set, with M chosen by crossvalidation. Report the test error obtained, along with the value of M selected by cross-validation.

(g) Comment on the results obtained. How accurately can we predict the number of college applications received? Is there much difference among the test errors resulting from these five approaches?

Answer

We will use the `caret` package, since at the moment, `mlr3` does not have learners for `PCR` and `PLS`.

```
colDat<-ISLR::College
colDat %>% summary %>% print
```

```
## Private      Apps      Accept      Enroll      Top10perc
## No :212      Min.   :   81      Min.   :   72      Min.   :   35      Min.   :   1.00
## Yes:565      1st Qu.:  776      1st Qu.:  604      1st Qu.:  242      1st Qu.:15.00
##          Median : 1558      Median : 1110      Median :  434      Median :23.00
##          Mean   : 3002      Mean   : 2019      Mean   :   780      Mean   :27.56
##          3rd Qu.: 3624      3rd Qu.: 2424      3rd Qu.:  902      3rd Qu.:35.00
##          Max.   :48094      Max.   :26330      Max.   :6392      Max.   :96.00
## Top25perc    F.Undergrad  P.Undergrad      Outstate
## Min.   :   9.0      Min.   :  139      Min.   :   1.0      Min.   : 2340
## 1st Qu.: 41.0      1st Qu.:  992      1st Qu.:  95.0      1st Qu.: 7320
## Median : 54.0      Median : 1707      Median : 353.0      Median : 9990
## Mean   : 55.8      Mean   : 3700      Mean   : 855.3      Mean   :10441
## 3rd Qu.: 69.0      3rd Qu.: 4005      3rd Qu.: 967.0      3rd Qu.:12925
## Max.   :100.0      Max.   :31643      Max.   :21836.0      Max.   :21700
## Room.Board   Books      Personal      PhD
## Min.   :1780      Min.   :  96.0      Min.   : 250      Min.   :  8.00
## 1st Qu.:3597      1st Qu.: 470.0      1st Qu.: 850      1st Qu.: 62.00
## Median :4200      Median : 500.0      Median :1200      Median : 75.00
## Mean   :4358      Mean   : 549.4      Mean   :1341      Mean   : 72.66
## 3rd Qu.:5050      3rd Qu.: 600.0      3rd Qu.:1700      3rd Qu.: 85.00
## Max.   :8124      Max.   :2340.0      Max.   :6800      Max.   :103.00
## Terminal     S.F.Ratio  perc.alumni      Expend
## Min.   : 24.0      Min.   :  2.50      Min.   :  0.00      Min.   : 3186
## 1st Qu.: 71.0      1st Qu.:11.50      1st Qu.:13.00      1st Qu.: 6751
## Median : 82.0      Median :13.60      Median :21.00      Median : 8377
## Mean   : 79.7      Mean   :14.09      Mean   :22.74      Mean   : 9660
## 3rd Qu.: 92.0      3rd Qu.:16.50      3rd Qu.:31.00      3rd Qu.:10830
## Max.   :100.0      Max.   :39.80      Max.   :64.00      Max.   :56233
## Grad.Rate
## Min.   : 10.00
## 1st Qu.: 53.00
## Median : 65.00
## Mean   : 65.46
## 3rd Qu.: 78.00
## Max.   :118.00
```

```
colDat %>% str %>% print
```

```
## 'data.frame':   777 obs. of  18 variables:
## $ Private      : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 2 2 2 2 ...
## $ Apps         : num  1660 2186 1428 417 193 ...
## $ Accept       : num  1232 1924 1097 349 146 ...
## $ Enroll       : num   721 512 336 137 55 158 103 489 227 172 ...
## $ Top10perc    : num   23 16 22 60 16 38 17 37 30 21 ...
## $ Top25perc    : num   52 29 50 89 44 62 45 68 63 44 ...
## $ F.Undergrad  : num  2885 2683 1036 510 249 ...
## $ P.Undergrad  : num   537 1227 99 63 869 ...
## $ Outstate     : num  7440 12280 11250 12960 7560 ...
## $ Room.Board   : num  3300 6450 3750 5450 4120 ...
## $ Books        : num   450 750 400 450 800 500 500 450 300 660 ...
## $ Personal     : num  2200 1500 1165 875 1500 ...
## $ PhD          : num   70 29 53 92 76 67 90 89 79 40 ...
## $ Terminal     : num   78 30 66 97 72 73 93 100 84 41 ...
## $ S.F.Ratio    : num  18.1 12.2 12.9 7.7 11.9 9.4 11.5 13.7 11.3 11.5 ...
## $ perc.alumni  : num   12 16 30 37 2 11 26 37 23 15 ...
## $ Expend       : num  7041 10527 8735 19016 10922 ...
## $ Grad.Rate    : num   60 56 54 59 15 55 63 73 80 52 ...
## NULL
```

```
colDat %>% supply(unique) %>% supply(length) %>% print
```

##	Private	Apps	Accept	Enroll	Top10perc	Top25perc
##	2	711	693	581	82	89
##	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal
##	714	566	640	553	122	294
##	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
##	78	65	173	61	744	81

Clearly, there are no psuedo-factors which might have been converted at this stage.

a) Train-Test split

```
train_ind<-createDataPartition(colDat$Apps,p=0.8,times=1,list=FALSE)
train_set<-colDat[train_ind,]
test_set<-colDat[-train_ind,]
```

b) Linear least squares

```
linCol<-train(Apps~.,data=train_set,method="lm")
linCol %>% summary
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5145.6  -414.8   -20.3   340.5  7526.8
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.918e+02  4.506e+02  -0.648  0.517486
## PrivateYes  -5.351e+02  1.532e+02  -3.494  0.000511 ***
## Accept       1.617e+00  4.258e-02  37.983   < 2e-16 ***
## Enroll      -1.012e+00  1.959e-01  -5.165  3.26e-07 ***
## Top10perc    5.379e+01  6.221e+00   8.647   < 2e-16 ***
## Top25perc   -1.632e+01  5.046e+00  -3.235  0.001282 **
## F.Undergrad  6.836e-02  3.457e-02   1.978  0.048410 *
## P.Undergrad  7.929e-02  3.367e-02   2.355  0.018854 *
## Outstate    -7.303e-02  2.098e-02  -3.481  0.000536 ***
## Room.Board   1.695e-01  5.367e-02   3.159  0.001663 **
## Books        9.998e-02  2.578e-01   0.388  0.698328
## Personal    -3.145e-03  6.880e-02  -0.046  0.963553
## PhD        -8.926e+00  5.041e+00  -1.771  0.077112 .
## Terminal    -2.298e+00  5.608e+00  -0.410  0.682152
## S.F.Ratio    6.038e+00  1.420e+01   0.425  0.670757
## perc.alumni -5.085e-01  4.560e+00  -0.112  0.911249
## Expend       4.668e-02  1.332e-02   3.505  0.000490 ***
## Grad.Rate    9.042e+00  3.379e+00   2.676  0.007653 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1042 on 606 degrees of freedom
## Multiple R-squared:  0.9332, Adjusted R-squared:  0.9313
## F-statistic: 497.7 on 17 and 606 DF,  p-value: < 2.2e-16
```

```
linPred<-predict(linCol,test_set)
linPred %>% postResample(obs = test_set$Apps)
```

```
##           RMSE      Rsquared      MAE
## 1071.6360025    0.9017032  625.7827996
```

Do note that the [metrics are calculated](#) in a manner to ensure no negative values are obtained.

c) Ridge regression with CV for λ

```
L2Grid <- expand.grid(alpha=0,
                      lambda=10^seq(from=-3,to=30,length=100))
```

```
ridgCol<-train(Apps~.,data=train_set,method="glmnet",tuneGrid = L2Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

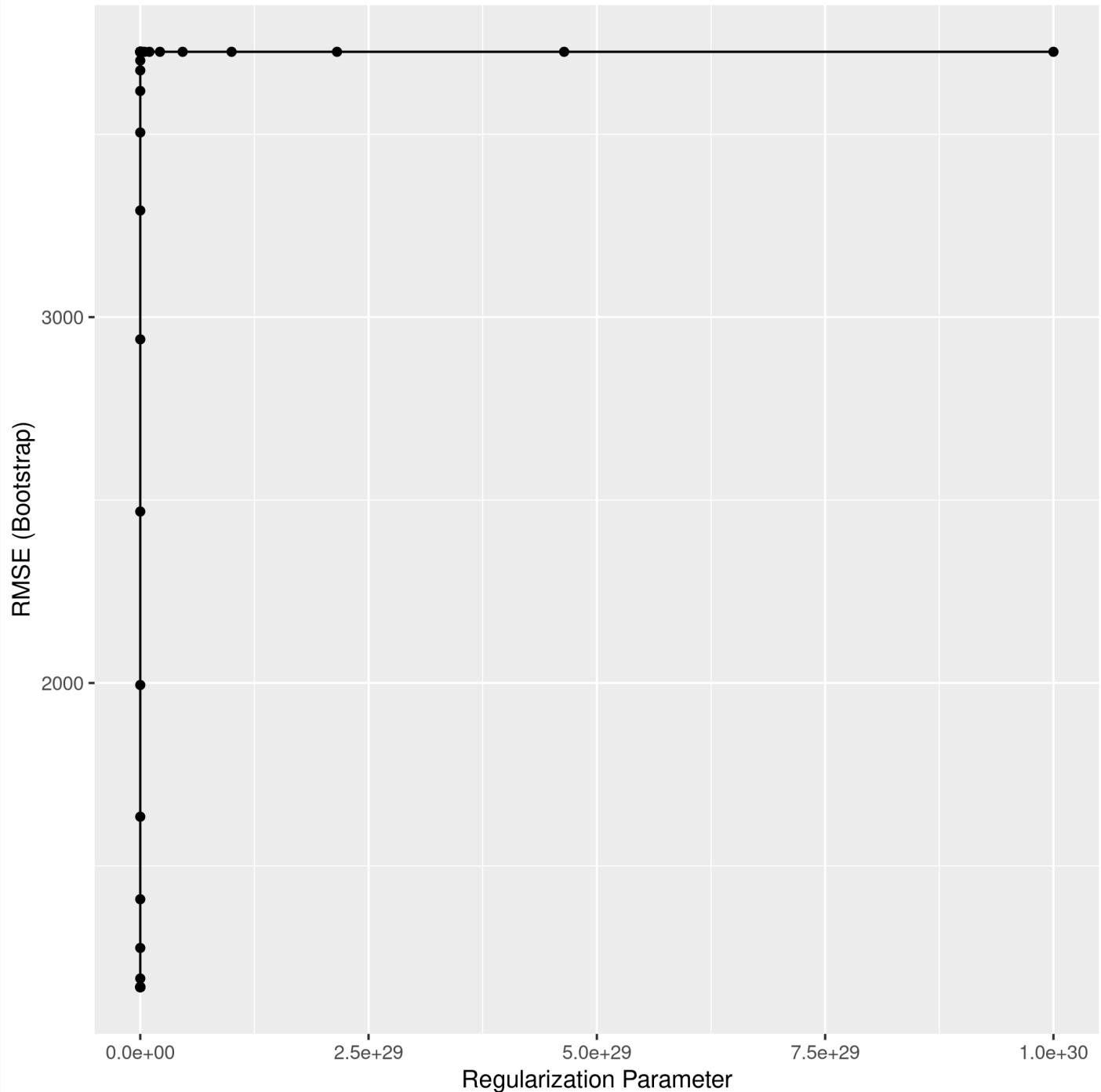
```
ridgCol %>% summary %>% print
```


##	Length	Class	Mode
## a0	100	-none-	numeric
## beta	1700	dgCMatrix	S4
## df	100	-none-	numeric
## dim	2	-none-	numeric
## lambda	100	-none-	numeric
## dev.ratio	100	-none-	numeric
## nulldev	1	-none-	numeric
## npasses	1	-none-	numeric
## jerr	1	-none-	numeric
## offset	1	-none-	logical
## call	5	-none-	call
## nobs	1	-none-	numeric
## lambdaOpt	1	-none-	numeric
## xNames	17	-none-	character
## problemType	1	-none-	character
## tuneValue	2	data.frame	list
## obsLevels	1	-none-	logical
## param	0	-none-	list

```
coef(ridgCol$finalModel, ridgCol$bestTune$lambda) %>% print
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) -1.407775e+03
## PrivateYes  -5.854245e+02
## Accept      1.042778e+00
## Enroll      3.511219e-01
## Top10perc   2.780211e+01
## Top25perc   2.883536e-02
## F.Undergrad 6.825141e-02
## P.Undergrad 5.281320e-02
## Outstate   -2.011504e-02
## Room.Board  2.155224e-01
## Books       1.517585e-01
## Personal    -3.711406e-02
## PhD         -4.453155e+00
## Terminal    -3.783231e+00
## S.F.Ratio    6.897360e+00
## perc.alumni -9.301831e+00
## Expend       5.601144e-02
## Grad.Rate    1.259989e+01
```

```
ggplot(ridgCol)
```



```
ridgPred<-predict(ridgCol,test_set)
ridgPred %>% postResample(obs = test_set$Apps)
```

```
##      RMSE      Rsquared      MAE
## 1047.7545250  0.9051726  644.4535063
```

d) LASSO with CV for λ

```
L1Grid <- expand.grid(alpha=1, # for lasso
                     lambda=10^seq(from=-3,to=30,length=100))
```

```
lassoCol<-train(Apps~.,data=train_set,method="glmnet",tuneGrid = L1Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

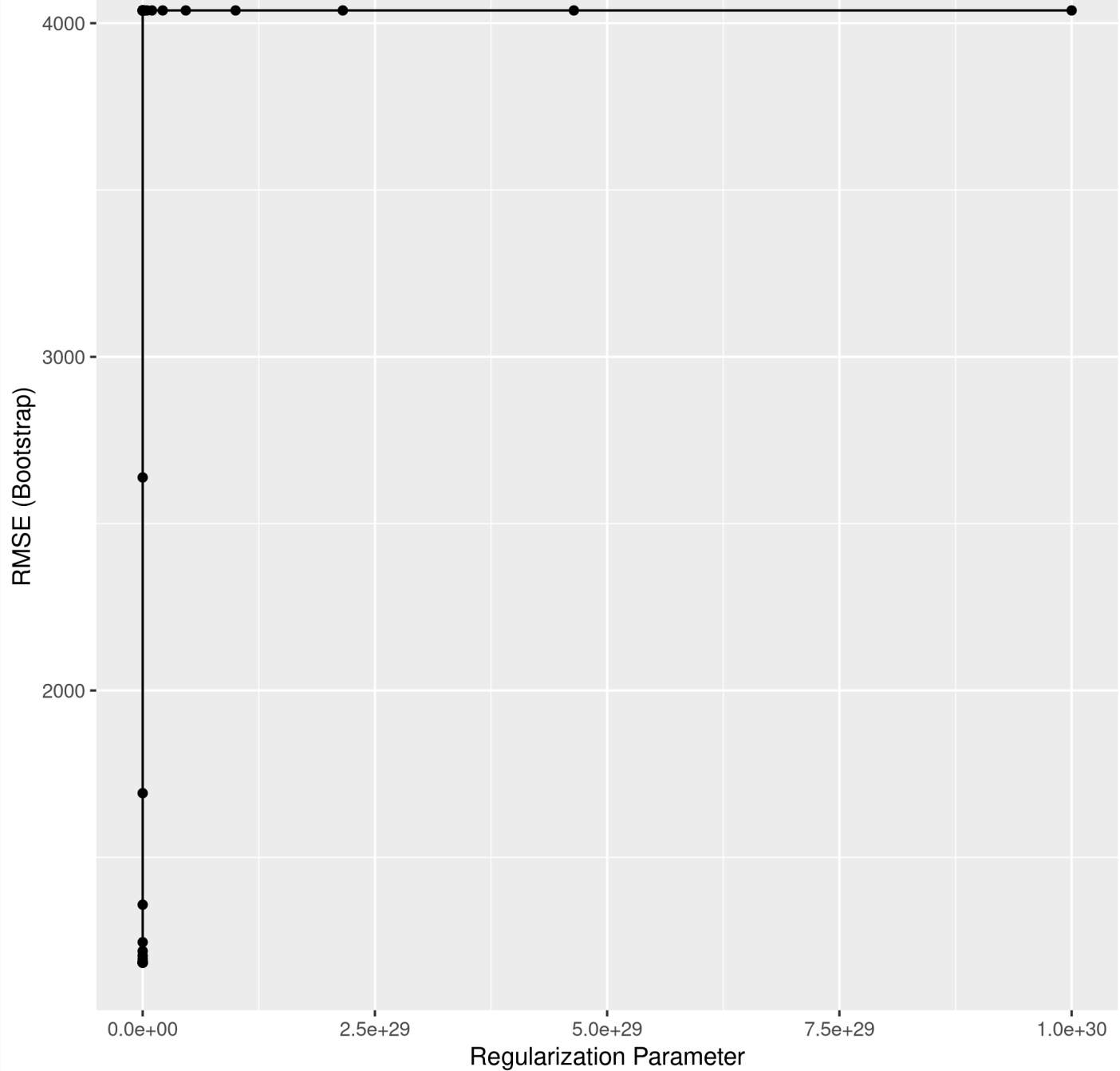
```
lassoCol %>% summary %>% print
```

```
##           Length Class      Mode
## a0           81  -none-   numeric
## beta        1377 dgCMatrix S4
## df           81  -none-   numeric
## dim           2  -none-   numeric
## lambda        81  -none-   numeric
## dev.ratio     81  -none-   numeric
## nulldev        1  -none-   numeric
## npasses        1  -none-   numeric
## jerr           1  -none-   numeric
## offset         1  -none-   logical
## call           5  -none-    call
## nob           1  -none-   numeric
## lambdaOpt       1  -none-   numeric
## xNames         17  -none-   character
## problemType     1  -none-   character
## tuneValue        2 data.frame list
## obsLevels        1  -none-   logical
## param           0  -none-    list
```

```
coef(lassoCol$finalModel, lassoCol$bestTune$lambda) %>% print
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) -325.51554340
## PrivateYes  -532.28956305
## Accept       1.60370798
## Enroll       -0.90158328
## Top10perc    51.96610325
## Top25perc   -14.87886847
## F.Undergrad  0.05352324
## P.Undergrad  0.07832395
## Outstate    -0.07047302
## Room.Board   0.16783269
## Books        0.08836704
## Personal     .
## PhD         -8.67634519
## Terminal    -2.18494018
## S.F.Ratio    5.25050018
## perc.alumni -0.67848535
## Expend       0.04597728
## Grad.Rate    8.67569015
```

```
ggplot(lassoCol)
```



```
lassoPred<-predict(lassoCol,test_set)
lassoPred %>% postResample(obs = test_set$Apps)
```

```
##      RMSE      Rsquared      MAE
## 1068.9834769  0.9021268  622.7029418
```

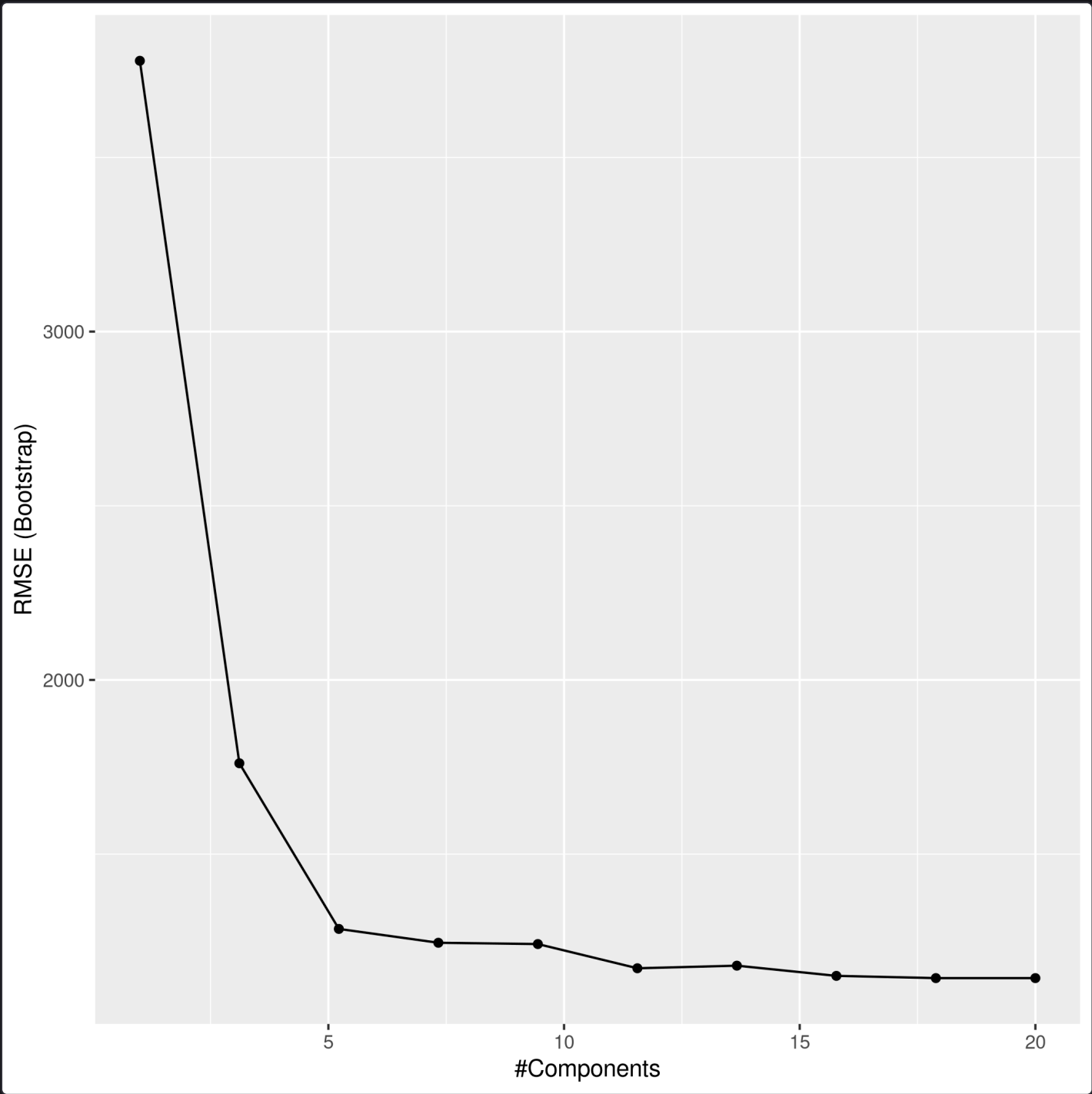
e) PCR with CV for M

```
mGrid <- expand.grid(ncomp=seq(from=1,to=20,length=10))
```

```
pcrCol<-train(Apps~.,data=train_set,method="pcr",tuneGrid = mGrid)
pcrCol %>% summary %>% print
```

```
## Data:      X dimension: 624 17
## Y dimension: 624 1
## Fit method: svdpc
## Number of components considered: 17
## TRAINING: % variance explained
##           1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X          48.2314   87.24   95.02   97.26   98.63   99.43   99.91
## .outcome    0.2419   76.54   77.88   80.19   91.27   91.34   91.34
##           8 comps  9 comps 10 comps 11 comps 12 comps 13 comps 14 comps
## X          99.96  100.00  100.00  100.00  100.00  100.00  100.00
## .outcome    91.65   91.66   92.26   92.65   92.66   92.67   92.76
##          15 comps 16 comps 17 comps
## X          100.00  100.00  100.00
## .outcome    93.17   93.18   93.32
## NULL
```

```
ggplot(pcrCol)
```



```
pcrPred<-predict(pcrCol,test_set)
pcrPred %>% postResample(obs = test_set$Apps)
```

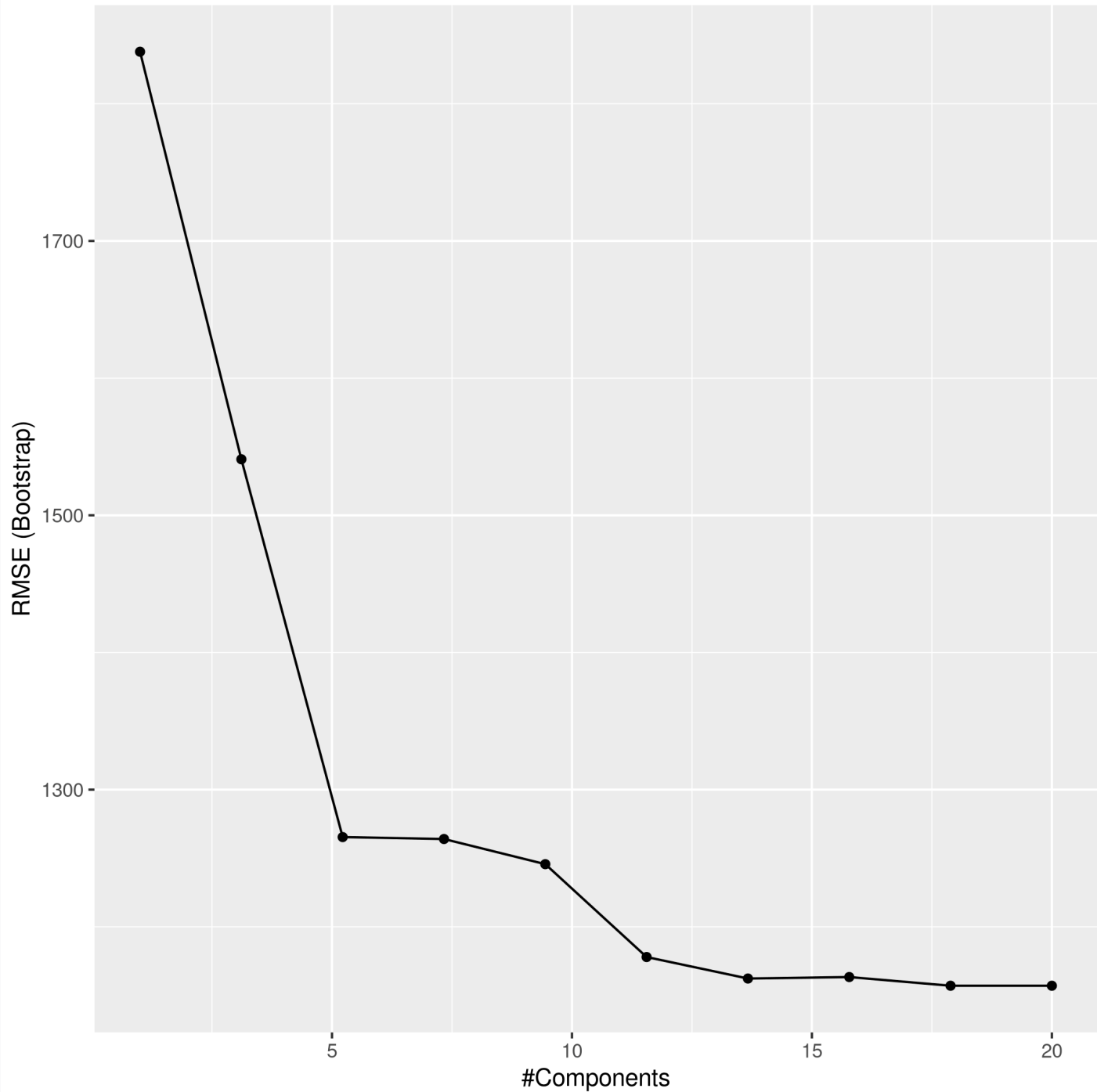
```
##           RMSE      Rsquared      MAE
## 1071.6360025    0.9017032  625.7827996
```

f) PLS with CV for M

```
plsCol<-train(Apps~.,data=train_set,method="pls",tuneGrid = mGrid)
plsCol %>% summary %>% print
```

```
## Data:      X dimension: 624 17
## Y dimension: 624 1
## Fit method: oscorespls
## Number of components considered: 17
## TRAINING: % variance explained
##           1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X           39.02    56.4    91.83    96.61    98.62    99.22    99.49
## .outcome     78.04    84.1    86.88    91.09    91.38    91.49    91.66
##           8 comps  9 comps 10 comps 11 comps 12 comps 13 comps 14 comps
## X           99.96    99.99   100.00   100.00   100.00   100.00   100.00
## .outcome     91.68    91.85    92.64    92.87    93.16    93.18    93.18
##          15 comps 16 comps 17 comps
## X           100.00   100.00   100.00
## .outcome     93.18    93.19    93.32
## NULL
```

```
ggplot(plsCol)
```



```
plsPred<-predict(plsCol,test_set)
plsPred %>% postResample(obs = test_set$Apps)
```

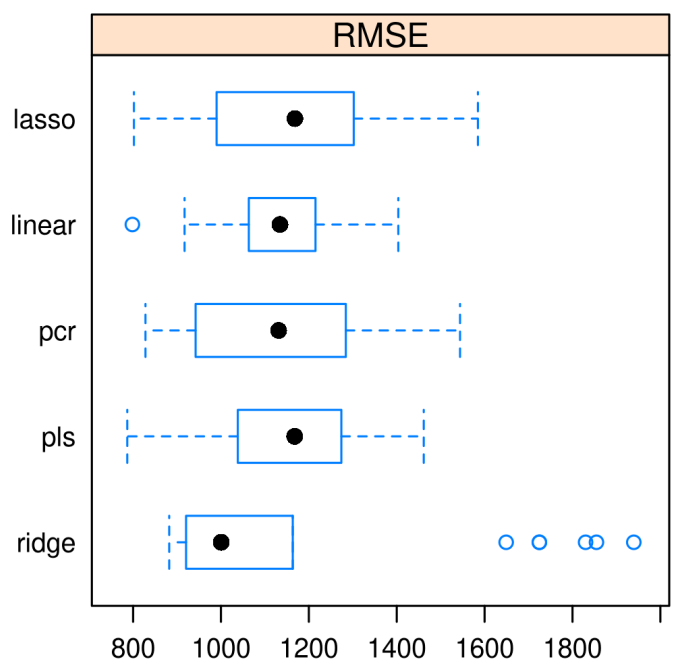
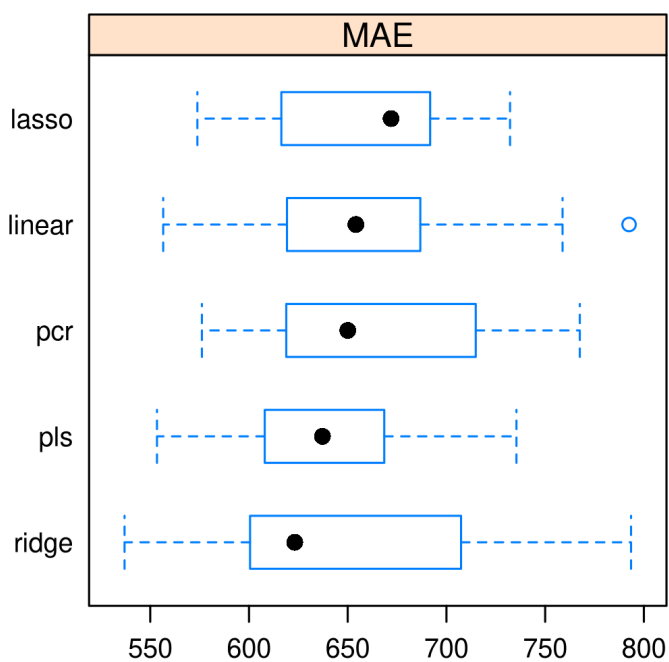
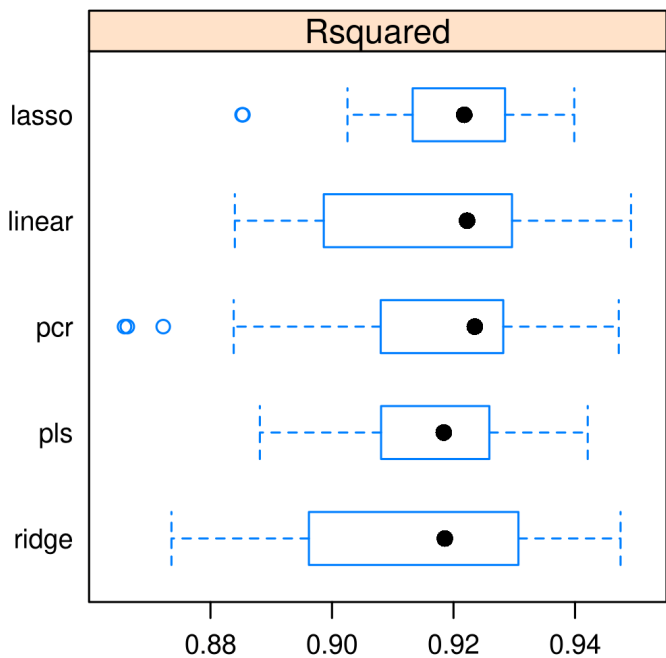
##	RMSE	Rsquared	MAE
##	1071.6360039	0.9017032	625.7827987

g) Comments and Comparison

```
models <- list(ridge = ridgCol, lasso = lassoCol, pcr = pcrCol, pls=plsCol,linear=linCol)
resamples(models) %>% summary
```

```
##
## Call:
## summary.resamples(object = .)
##
## Models: ridge, lasso, pcr, pls, linear
## Number of resamples: 25
##
## MAE
##           Min.   1st Qu.   Median     Mean   3rd Qu.     Max. NA's
## ridge  536.9612 600.5398 623.2005 649.6713 707.4014 793.4972    0
## lasso  573.8563 616.3883 671.9453 655.8858 691.7620 732.2155    0
## pcr    576.1427 618.8694 650.0360 662.9040 714.8491 767.5535    0
## pls    553.3999 607.9757 637.1985 638.6619 668.5120 735.4479    0
## linear 556.5553 619.2395 654.1478 659.4635 686.7747 792.4912    0
##
## RMSE
##           Min.   1st Qu.   Median     Mean   3rd Qu.     Max. NA's
## ridge  882.2646 920.5934 1000.519 1168.603 1163.377 1939.541    0
## lasso  801.9415 990.0724 1168.234 1184.329 1302.221 1584.712    0
## pcr    828.1370 942.2678 1131.207 1144.071 1284.178 1544.078    0
## pls    786.7989 1038.3265 1167.764 1157.026 1274.041 1461.434    0
## linear 798.3771 1063.3690 1134.291 1135.977 1215.115 1403.576    0
##
## Rsquared
##           Min.   1st Qu.   Median     Mean   3rd Qu.     Max. NA's
## ridge  0.8735756 0.8962010 0.9185736 0.9136429 0.9306819 0.9474913    0
## lasso  0.8851991 0.9132766 0.9217660 0.9191638 0.9284838 0.9398772    0
## pcr    0.8658504 0.9080179 0.9235117 0.9146884 0.9281892 0.9471991    0
## pls    0.8881249 0.9080786 0.9183968 0.9173632 0.9258994 0.9420894    0
## linear 0.8840049 0.8986452 0.9222319 0.9160913 0.9296275 0.9492198    0
```

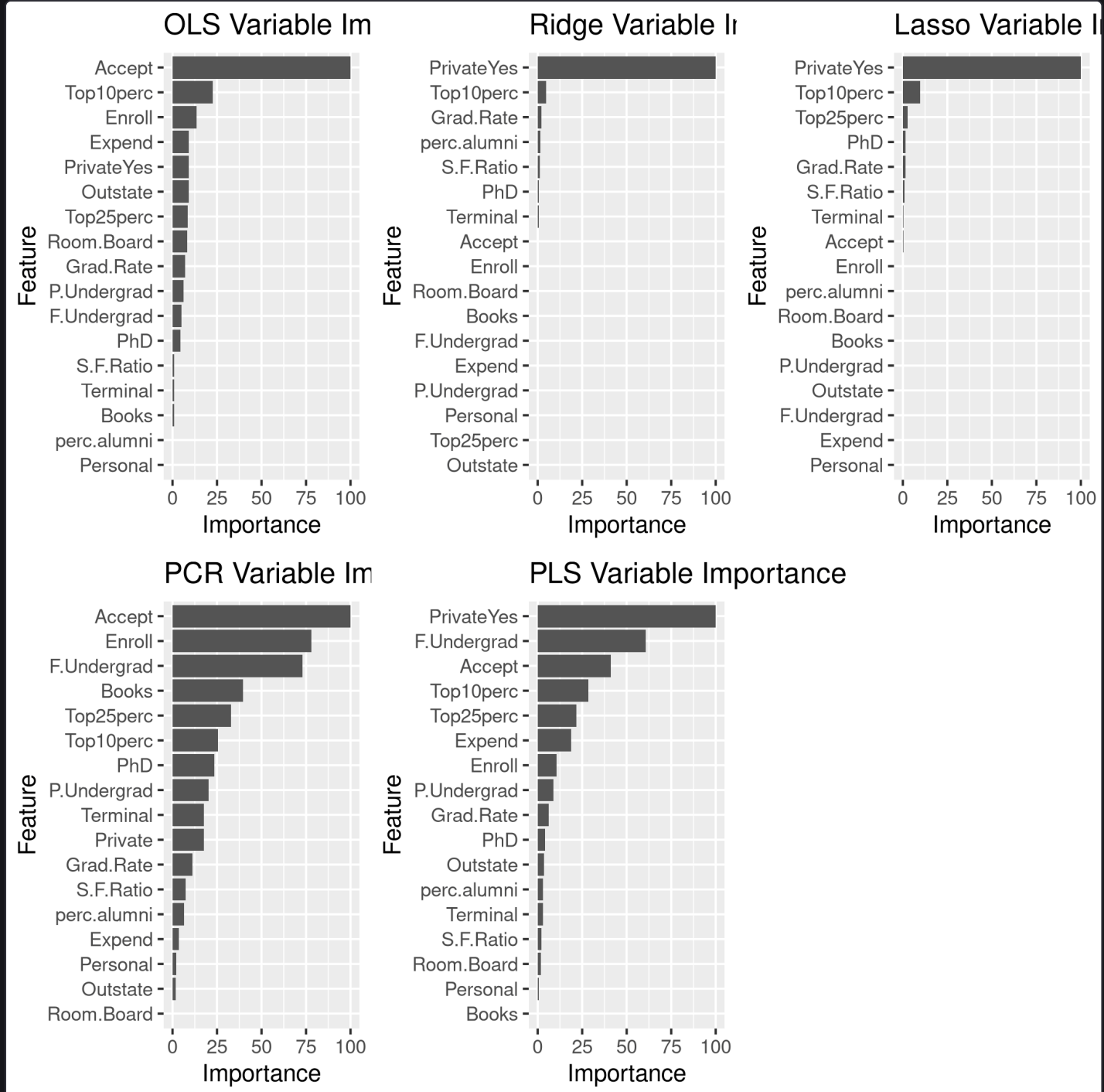
```
resamples(models) %>% bwplot(scales="free")
```

- Given the tighter spread of **PLS**, it seems more reliable than **PCR**
- **Ridge** is just poor in every way
- **OLS** does well, but it also has a worrying outlier
- **LASSO** appears to be doing alright as well We also have kept track of the performance on the **test_set**

We might want to see the variable significance values as well.

```
lgp<-linCol %>% varImp %>% ggplot + ggtitle("OLS Variable Importance")
rgp<-ridgCol %>% varImp %>% ggplot + ggtitle("Ridge Variable Importance")
lsgp<-lassoCol %>% varImp %>% ggplot + ggtitle("Lasso Variable Importance")
pcgp<-pcrCol %>% varImp %>% ggplot + ggtitle("PCR Variable Importance")
plgp<-plsCol %>% varImp %>% ggplot + ggtitle("PLS Variable Importance")
grid.arrange(lgp,rgp,lsgp,pcgp,plgp,ncol=3)
```



Question 6.10 - Pages 263-264

We have seen that as the number of features used in a model increases, the training error will necessarily decrease, but the test error may not. We will now explore this in a simulated data set.

(a) Generate a data set with $p = 20$ features, $n = 1,000$ observations, and an associated quantitative response vector generated according to the model

$$Y = X\beta + \eta,$$

where β has some elements that are exactly equal to zero.

(b) Split your data set into a training set containing 100 observations and a test set containing 900 observations.

(c) Perform best subset selection on the training set, and plot the training set MSE associated with the best model of each size.

(d) Plot the test set MSE associated with the best model of each size.

(e) For which model size does the test set MSE take on its minimum value? Comment on your results. If it takes on its minimum value for a model containing only an intercept or a model containing all of the features, then play around with the way that you are generating the data in (a) until you come up with a scenario in which the test set MSE is minimized for an intermediate model size.

(f) How does the model at which the test set MSE is minimized compare to the true model used to generate the data? Comment on the coefficient values.

(g) Create a plot displaying $\sqrt{\sum_{j=1}^p (\beta_j - \hat{\beta}_j^r)^2}$ for a range of values of r , where $\hat{\beta}_j^r$ is the j th coefficient estimate for the best model containing r coefficients. Comment on what you observe. How does this compare to the test MSE plot from (d)?

Answer

Model creation

```
p=20
n=1000
noise<-rnorm(n)
xmat<-matrix(rnorm(n*p),nrow=n,ncol=p)
beta<-sample(-10:34,20)
beta[sample(1:20,4)]=0
myY<-xmat %*% beta + noise
modelDat<-data.frame(x=as.matrix(xmat),y=myY)
```

- As always we will want to take a peak

```
modelDat %>% str %>% print
```

```
## 'data.frame': 1000 obs. of 21 variables:
## $ x.1 : num -0.406 -1.375 0.858 -0.231 -0.601 ...
## $ x.2 : num -0.129 -0.218 -0.17 0.573 -0.513 ...
## $ x.3 : num 0.127 -0.224 1.014 0.896 0.159 ...
## $ x.4 : num 0.499 -0.151 -0.488 -0.959 2.187 ...
## $ x.5 : num -0.235 -0.345 -0.773 -0.346 0.773 ...
## $ x.6 : num 0.26 -0.429 -1.183 -1.159 0.959 ...
## $ x.7 : num 0.567 1.647 0.149 -0.593 -0.902 ...
## $ x.8 : num -0.092 0.8391 -1.4835 0.0229 -0.1353 ...
## $ x.9 : num -0.998 -1.043 -0.563 -0.377 0.324 ...
## $ x.10: num -0.4401 -0.195 -0.5139 -0.0156 -0.9543 ...
## $ x.11: num -0.147 0.829 0.165 0.101 -0.105 ...
## $ x.12: num -0.0118 1.02 1.0794 1.3184 -2.2844 ...
## $ x.13: num -1.683 0.487 -1.142 -0.744 -0.175 ...
## $ x.14: num 0.228 -1.031 -2.798 -0.646 0.56 ...
## $ x.15: num -0.718 0.508 0.637 -0.556 0.585 ...
## $ x.16: num -1.6378 0.581 -0.9939 0.0537 -0.5854 ...
## $ x.17: num 1.758 -0.616 1.377 -0.876 -1.174 ...
## $ x.18: num -1.438 0.373 1.364 0.399 0.949 ...
## $ x.19: num -0.715 -0.731 1.142 0.149 0.916 ...
## $ x.20: num 2.774 -2.024 1.316 0.138 0.187 ...
## $ y : num 77.5 -82.8 -38.9 -79.7 64.9 ...
## NULL
```

```
modelDat %>% summary %>% print
```

```
##      x.1      x.2      x.3      x.4
## Min.   :-2.79766 Min.   :-3.13281 Min.   :-2.71232 Min.   :-4.29604
## 1st Qu.: -0.60516 1st Qu.: -0.66759 1st Qu.: -0.60561 1st Qu.: -0.66598
## Median :  0.04323 Median :  0.03681 Median :  0.06556 Median :  0.06589
## Mean   :  0.06879 Mean   :  0.01004 Mean   :  0.06443 Mean   :  0.02244
## 3rd Qu.:  0.74049 3rd Qu.:  0.68234 3rd Qu.:  0.70521 3rd Qu.:  0.71174
## Max.   :  3.50354 Max.   :  3.47268 Max.   :  3.02817 Max.   :  3.27326
##      x.5      x.6      x.7      x.8
## Min.   :-3.228376 Min.   :-4.24014 Min.   :-2.98577 Min.   :-3.27770
## 1st Qu.: -0.698220 1st Qu.: -0.69448 1st Qu.: -0.59092 1st Qu.: -0.52939
## Median : -0.058778 Median : -0.01141 Median :  0.01732 Median :  0.05703
## Mean   :  0.000126 Mean   : -0.05158 Mean   :  0.04767 Mean   :  0.08231
## 3rd Qu.:  0.663570 3rd Qu.:  0.64217 3rd Qu.:  0.67438 3rd Qu.:  0.72849
## Max.   :  3.036307 Max.   :  3.27572 Max.   :  2.72163 Max.   :  3.33409
##      x.9      x.10     x.11     x.12
## Min.   :-3.08957 Min.   :-3.21268 Min.   :-3.00572 Min.   :-3.72016
## 1st Qu.: -0.65456 1st Qu.: -0.69401 1st Qu.: -0.68226 1st Qu.: -0.63043
## Median : -0.04242 Median : -0.03069 Median : -0.04777 Median :  0.07079
## Mean   :  0.02049 Mean   : -0.02400 Mean   : -0.03729 Mean   :  0.03769
## 3rd Qu.:  0.71209 3rd Qu.:  0.61540 3rd Qu.:  0.64873 3rd Qu.:  0.67155
## Max.   :  3.23110 Max.   :  2.76059 Max.   :  2.87306 Max.   :  3.48569
##      x.13     x.14     x.15     x.16
## Min.   :-3.20126 Min.   :-3.55432 Min.   :-2.857575 Min.   :-3.5383
## 1st Qu.: -0.68535 1st Qu.: -0.66752 1st Qu.: -0.658708 1st Qu.: -0.7813
## Median : -0.01329 Median : -0.03302 Median :  0.020581 Median : -0.0740
## Mean   :  0.01094 Mean   :  0.02113 Mean   :  0.007976 Mean   : -0.0883
## 3rd Qu.:  0.64877 3rd Qu.:  0.74919 3rd Qu.:  0.670464 3rd Qu.:  0.5568
## Max.   :  2.78973 Max.   :  3.47923 Max.   :  2.891527 Max.   :  3.0938
##      x.17     x.18     x.19     x.20
## Min.   :-3.28570 Min.   :-4.06416 Min.   :-3.0443 Min.   :-4.06307
## 1st Qu.: -0.72302 1st Qu.: -0.72507 1st Qu.: -0.6684 1st Qu.: -0.70518
## Median : -0.02439 Median : -0.04941 Median : -0.0610 Median : -0.07697
## Mean   : -0.01459 Mean   : -0.03164 Mean   : -0.0414 Mean   : -0.05302
## 3rd Qu.:  0.62692 3rd Qu.:  0.68115 3rd Qu.:  0.6381 3rd Qu.:  0.58597
## Max.   :  2.86446 Max.   :  3.32958 Max.   :  3.1722 Max.   :  3.01358
##      y
## Min.   :-199.268
## 1st Qu.: -54.758
## Median :  -1.607
## Mean   :  -1.710
## 3rd Qu.:  49.367
## Max.   : 278.244
```

b) Train Test Split

```
train_ind = sample(modelDat %>% nrow,100)
test_ind = setdiff(seq_len(modelDat %>% nrow), train_set)
```

Best subset selection

```
train_set<-modelDat[train_ind,]
test_set<-modelDat[-train_ind,]
```

```
linCol<-train(y~.,data=train_set,method="lm")
linCol %>% summary
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.12474 -0.53970 -0.00944  0.42398  2.21086
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.06052    0.09604   -0.630   0.530
## x.1          -0.02265    0.09198   -0.246   0.806
## x.2          28.91650    0.09879  292.719 <2e-16 ***
## x.3          14.16532    0.09343  151.610 <2e-16 ***
## x.4          28.16256    0.09828  286.564 <2e-16 ***
## x.5           0.13742    0.09658    1.423   0.159
## x.6          27.01497    0.08540  316.341 <2e-16 ***
## x.7          31.15917    0.09003  346.092 <2e-16 ***
## x.8          -9.66308    0.11095  -87.094 <2e-16 ***
## x.9           0.11641    0.10768    1.081   0.283
## x.10         19.06687    0.09662  197.344 <2e-16 ***
## x.11         -9.09956    0.08627 -105.472 <2e-16 ***
## x.12         -8.01933    0.10198  -78.633 <2e-16 ***
## x.13          4.26852    0.09888   43.170 <2e-16 ***
## x.14         20.22366    0.09853  205.247 <2e-16 ***
## x.15         -0.16607    0.10466   -1.587   0.117
## x.16          7.95594    0.11250   70.721 <2e-16 ***
## x.17         10.89851    0.11157   97.684 <2e-16 ***
## x.18         -1.09760    0.09391  -11.688 <2e-16 ***
## x.19         22.05197    0.08697  253.553 <2e-16 ***
## x.20         20.88796    0.09274  225.221 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8583 on 79 degrees of freedom
## Multiple R-squared:  0.9999, Adjusted R-squared:  0.9999
## F-statistic: 4.71e+04 on 20 and 79 DF,  p-value: < 2.2e-16
```

```
linPred<-predict(linCol,test_set)
linPred %>% postResample(obs = test_set$y)
```

```
##      RMSE  Rsquared      MAE
## 1.2151815 0.9997265 0.9638378
```

```
L2Grid <- expand.grid(alpha=0,
                     lambda=10^seq(from=-3,to=30,length=100))
```

```
ridgCol<-train(y~.,data=train_set,method="glmnet",tuneGrid = L2Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

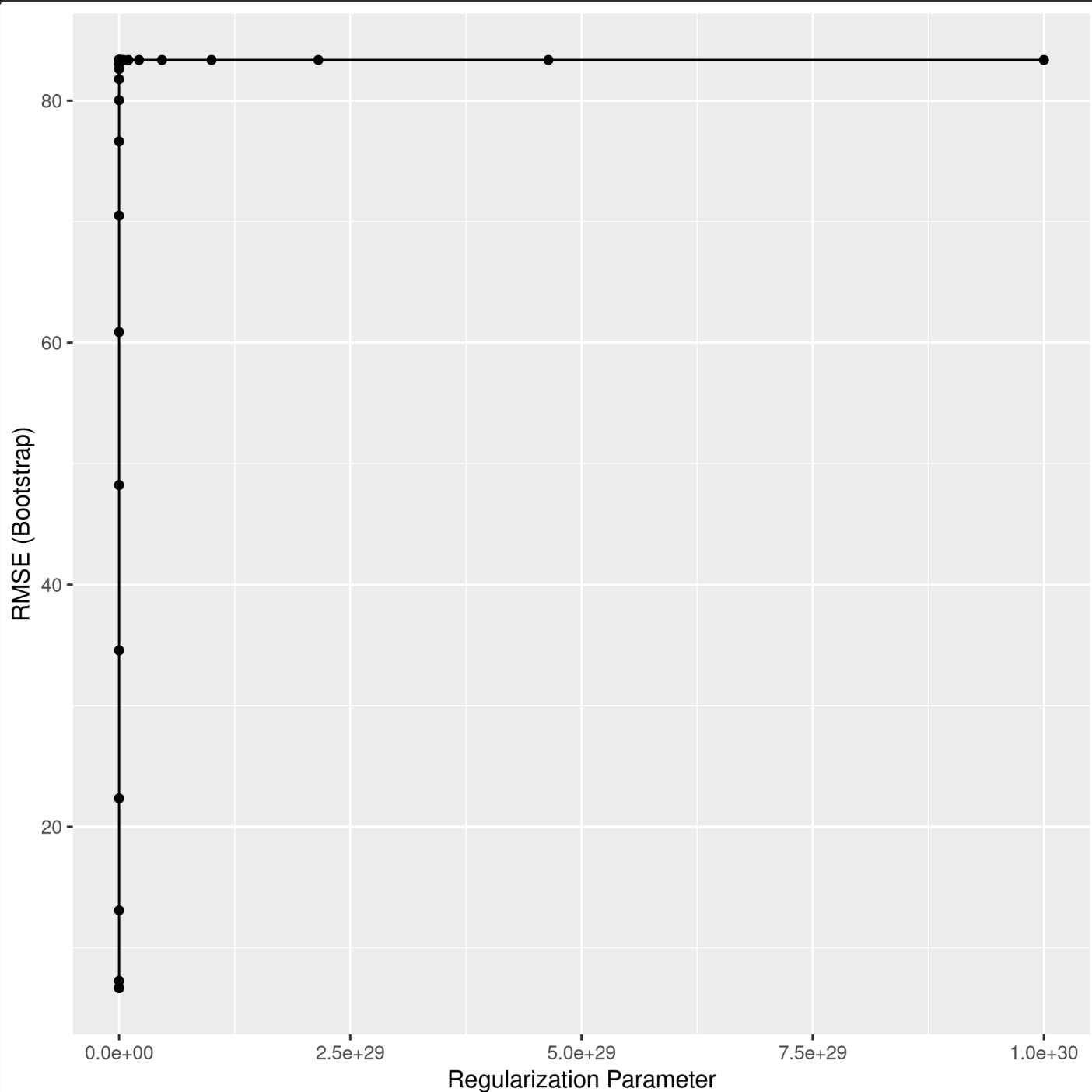
```
ridgCol %>% summary %>% print
```

##	Length	Class	Mode
## a0	100	-none-	numeric
## beta	2000	dgCMatrix	S4
## df	100	-none-	numeric
## dim	2	-none-	numeric
## lambda	100	-none-	numeric
## dev.ratio	100	-none-	numeric
## nulldev	1	-none-	numeric
## npasses	1	-none-	numeric
## jerr	1	-none-	numeric
## offset	1	-none-	logical
## call	5	-none-	call
## nobs	1	-none-	numeric
## lambdaOpt	1	-none-	numeric
## xNames	20	-none-	character
## problemType	1	-none-	character
## tuneValue	2	data.frame	list
## obsLevels	1	-none-	logical
## param	0	-none-	list

```
coef(ridgCol$finalModel, ridgCol$bestTune$lambda) %>% print
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  0.03898376
## x.1          -0.12140945
## x.2          27.63771674
## x.3          13.46853844
## x.4          26.54402352
## x.5          -0.13838118
## x.6          25.87706885
## x.7          29.90687677
## x.8          -9.42088971
## x.9          -0.08983349
## x.10         17.45444598
## x.11         -8.33991071
## x.12         -7.23653865
## x.13          3.35145521
## x.14         19.42178898
## x.15         -0.02794731
## x.16          7.63951382
## x.17         11.08083907
## x.18         -1.36872894
## x.19         20.90257005
## x.20         20.07494414
```

```
ggplot(ridgCol)
```



```
ridgPred<-predict(ridgCol,test_set)
ridgPred %>% postResample(obs = test_set$y)
```

```
##      RMSE  Rsquared      MAE
## 3.7554417 0.9994231 3.0184859
```

```
L1Grid <- expand.grid(alpha=1, # for lasso
                      lambda=10^seq(from=-3,to=30,length=100))
```

```
lassoCol<-train(y~.,data=train_set,method="glmnet",tuneGrid = L1Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

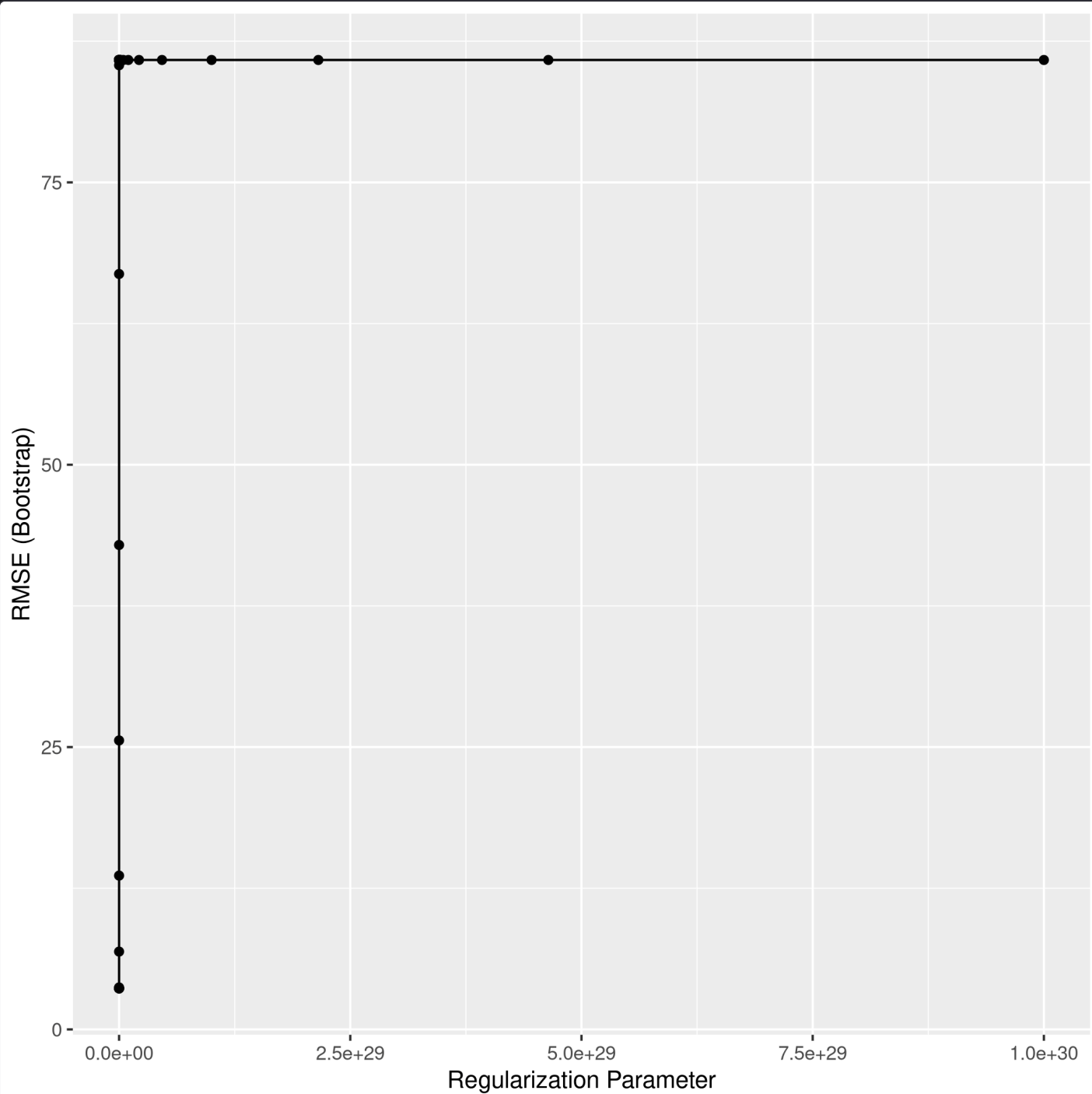
```
lassoCol %>% summary %>% print
```

##	Length	Class	Mode
## a0	47	-none-	numeric
## beta	940	dgCMatrix	S4
## df	47	-none-	numeric
## dim	2	-none-	numeric
## lambda	47	-none-	numeric
## dev.ratio	47	-none-	numeric
## nulldev	1	-none-	numeric
## npasses	1	-none-	numeric
## jerr	1	-none-	numeric
## offset	1	-none-	logical
## call	5	-none-	call
## nobs	1	-none-	numeric
## lambdaOpt	1	-none-	numeric
## xNames	20	-none-	character
## problemType	1	-none-	character
## tuneValue	2	data.frame	list
## obsLevels	1	-none-	logical
## param	0	-none-	list

```
coef(lassoCol$finalModel, lassoCol$bestTune$lambda) %>% print
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept)  0.1158884
## x.1          .
## x.2          28.5897869
## x.3          13.3637110
## x.4          27.2558797
## x.5          .
## x.6          26.6625588
## x.7          30.6841774
## x.8          -9.1388677
## x.9          .
## x.10         17.9220939
## x.11         -8.2461257
## x.12         -7.0603651
## x.13         3.2052101
## x.14         19.7219890
## x.15         .
## x.16         7.2082509
## x.17         10.4137411
## x.18         -0.6693664
## x.19         21.5357460
## x.20         20.5226071
```

```
ggplot(lassoCol)
```

```
lassoPred<-predict(lassoCol,test_set)
lassoPred %>% postResample(obs = test_set$y)
```

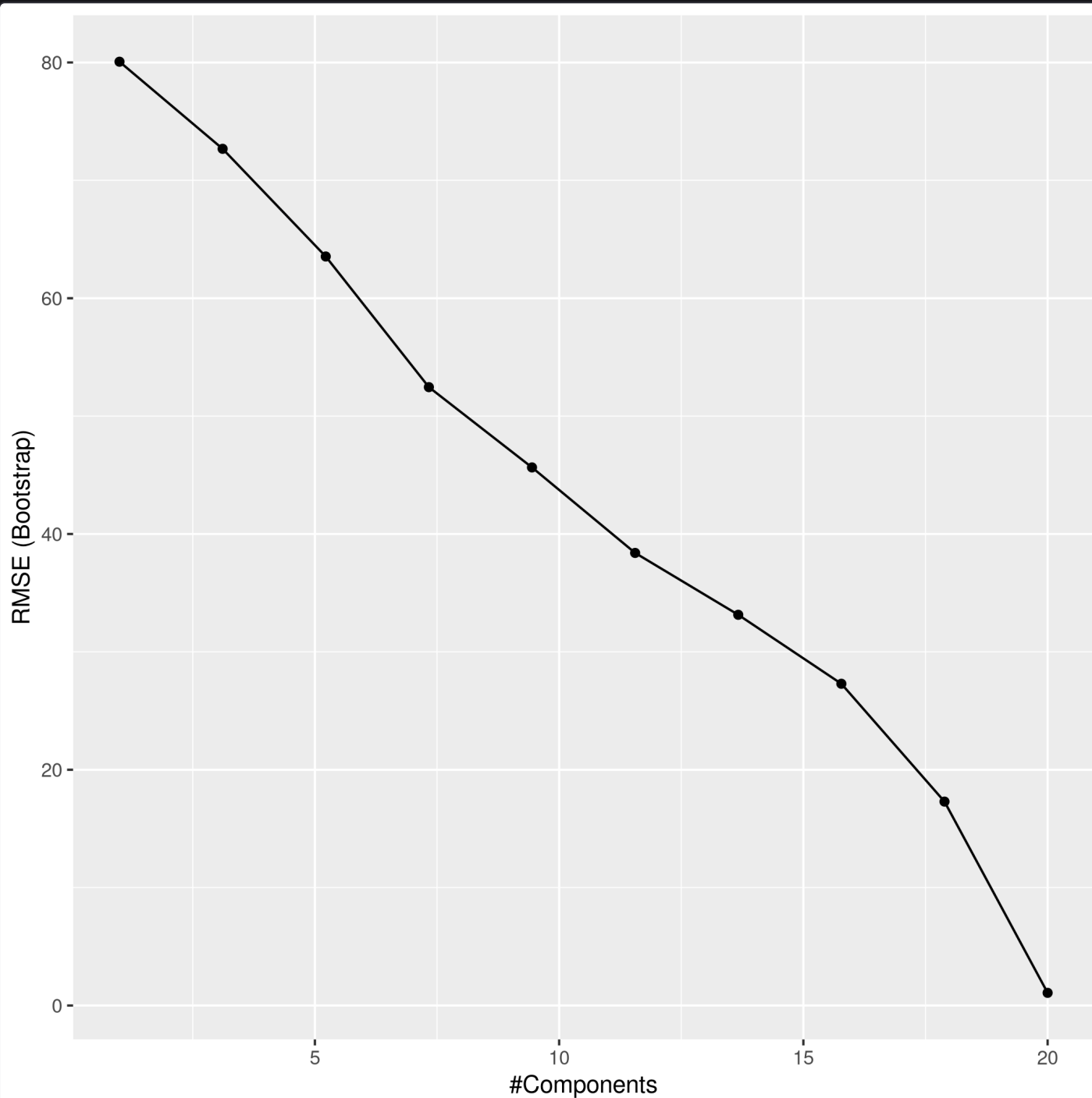
```
##      RMSE  Rsquared    MAE
## 2.7289452 0.9992454 2.2029482
```

```
mGrid <- expand.grid(ncomp=seq(from=1,to=20,length=10))
```

```
pcrCol<-train(y~.,data=train_set,method="pcr",tuneGrid = mGrid)
pcrCol %>% summary %>% print
```

```
## Data:    X dimension: 100 20
## Y dimension: 100 1
## Fit method: svdpc
## Number of components considered: 20
## TRAINING: % variance explained
##          1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X          10.040   18.46   26.62   34.56   41.87   48.54   54.56
## .outcome    8.425   34.90   41.09   43.12   45.06   48.09   66.44
##          8 comps  9 comps 10 comps 11 comps 12 comps 13 comps 14 comps
## X          60.33   65.37   70.01   74.46   78.72   82.32   85.67
## .outcome    85.76   88.81   89.93   91.66   91.91   92.04   92.08
##          15 comps 16 comps 17 comps 18 comps 19 comps 20 comps
## X          88.85   91.94   94.59   96.73   98.51  100.00
## .outcome    92.15   94.96   99.51   99.57   99.76   99.99
## NULL
```

```
ggplot(pcrCol)
```



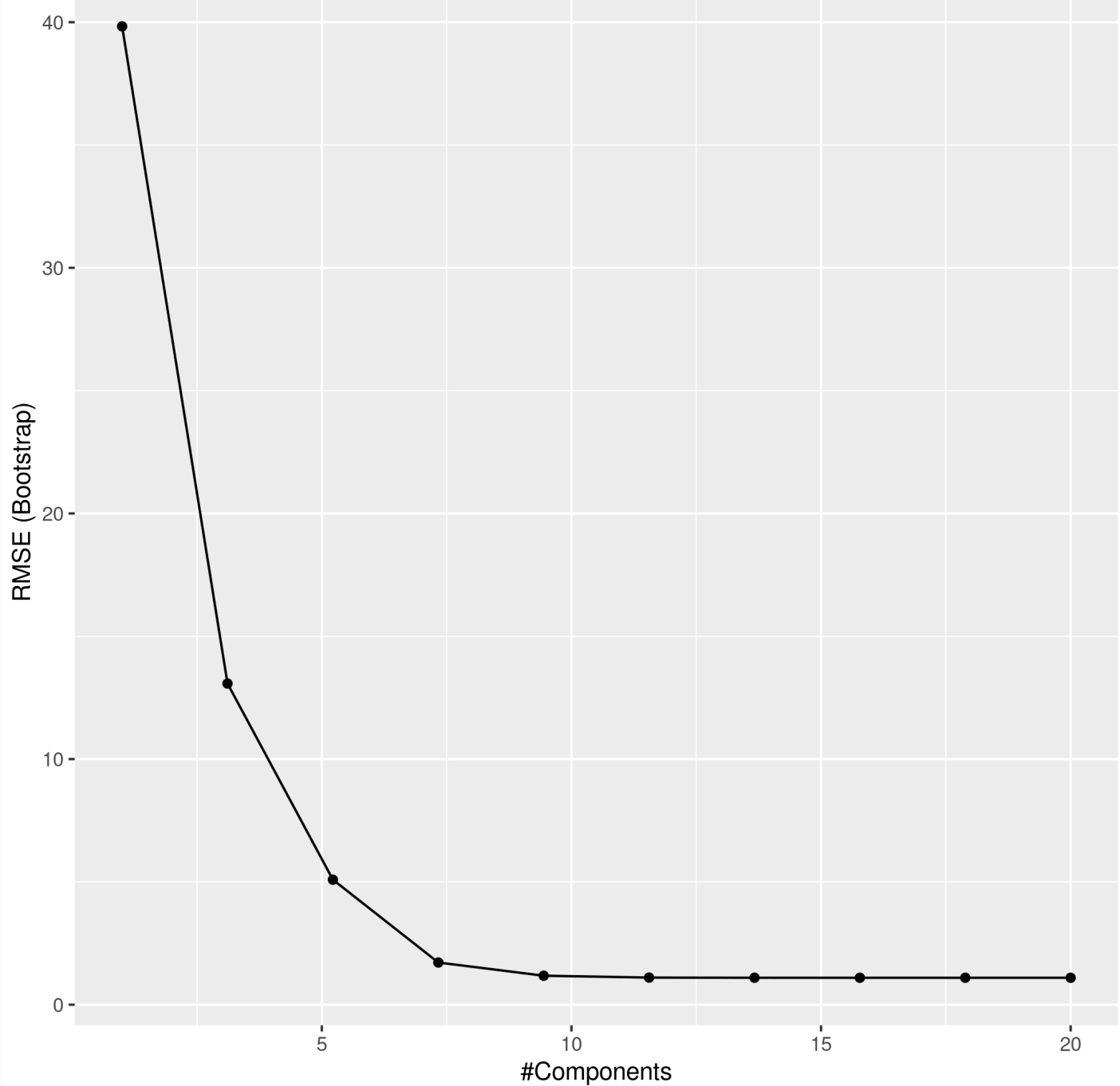
```
pcrPred<-predict(pcrCol,test_set)
pcrPred %>% postResample(obs = test_set$y)
```

```
##      RMSE  Rsquared      MAE
## 1.2151815 0.9997265 0.9638378
```

```
plsCol<-train(y~.,data=train_set,method="pls",tuneGrid = mGrid)
plsCol %>% summary %>% print
```

```
## Data:      X dimension: 100 20
## Y dimension: 100 1
## Fit method: oscorespls
## Number of components considered: 20
## TRAINING: % variance explained
##           1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X           7.762   14.79   21.01   26.89   31.55   36.13   41.12
## .outcome    92.765   98.81   99.75   99.96   99.98   99.99   99.99
##           8 comps  9 comps 10 comps 11 comps 12 comps 13 comps 14 comps
## X           46.35   51.21   56.12   60.34   65.63   71.57   76.16
## .outcome    99.99   99.99   99.99   99.99   99.99   99.99   99.99
##          15 comps 16 comps 17 comps 18 comps 19 comps 20 comps
## X           80.72   84.69   88.98   92.69   96.71   100.00
## .outcome    99.99   99.99   99.99   99.99   99.99   99.99
## NULL
```

```
ggplot(plsCol)
```



```
plsPred<-predict(plsCol,test_set)
plsPred %>% postResample(obs = test_set$y)
```

```
##      RMSE  Rsquared    MAE
## 1.2151815 0.9997265 0.9638378
```

d) Test MSE for best models

- All the models have the same R^2 but Ridge does the worst followed by LASSO

For the rest of the question, we will consider the `OLS` model.

```
modelFit<-regsubsets(y~.,data=modelDat,nvmax=20)
modelFit %>% summary %>% print
```

```
## Subset selection object
## Call: regsubsets.formula(y ~ ., data = modelDat, nvmax = 20)
## 20 Variables (and intercept)
##      Forced in Forced out
## x.1      FALSE      FALSE
## x.2      FALSE      FALSE
## x.3      FALSE      FALSE
## x.4      FALSE      FALSE
## x.5      FALSE      FALSE
## x.6      FALSE      FALSE
## x.7      FALSE      FALSE
## x.8      FALSE      FALSE
## x.9      FALSE      FALSE
## x.10     FALSE      FALSE
## x.11     FALSE      FALSE
## x.12     FALSE      FALSE
## x.13     FALSE      FALSE
## x.14     FALSE      FALSE
## x.15     FALSE      FALSE
## x.16     FALSE      FALSE
## x.17     FALSE      FALSE
## x.18     FALSE      FALSE
## x.19     FALSE      FALSE
## x.20     FALSE      FALSE
## 1 subsets of each size up to 20
## Selection Algorithm: exhaustive
##      x.1 x.2 x.3 x.4 x.5 x.6 x.7 x.8 x.9 x.10 x.11 x.12 x.13 x.14 x.15
## 1 ( 1 ) " " " " " " " " " " " " " " " " " " " " " " " " " " " "
## 2 ( 1 ) " " " " " " " " " " " " " " " " " " " " " " " " " " " "
## 3 ( 1 ) " " " * " " " " " " " " " " " " " " " " " " " " " " " "
## 4 ( 1 ) " " " * " " " " * " " " " " " " " " " " " " " " " " " "
## 5 ( 1 ) " " " * " " " " * " " " " " " " " " " " " " " " * " " "
## 6 ( 1 ) " " " * " " " " * " " * " " " " " " " " " " " " " " "
## 7 ( 1 ) " " " * " " " " * " " * " " " " " " " " " " " * " " "
## 8 ( 1 ) " " " * " " " " * " " * " " " " " " " " " " " * " " "
## 9 ( 1 ) " " " * " " * " * " " " " " " " " " " " " " " * " " "
## 10 ( 1 ) " " " * " " * " * " " " " " " " " " " " " " " * " " "
## 11 ( 1 ) " " " * " " * " * " " " " " " " " " " * " " " " "
## 12 ( 1 ) " " " * " " * " * " " " " " " " " " " * " " " " "
## 13 ( 1 ) " " " * " " * " * " " " " " " " " " " * " " * " " "
## 14 ( 1 ) " " " * " " * " * " " " " " " " " " " * " " " " "
## 15 ( 1 ) " " " * " " * " * " " " " " " " " " " * " " " * " "
## 16 ( 1 ) " " " * " " * " * " " " " " " " " " " * " " " * " "
## 17 ( 1 ) " * " " * " " * " " " " " " " " " " " " " " " * " " "
## 18 ( 1 ) " * " " * " " * " " " " " " " " " " " " " " " * " " "
## 19 ( 1 ) " * " " * " " * " " * " " " " " " " " " " " * " " "
## 20 ( 1 ) " * " " * " " * " " * " " " " " " " " " " " * " " "
##      x.16 x.17 x.18 x.19 x.20
## 1 ( 1 ) " " " " " " " "
## 2 ( 1 ) " " " " " " " "
## 3 ( 1 ) " " " " " " " "
## 4 ( 1 ) " " " " " " " "
## 5 ( 1 ) " " " " " " " "
## 6 ( 1 ) " " " " " " " * " "
## 7 ( 1 ) " " " " " " " * " "
## 8 ( 1 ) " " " " " " " * " "
## 9 ( 1 ) " " " " " " " * " "
## 10 ( 1 ) " " " * " " " " " * "
## 11 ( 1 ) " " " * " " " " " * "
## 12 ( 1 ) " " " * " " " " " * "
## 13 ( 1 ) " " " * " " " " " * "
## 14 ( 1 ) " * " " * " " " " * "
## 15 ( 1 ) " * " " * " " " " * "
## 16 ( 1 ) " * " " * " " * " " * "
## 17 ( 1 ) " * " " * " " * " " *
```

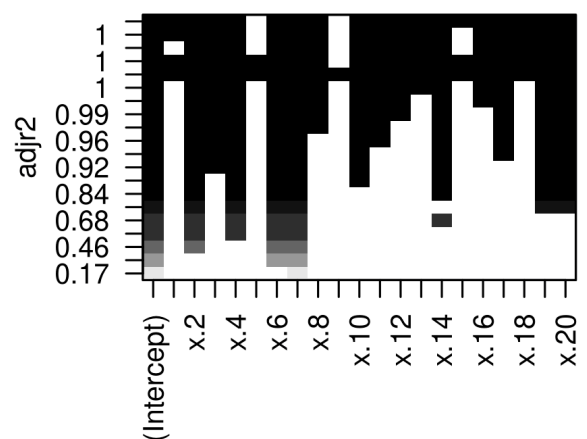
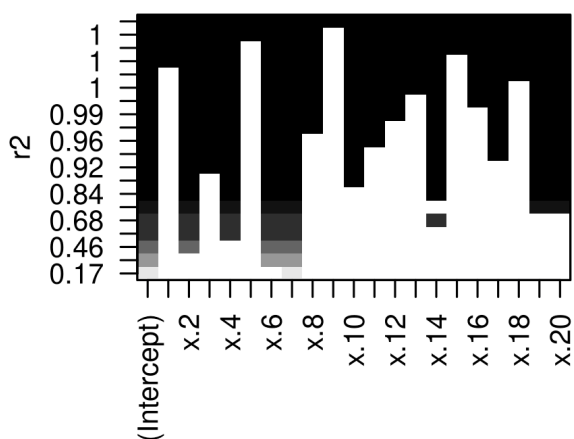
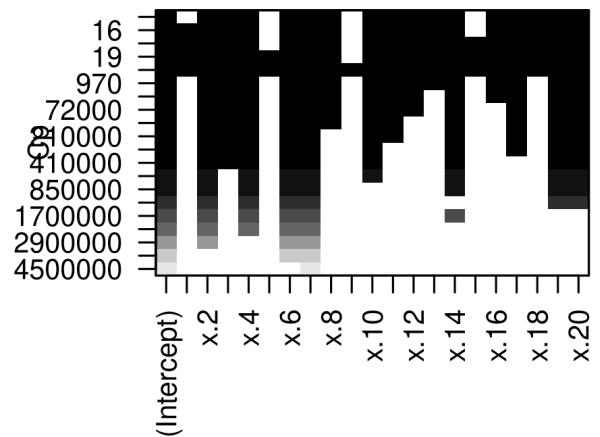
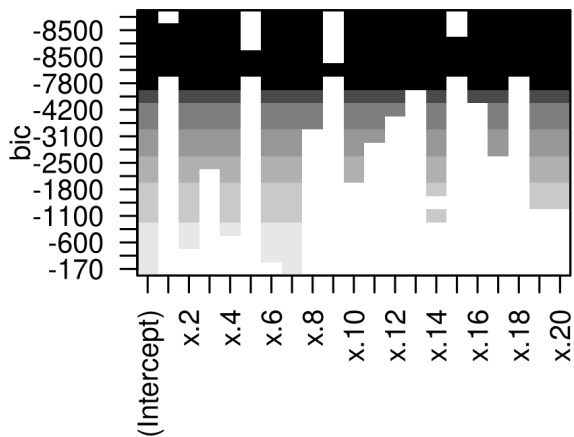
```
## 18 ( 1 ) "*" "*" "*" "*"
## 19 ( 1 ) "*" "*" "*" "*" "*"
## 20 ( 1 ) "*" "*" "*" "*" "*"

```

We might want to take a look at these.

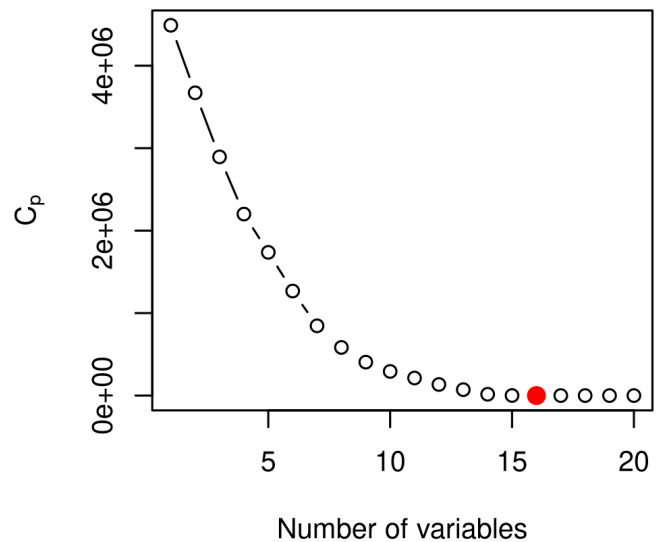
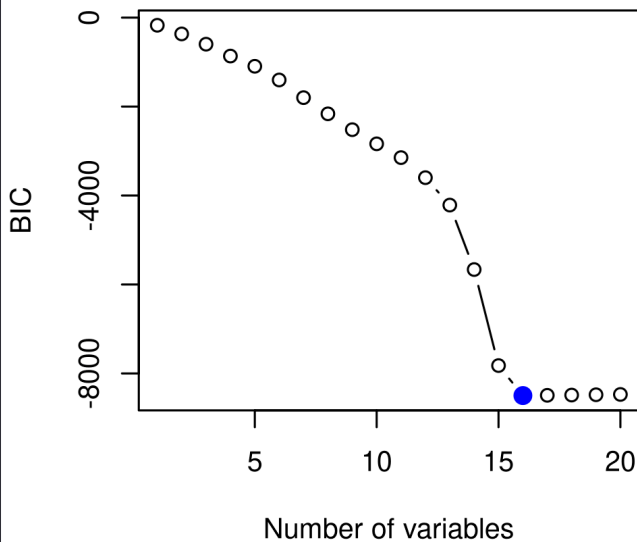
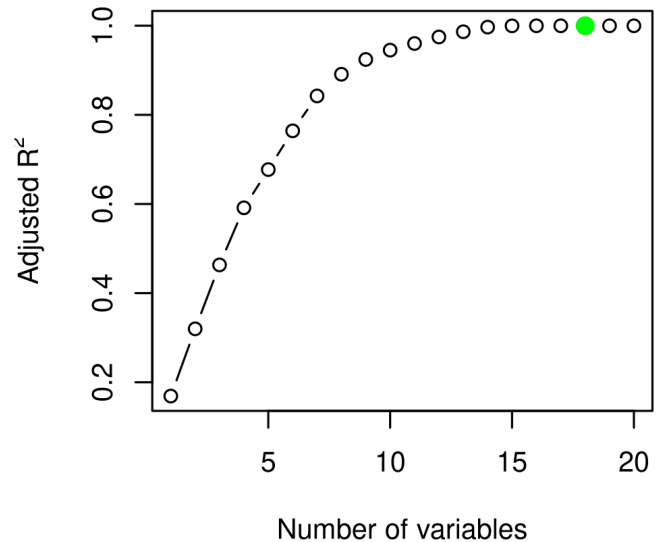
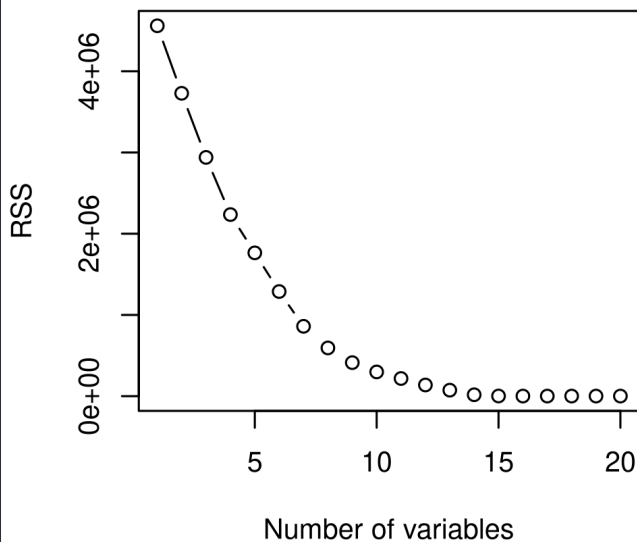
```
par(mfrow=c(2,2))
plot(modelFit)
plot(modelFit,scale='Cp')
plot(modelFit,scale='r2')
plot(modelFit,scale='adjr2')

```



```
plotLEAP(modelFit %>% summary)

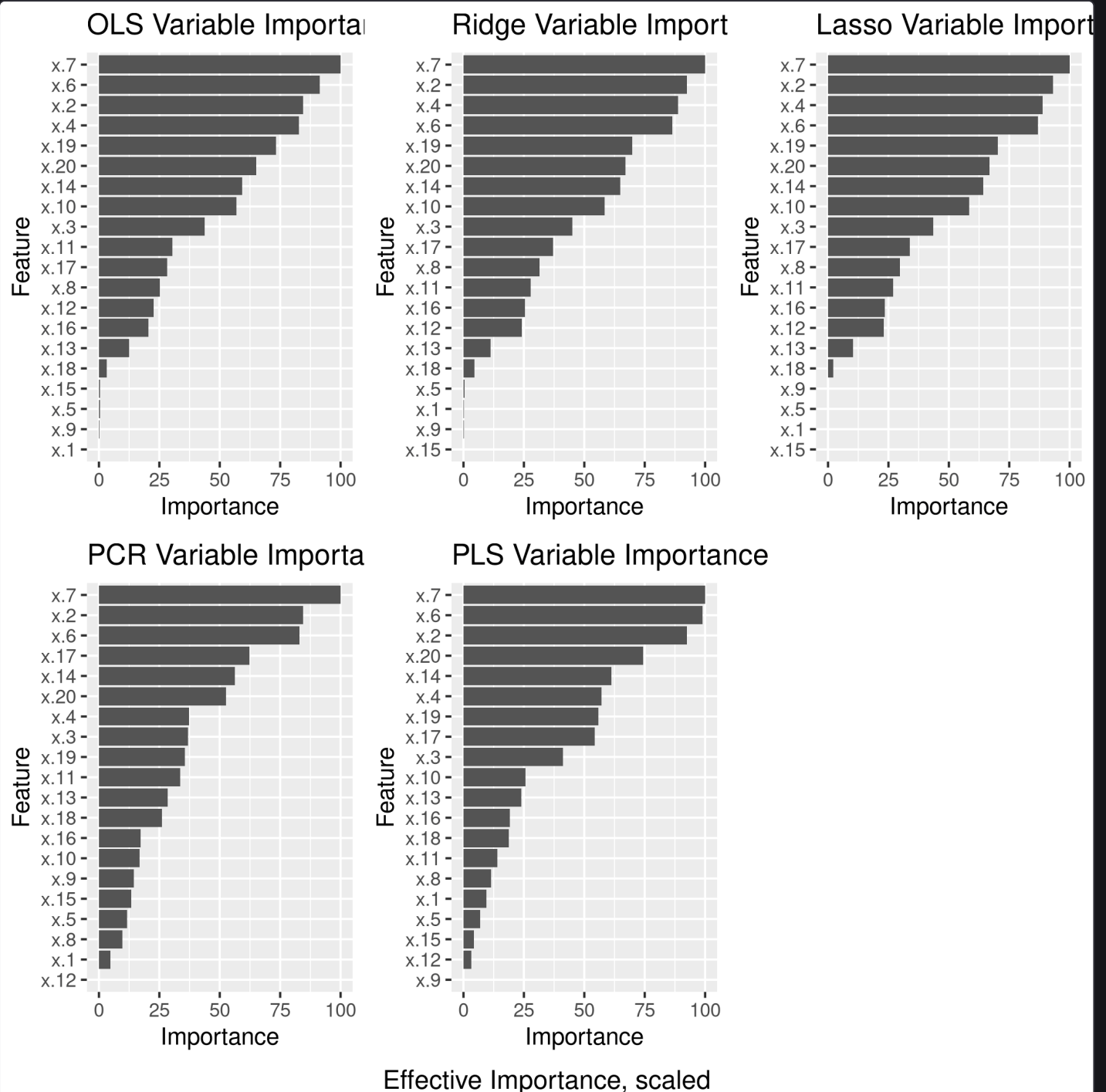
```



It would appear that 16 variables would be a good bet. We note that the lasso model did void out 4 parameters, namely x_1, x_3, x_{13} and x_{17} .

Lets take a quick look at the various model variable significance values.

```
lgp<-linCol %>% varImp %>% ggplot + ggtitle("OLS Variable Importance")
rgp<-ridgCol %>% varImp %>% ggplot + ggtitle("Ridge Variable Importance")
lsgp<-lassoCol %>% varImp %>% ggplot + ggtitle("Lasso Variable Importance")
pcgp<-pcrCol %>% varImp %>% ggplot + ggtitle("PCR Variable Importance")
plgp<-plsCol %>% varImp %>% ggplot + ggtitle("PLS Variable Importance")
grid.arrange(lgp,rgp,lsgp,pcgp,plgp,ncol=3,bottom="Effective Importance, scaled")
```



e) Model size

The test set numeric minimum RMSE is a tie between OLS and PCR, and this was achieved for the (effective) 16 variable OLS model, as well as the 18 variable PCR model.

f) Best model

We will consider the OLS and PCR models and its parameters.

```
linCol$finalModel %>% print
```



```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Coefficients:
## (Intercept)      x.1      x.2      x.3      x.4      x.5
## -0.06052    -0.02265    28.91650    14.16532    28.16256     0.13742
##      x.6      x.7      x.8      x.9     x.10     x.11
##  27.01497   31.15917   -9.66308     0.11641   19.06687   -9.09956
##      x.12     x.13     x.14     x.15     x.16     x.17
##  -8.01933     4.26852   20.22366   -0.16607     7.95594   10.89851
##      x.18     x.19     x.20
##  -1.09760    22.05197    20.88796
```

```
pcrCol$bestTune %>% print
```

```
##      ncomp
## 10      20
```

Now to compare this to the original.

```
beta %>% print
```

```
## [1]  0 29 14 28  0 27 31 -10  0 19 -9 -8  4 20  0  8 11 -1 22
## [20] 21
```

```
t=data.frame(linCol$finalModel$coefficients[-1]) %>% rename("Model_Coeffs"=1) %>% add_column(beta) %>%
rename("Original_Coeffs"=2)
print(t)
```

```
##      Model_Coeffs Original_Coeffs
## x.1    -0.02265289             0
## x.2    28.91649699            29
## x.3    14.16532050            14
## x.4    28.16255937            28
## x.5      0.13741621             0
## x.6    27.01497459            27
## x.7    31.15917172            31
## x.8    -9.66308362           -10
## x.9      0.11641282             0
## x.10   19.06687041            19
## x.11   -9.09955826            -9
## x.12   -8.01932598            -8
## x.13     4.26852334             4
## x.14   20.22366153            20
## x.15   -0.16606531             0
## x.16     7.95593559             8
## x.17   10.89851353            11
## x.18   -1.09759687            -1
## x.19   22.05196537            22
## x.20   20.88795623            21
```

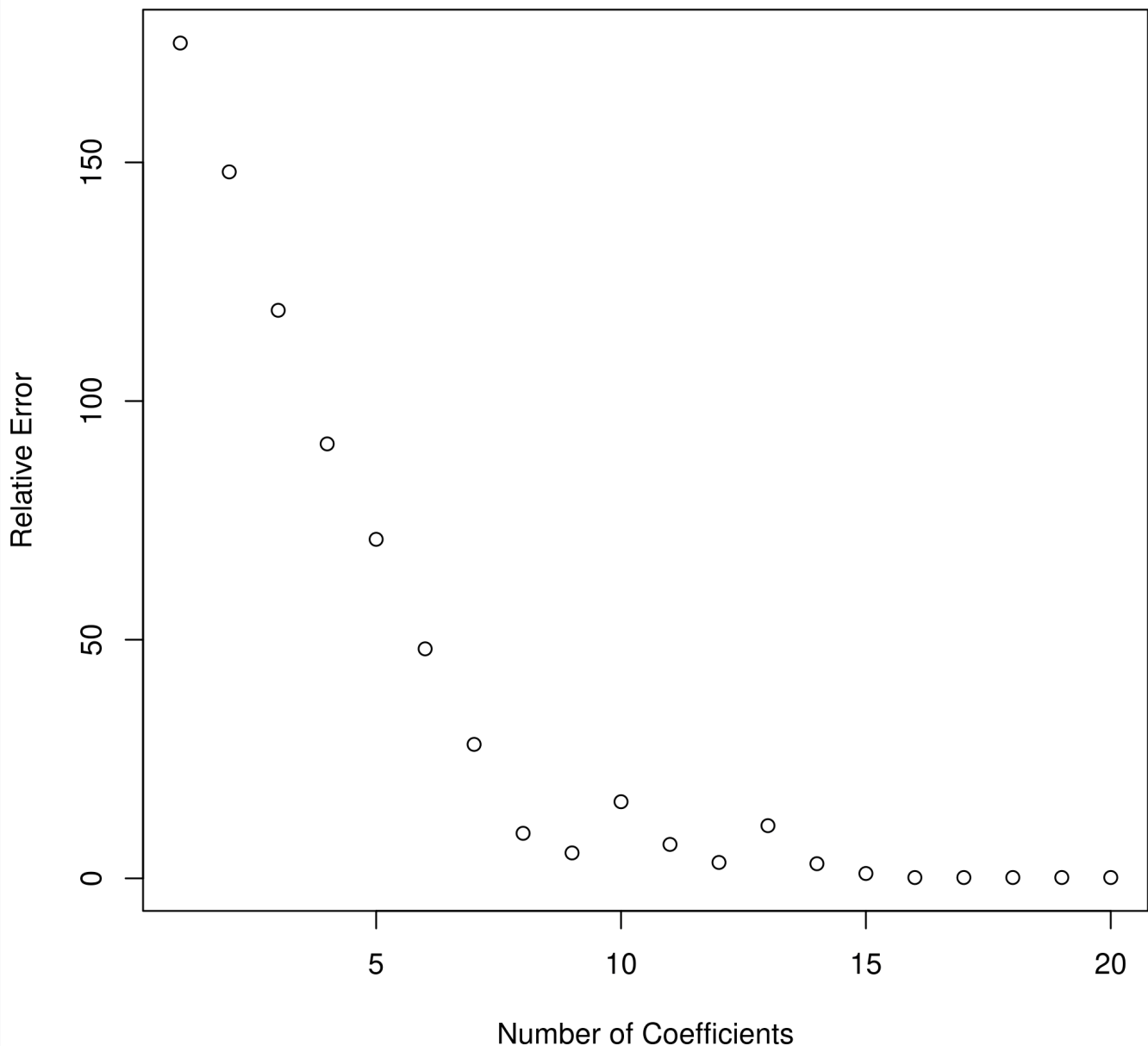
We see that the coefficients are pretty similar.

g) Plotting differences

```

val.errors = rep(NaN, p)
a = rep(NaN, p)
b = rep(NaN, p)
x_cols = colnames(xmat, do.NULL = FALSE, prefix = "x.")
for (i in 1:p) {
  coefi = coef(modelFit, id = i)
  a[i] = length(coefi) - 1 ## Not counting the intercept
  b[i] = sqrt(sum((beta[x_cols %in% names(coefi)] - coefi[names(coefi) %in% x_cols])^2) +
    sum(beta[!(x_cols %in% names(coefi))])^2) ## Handling the intercept
}
plot(x = a, y = b, xlab = "Number of Coefficients", ylab = "Relative Error")

```



Question 6.11 - Page 264

We will now try to predict per capita crime rate in the Boston data set.

(a) Try out some of the regression methods explored in this chapter, such as best subset selection, the lasso, ridge regression, and PCR. Present and discuss results for the approaches that you consider.

(b) Propose a model (or set of models) that seem to perform well on this data set, and justify your answer. Make sure that you are evaluating model performance using validation set error, crossvalidation, or some other reasonable alternative, as opposed to using training error.

(c) Does your chosen model involve all of the features in the data set? Why or why not?

Answer

```
boston<-MASS::Boston
```

- Summarize

```
boston %>% str %>% print
```

```
## 'data.frame':   506 obs. of  14 variables:
## $ crim      : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn        : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus     : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
## $ chas      : int   0 0 0 0 0 0 0 0 0 0 ...
## $ nox       : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
## $ rm        : num  6.58 6.42 7.18 7 7.15 ...
## $ age       : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
## $ dis       : num  4.09 4.97 4.97 6.06 6.06 ...
## $ rad       : int   1 2 2 3 3 3 5 5 5 5 ...
## $ tax       : num  296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio   : num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
## $ black     : num  397 397 393 395 397 ...
## $ lstat     : num  4.98 9.14 4.03 2.94 5.33 ...
## $ medv      : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
## NULL
```

```
boston %>% summary %>% print
```

```
##      crim      zn      indus      chas
## Min.   : 0.00632 Min.   : 0.00 Min.   : 0.46 Min.   :0.00000
## 1st Qu.: 0.08204 1st Qu.: 0.00 1st Qu.: 5.19 1st Qu.:0.00000
## Median : 0.25651 Median : 0.00 Median : 9.69 Median :0.00000
## Mean   : 3.61352 Mean   : 11.36 Mean   :11.14 Mean   :0.06917
## 3rd Qu.: 3.67708 3rd Qu.: 12.50 3rd Qu.:18.10 3rd Qu.:0.00000
## Max.   :88.97620 Max.   :100.00 Max.   :27.74 Max.   :1.00000
##      nox      rm      age      dis
## Min.   :0.3850 Min.   :3.561 Min.   : 2.90 Min.   : 1.130
## 1st Qu.:0.4490 1st Qu.:5.886 1st Qu.: 45.02 1st Qu.: 2.100
## Median :0.5380 Median :6.208 Median : 77.50 Median : 3.207
## Mean   :0.5547 Mean   :6.285 Mean   : 68.57 Mean   : 3.795
## 3rd Qu.:0.6240 3rd Qu.:6.623 3rd Qu.: 94.08 3rd Qu.: 5.188
## Max.   :0.8710 Max.   :8.780 Max.   :100.00 Max.   :12.127
##      rad      tax      ptratio      black
## Min.   : 1.000 Min.   :187.0 Min.   :12.60 Min.   : 0.32
## 1st Qu.: 4.000 1st Qu.:279.0 1st Qu.:17.40 1st Qu.:375.38
## Median : 5.000 Median :330.0 Median :19.05 Median :391.44
## Mean   : 9.549 Mean   :408.2 Mean   :18.46 Mean   :356.67
## 3rd Qu.:24.000 3rd Qu.:666.0 3rd Qu.:20.20 3rd Qu.:396.23
## Max.   :24.000 Max.   :711.0 Max.   :22.00 Max.   :396.90
##      lstat      medv
## Min.   : 1.73 Min.   : 5.00
## 1st Qu.: 6.95 1st Qu.:17.02
## Median :11.36 Median :21.20
## Mean   :12.65 Mean   :22.53
## 3rd Qu.:16.95 3rd Qu.:25.00
## Max.   :37.97 Max.   :50.00
```

```
boston %>% sapply(unique) %>% sapply(length) %>% print
```

```
##      crim      zn      indus      chas      nox      rm      age      dis      rad      tax
##      504      26      76      2      81      446      356      412      9      66
## ptratio      black      lstat      medv
##      46      357      455      229
```

a) Test regression models

```
train_ind = sample(boston %>% nrow,100)
test_ind = setdiff(seq_len(boston %>% nrow), train_set)
```

```
train_set<-boston[train_ind,]
test_set<-boston[-train_ind,]
```

```
linCol<-train(crim~.,data=train_set,method="lm")
linCol %>% summary
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.2431 -1.0344 -0.0563  0.8187  8.1318
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.9339246  7.6508393   0.122   0.9031
## zn           0.0046819  0.0157375   0.297   0.7668
## indus        0.0276209  0.0875254   0.316   0.7531
## chas        -1.1602278  1.2386869  -0.937   0.3516
## nox         -7.5024503  5.0207818  -1.494   0.1388
## rm           1.1240874  0.7462340   1.506   0.1356
## age          0.0020182  0.0137404   0.147   0.8836
## dis         -0.3934753  0.2454365  -1.603   0.1126
## rad           0.4540613  0.0791580   5.736 1.41e-07 ***
## tax           0.0008469  0.0052593   0.161   0.8724
## ptratio     -0.2978204  0.1637629  -1.819   0.0725 .
## black         0.0030642  0.0045281   0.677   0.5004
## lstat         0.1322779  0.0626485   2.111   0.0376 *
## medv        -0.0841382  0.0590700  -1.424   0.1580
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.362 on 86 degrees of freedom
## Multiple R-squared:  0.775, Adjusted R-squared:  0.741
## F-statistic: 22.78 on 13 and 86 DF, p-value: < 2.2e-16
```

```
linPred<-predict(linCol,test_set)
linPred %>% postResample(obs = test_set$crim)
```

```
##      RMSE  Rsquared      MAE
## 7.3794735 0.4056002 2.6774969
```

```
L2Grid <- expand.grid(alpha=0,
                      lambda=10^seq(from=-3,to=30,length=100))
```

```
ridgCol<-train(crim~.,data=train_set,method="glmnet",tuneGrid = L2Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

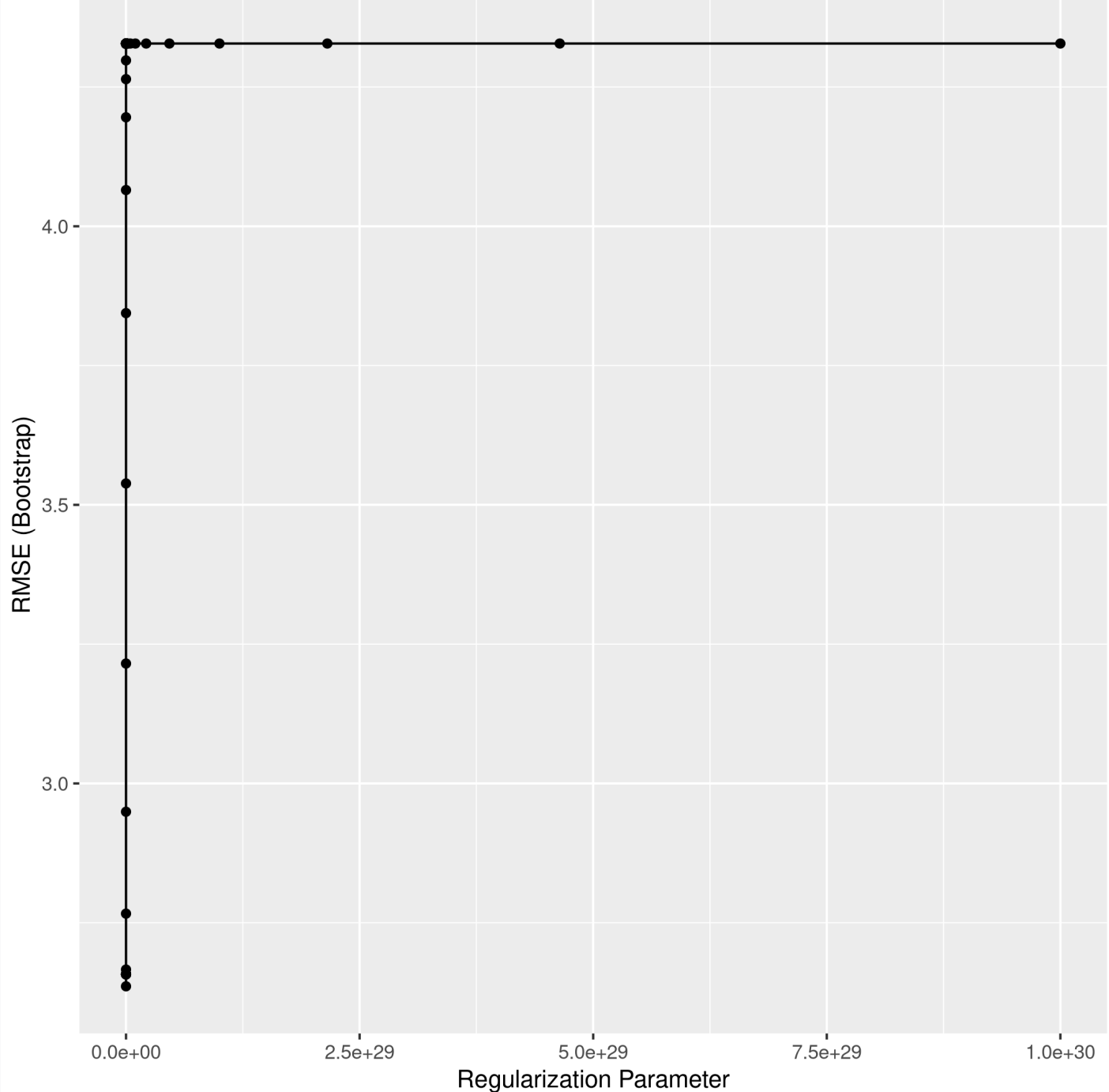
```
ridgCol %>% summary %>% print
```

##	Length	Class	Mode
## a0	100	-none-	numeric
## beta	1300	dgCMatrix	S4
## df	100	-none-	numeric
## dim	2	-none-	numeric
## lambda	100	-none-	numeric
## dev.ratio	100	-none-	numeric
## nulldev	1	-none-	numeric
## npasses	1	-none-	numeric
## jerr	1	-none-	numeric
## offset	1	-none-	logical
## call	5	-none-	call
## nobs	1	-none-	numeric
## lambdaOpt	1	-none-	numeric
## xNames	13	-none-	character
## problemType	1	-none-	character
## tuneValue	2	data.frame	list
## obsLevels	1	-none-	logical
## param	0	-none-	list

```
coef(ridgCol$finalModel, ridgCol$bestTune$lambda) %>% print
```

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) -3.881166065
## zn          0.002597790
## indus       -0.005103517
## chas        -0.674764337
## nox         -0.053645732
## rm          0.600064844
## age         0.001153570
## dis         -0.179295384
## rad         0.267082956
## tax         0.006447932
## ptratio     -0.075885753
## black       -0.001650403
## lstat       0.086462700
## medv        -0.027519270
```

```
ggplot(ridgCol)
```



```
ridgPred<-predict(ridgCol,test_set)
ridgPred %>% postResample(obs = test_set$crim)
```

```
##      RMSE  Rsquared    MAE
## 7.5065916 0.4017056 2.4777547
```

```
L1Grid <- expand.grid(alpha=1, # for lasso
                      lambda=10^seq(from=-3,to=30,length=100))
```

```
lassoCol<-train(crim~.,data=train_set,method="glmnet",tuneGrid = L1Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

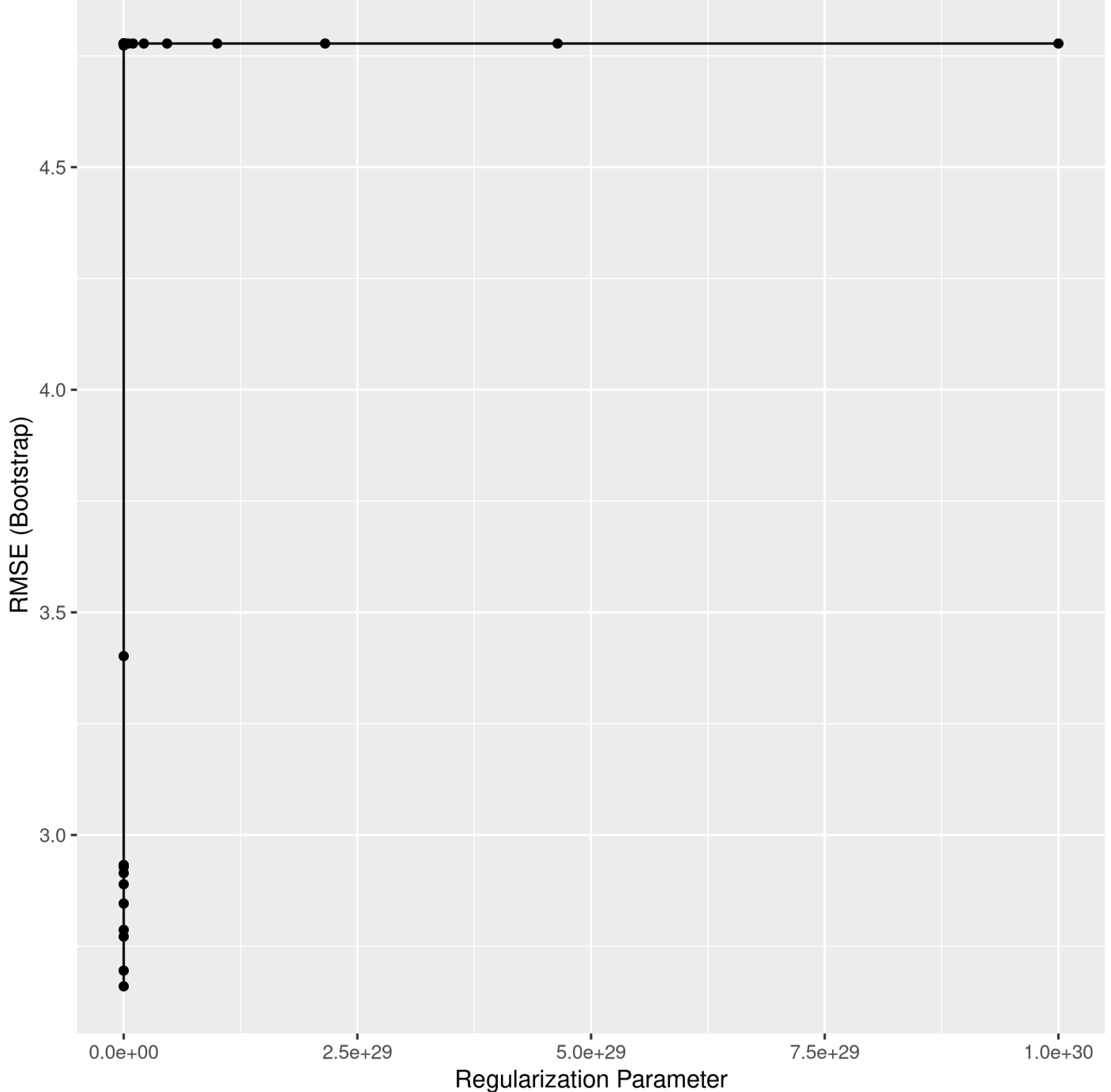
```
lassoCol %>% summary %>% print
```

##	Length	Class	Mode
## a0	78	-none-	numeric
## beta	1014	dgCMatrix	S4
## df	78	-none-	numeric
## dim	2	-none-	numeric
## lambda	78	-none-	numeric
## dev.ratio	78	-none-	numeric
## nulldev	1	-none-	numeric
## npasses	1	-none-	numeric
## jerr	1	-none-	numeric
## offset	1	-none-	logical
## call	5	-none-	call
## nobs	1	-none-	numeric
## lambdaOpt	1	-none-	numeric
## xNames	13	-none-	character
## problemType	1	-none-	character
## tuneValue	2	data.frame	list
## obsLevels	1	-none-	logical
## param	0	-none-	list

```
coef(lassoCol$finalModel, lassoCol$bestTune$lambda) %>% print
```

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) -2.024006430
## zn          .
## indus       .
## chas        .
## nox         .
## rm          .
## age         .
## dis         -0.008506188
## rad         0.386379255
## tax         0.001779579
## ptratio     .
## black       .
## lstat       0.040788606
## medv        .
```

```
ggplot(lassoCol)
```

```
lassoPred<-predict(lassoCol,test_set)
lassoPred %>% postResample(obs = test_set$crim)
```

```
##      RMSE  Rsquared    MAE
## 7.5868293 0.3859121 2.4892258
```

```
mGrid <- expand.grid(ncomp=seq(from=1,to=20,length=10))
```

- All the models have the same R^2 but Ridge does the worst followed by LASSO

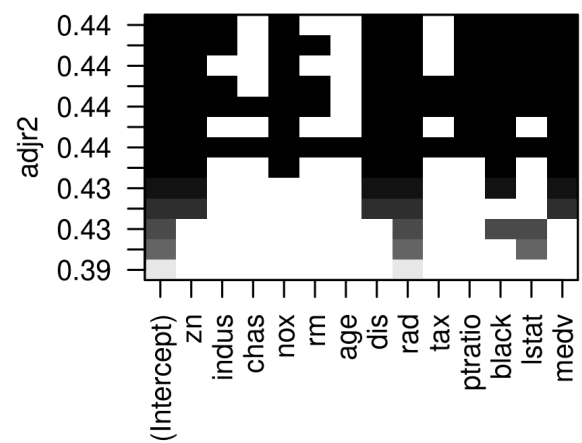
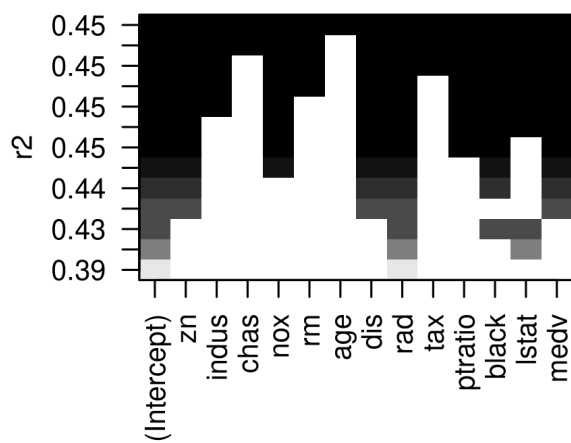
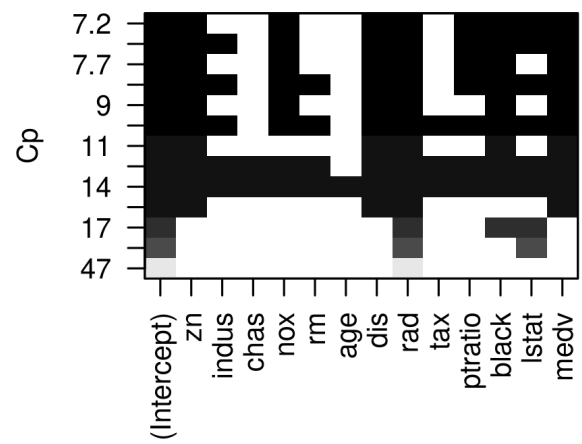
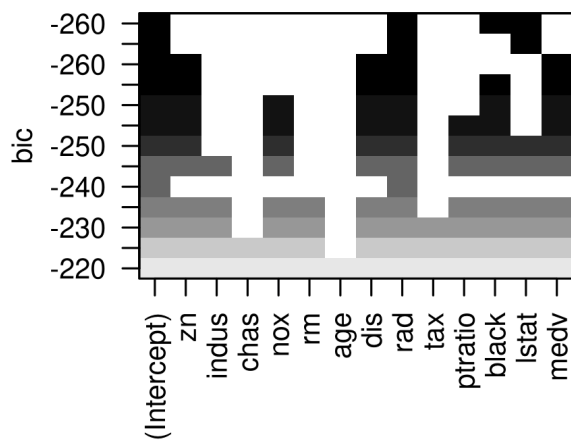
For the rest of the question, we will consider the `OLS` model.

```
modelFit<-regsubsets(crim~.,data=boston,nvmax=20)
modelFit %>% summary %>% print
```

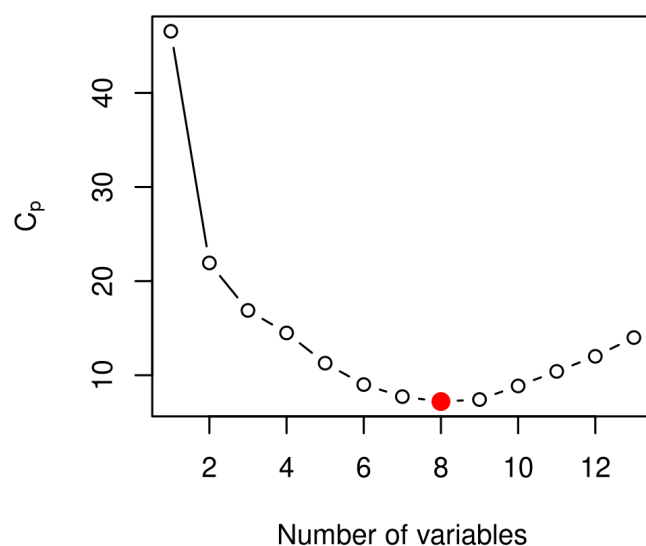
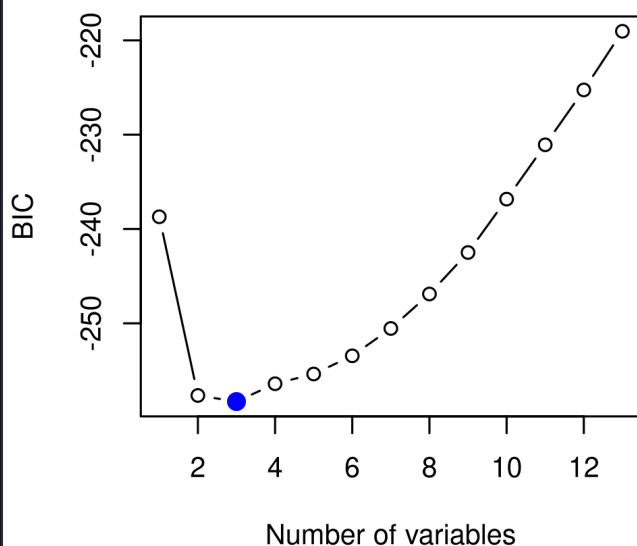
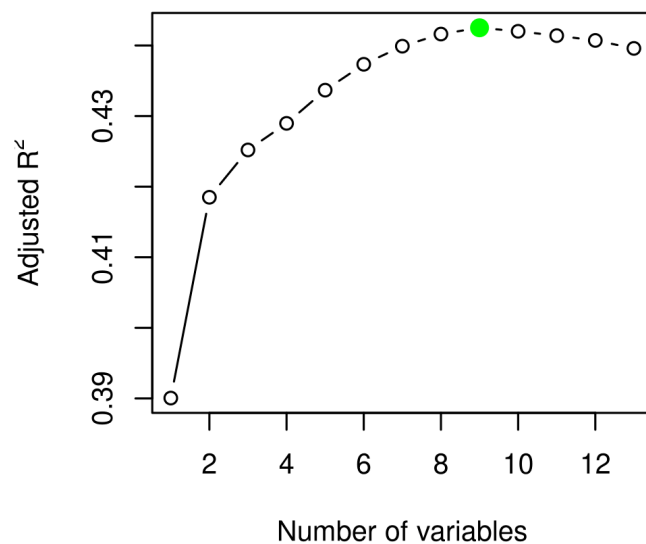
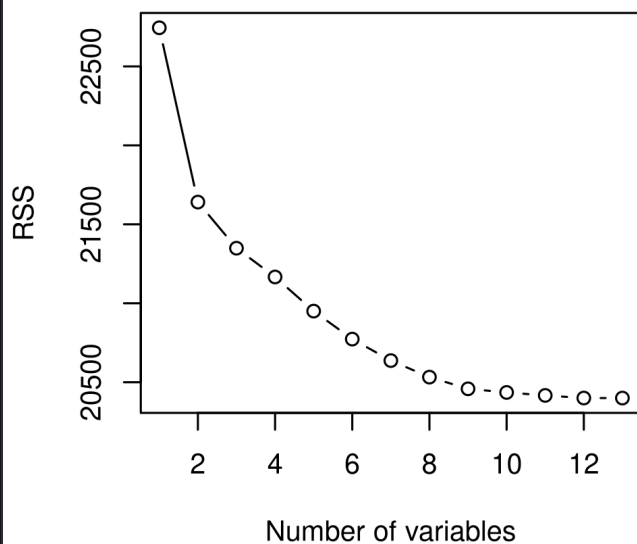
```
## Subset selection object
## Call: regsubsets.formula(crim ~ ., data = boston, nvmax = 20)
## 13 Variables (and intercept)
##           Forced in Forced out
## zn           FALSE      FALSE
## indus         FALSE      FALSE
## chas          FALSE      FALSE
## nox           FALSE      FALSE
## rm            FALSE      FALSE
## age          FALSE      FALSE
## dis          FALSE      FALSE
## rad           FALSE      FALSE
## tax           FALSE      FALSE
## ptratio      FALSE      FALSE
## black         FALSE      FALSE
## lstat        FALSE      FALSE
## medv         FALSE      FALSE
## 1 subsets of each size up to 13
## Selection Algorithm: exhaustive
##           zn indus chas nox rm age dis rad tax ptratio black lstat medv
## 1 ( 1 ) " " " " " " " " " " " " " " " " " " " " " " " " " " " "
## 2 ( 1 ) " " " " " " " " " " " " " " " " " " " " " " " " " " " "
## 3 ( 1 ) " " " " " " " " " " " " " " " " " " " " " " " " " " " "
## 4 ( 1 ) "*" " " " " " " " " " " " " "*" "*" " " " " " " " " " "
## 5 ( 1 ) "*" " " " " " " " " " " " " "*" "*" " " " " " " " " " "
## 6 ( 1 ) "*" " " " " " " "*" " " " " " "*" "*" " " " " " " " " " "
## 7 ( 1 ) "*" " " " " " " "*" " " " " " "*" "*" " " " "*" " " " " " "
## 8 ( 1 ) "*" " " " " " " "*" " " " " " "*" "*" " " " "*" " " " " " "
## 9 ( 1 ) "*" "*" " " " " "*" " " " " " "*" "*" " " " "*" " " " " " "
## 10 ( 1 ) "*" "*" " " " " "*" "*" " " " " "*" "*" " " " " "*" " " " " "
## 11 ( 1 ) "*" "*" " " " " "*" "*" " " " " "*" "*" " " " " "*" " " " " "
## 12 ( 1 ) "*" "*" " " " " "*" "*" " " " " "*" "*" " " " " "*" " " " " "
## 13 ( 1 ) "*" "*" " " " " "*" "*" " " " " "*" "*" " " " " "*" " " " " "
```

We might want to take a look at these.

```
par(mfrow=c(2,2))
plot(modelFit)
plot(modelFit,scale='Cp')
plot(modelFit,scale='r2')
plot(modelFit,scale='adjr2')
```



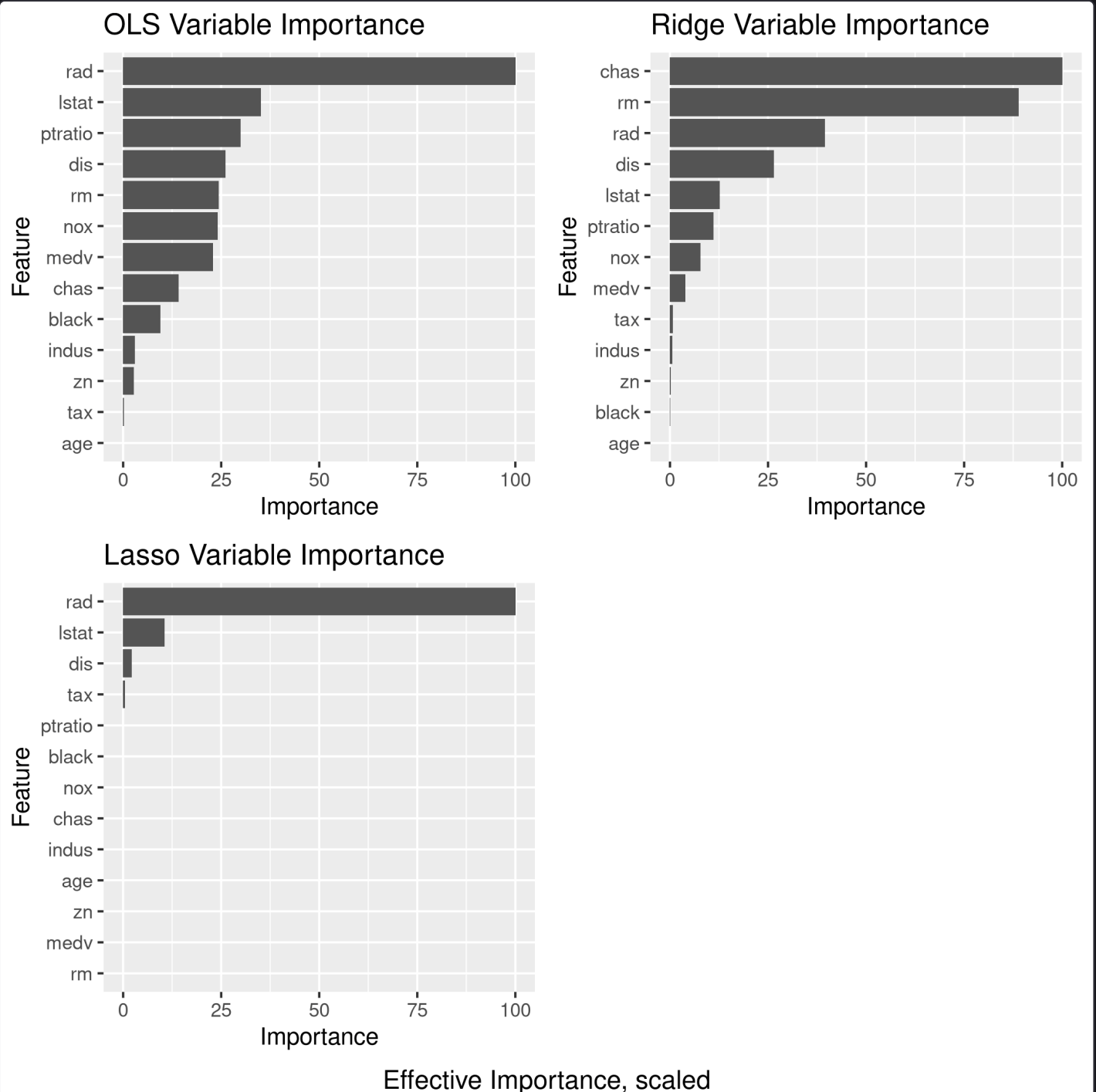
```
plotLEAP(modelFit %>% summary)
```



It would appear that 16 variables would be a good bet. We note that the lasso model did void out 4 parameters, namely x_1, x_3, x_{13} and x_{17} .

Lets take a quick look at the various model variable significance values.

```
lgp<-linCol %>% varImp %>% ggplot + ggtitle("OLS Variable Importance")
rgp<-ridgCol %>% varImp %>% ggplot + ggtitle("Ridge Variable Importance")
lsgp<-lassoCol %>% varImp %>% ggplot + ggtitle("Lasso Variable Importance")
grid.arrange(lgp,rgp,lsgp,ncol=2,bottom="Effective Importance, scaled")
```



b) Propose a model

- Given the data and plots, I would probably end up using the Ridge regression model
- Clearly, LASSO is not working very well since it seems to have taken mainly 3 variables, one of which is largely categorical (9 levels)

c) Model properties

```
boston %>% str %>% print
```

```
## 'data.frame':    506 obs. of  14 variables:
## $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn     : num  18 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 ...
## $ chas   : int   0 0 0 0 0 0 0 0 0 ...
## $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 ...
## $ rm     : num  6.58 6.42 7.18 7 7.15 ...
## $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
## $ dis    : num  4.09 4.97 4.97 6.06 6.06 ...
## $ rad    : int   1 2 2 3 3 3 5 5 5 ...
## $ tax    : num  296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio: num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 ...
## $ black  : num  397 397 393 395 397 ...
## $ lstat  : num  4.98 9.14 4.03 2.94 5.33 ...
## $ medv   : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
## NULL
```

```
boston %>% summary %>% print
```

```
##      crim           zn           indus          chas
## Min.   : 0.00632   Min.   : 0.00   Min.   : 0.46   Min.   :0.00000
## 1st Qu.: 0.08204   1st Qu.: 0.00   1st Qu.: 5.19   1st Qu.:0.00000
## Median : 0.25651   Median : 0.00   Median : 9.69   Median :0.00000
## Mean   : 3.61352   Mean   : 11.36   Mean   :11.14   Mean   :0.06917
## 3rd Qu.: 3.67708   3rd Qu.: 12.50   3rd Qu.:18.10   3rd Qu.:0.00000
## Max.   :88.97620   Max.   :100.00   Max.   :27.74   Max.   :1.00000
##      nox           rm           age           dis
## Min.   :0.3850   Min.   :3.561   Min.   : 2.90   Min.   : 1.130
## 1st Qu.:0.4490   1st Qu.:5.886   1st Qu.: 45.02   1st Qu.: 2.100
## Median :0.5380   Median :6.208   Median : 77.50   Median : 3.207
## Mean   :0.5547   Mean   :6.285   Mean   : 68.57   Mean   : 3.795
## 3rd Qu.:0.6240   3rd Qu.:6.623   3rd Qu.: 94.08   3rd Qu.: 5.188
## Max.   :0.8710   Max.   :8.780   Max.   :100.00   Max.   :12.127
##      rad           tax           ptratio        black
## Min.   : 1.000   Min.   :187.0   Min.   :12.60   Min.   : 0.32
## 1st Qu.: 4.000   1st Qu.:279.0   1st Qu.:17.40   1st Qu.:375.38
## Median : 5.000   Median :330.0   Median :19.05   Median :391.44
## Mean   : 9.549   Mean   :408.2   Mean   :18.46   Mean   :356.67
## 3rd Qu.:24.000   3rd Qu.:666.0   3rd Qu.:20.20   3rd Qu.:396.23
## Max.   :24.000   Max.   :711.0   Max.   :22.00   Max.   :396.90
##      lstat          medv
## Min.   : 1.73   Min.   : 5.00
## 1st Qu.: 6.95   1st Qu.:17.02
## Median :11.36   Median :21.20
## Mean   :12.65   Mean   :22.53
## 3rd Qu.:16.95   3rd Qu.:25.00
## Max.   :37.97   Max.   :50.00
```

```
boston %>% apply(unique) %>% sapply(length) %>% print
```

```
##      crim      zn      indus      chas      nox      rm      age      dis      rad      tax
##      504      26       76        2      81     446     356     412        9      66
## ptratio  black  lstat  medv
##      46     357     455     229
```

- A good idea would be removing `rad` and `chas` from the regression

```
boston<-boston %>% subset(select=-c(rad,chas))
```

```
train_ind = sample(boston %>% nrow,100)
test_ind = setdiff(seq_len(boston %>% nrow), train_ind)
```

```
train_set<-boston[train_ind,]
test_set<-boston[-train_ind,]
```

```
L2Grid <- expand.grid(alpha=0,
                      lambda=10^seq(from=-3,to=30,length=100))
```

```
ridgCol<-train(crim~.,data=train_set,method="glmnet",tuneGrid = L2Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

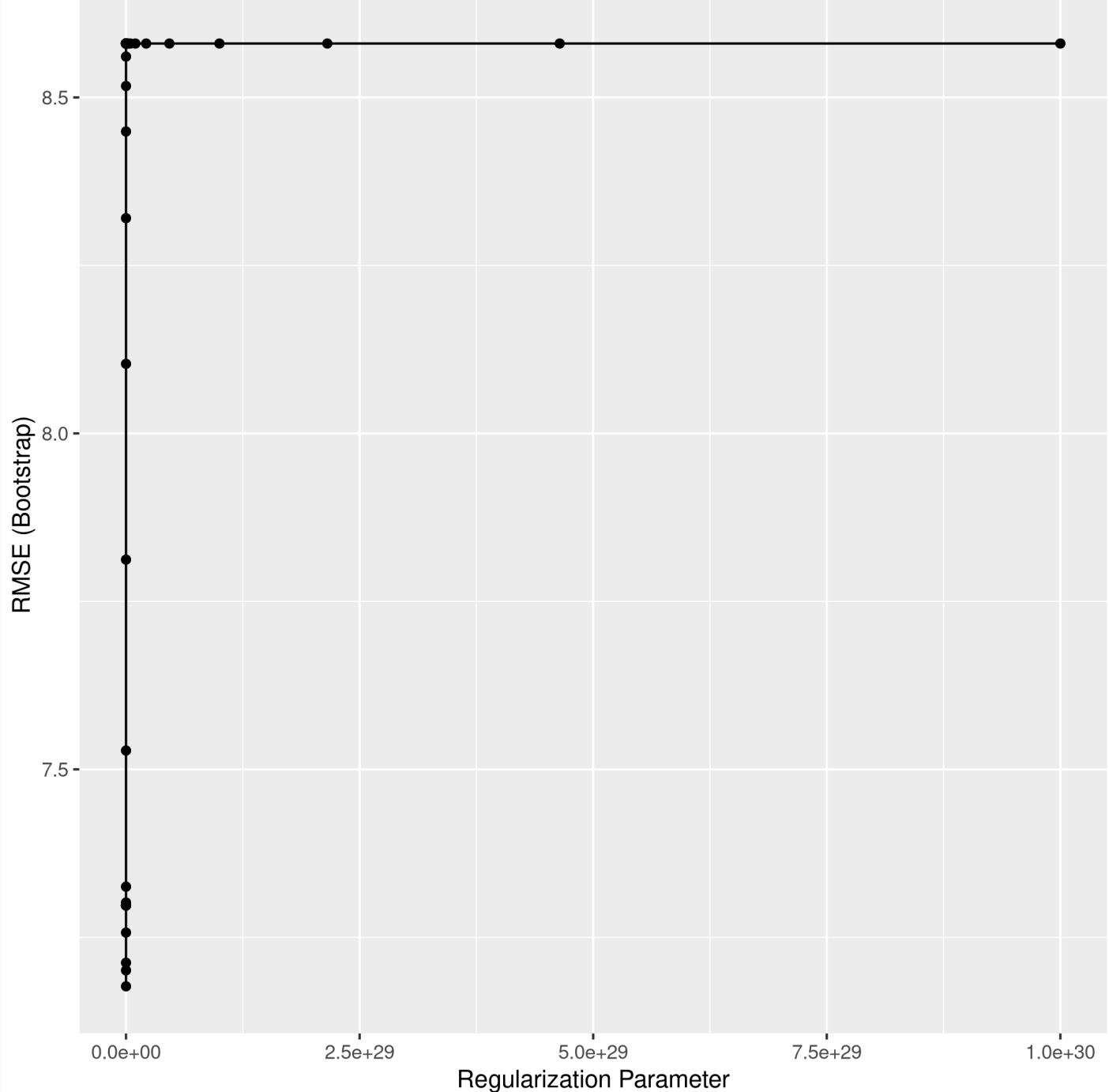
```
ridgCol %>% summary %>% print
```

```
##           Length Class      Mode
## a0           100  -none-   numeric
## beta         1100 dgCMatrix S4
## df            100  -none-   numeric
## dim            2  -none-   numeric
## lambda        100  -none-   numeric
## dev.ratio      100  -none-   numeric
## nulldev         1  -none-   numeric
## npasses         1  -none-   numeric
## jerr            1  -none-   numeric
## offset          1  -none-   logical
## call            5  -none-    call
## nobs            1  -none-   numeric
## lambda0pt       1  -none-   numeric
## xNames          11  -none-   character
## problemType     1  -none-   character
## tuneValue        2 data.frame list
## obsLevels        1  -none-   logical
## param            0  -none-    list
```

```
coef(ridgCol$finalModel, ridgCol$bestTune$lambda) %>% print
```

```
## 12 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) 0.23015398
## zn           0.01595378
## indus        -0.01006683
## nox           3.42104450
## rm           -0.05904911
## age           0.01419585
## dis          -0.16724715
## tax           0.01021790
## ptratio       0.05741508
## black        -0.01414707
## lstat         0.15519923
## medv         -0.04483670
```

```
ggplot(ridgCol)
```



```
ridgPred<-predict(ridgCol,test_set)
ridgPred %>% postResample(obs = test_set$crim)
```

```
##      RMSE  Rsquared      MAE
## 6.9365371 0.3619974 2.8570012
```

```
L1Grid <- expand.grid(alpha=1, # for lasso
                      lambda=10^seq(from=-3,to=30,length=100))
```

```
lassoCol<-train(crim~.,data=train_set,method="glmnet",tuneGrid = L1Grid)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo, :
## There were missing values in resampled performance measures.
```

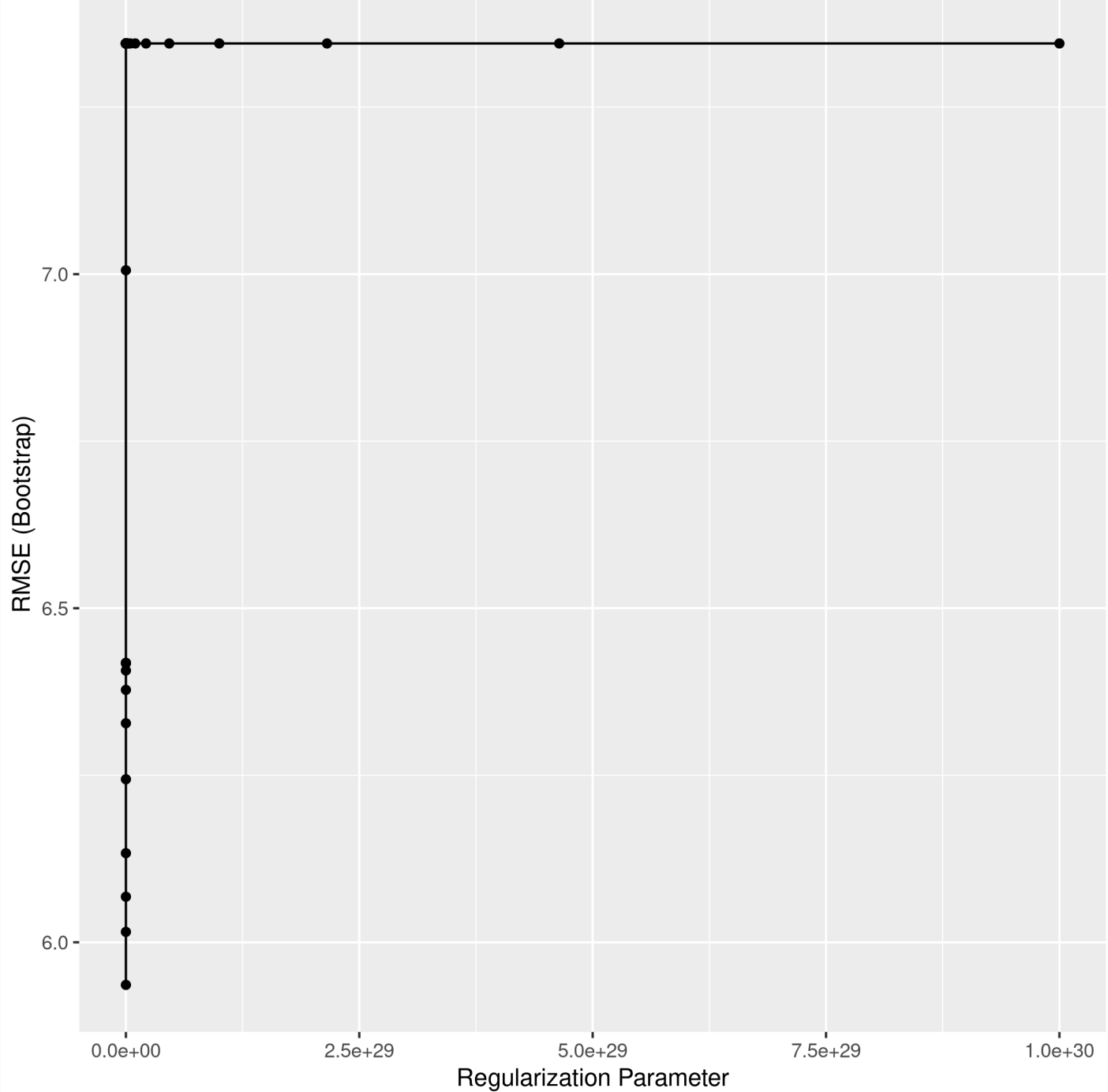
```
lassoCol %>% summary %>% print
```


##	Length	Class	Mode
## a0	74	-none-	numeric
## beta	814	dgCMatrix	S4
## df	74	-none-	numeric
## dim	2	-none-	numeric
## lambda	74	-none-	numeric
## dev.ratio	74	-none-	numeric
## nulldev	1	-none-	numeric
## npasses	1	-none-	numeric
## jerr	1	-none-	numeric
## offset	1	-none-	logical
## call	5	-none-	call
## nobs	1	-none-	numeric
## lambdaOpt	1	-none-	numeric
## xNames	11	-none-	character
## problemType	1	-none-	character
## tuneValue	2	data.frame	list
## obsLevels	1	-none-	logical
## param	0	-none-	list

```
coef(lassoCol$finalModel, lassoCol$bestTune$lambda) %>% print
```

```
## 12 x 1 sparse Matrix of class "dgCMatrix"
##              1
## (Intercept) 0.86635621
## zn          .
## indus       .
## nox         .
## rm          .
## age         .
## dis         .
## tax         0.01402174
## ptratio     .
## black       -0.01454496
## lstat       0.15290845
## medv        .
```

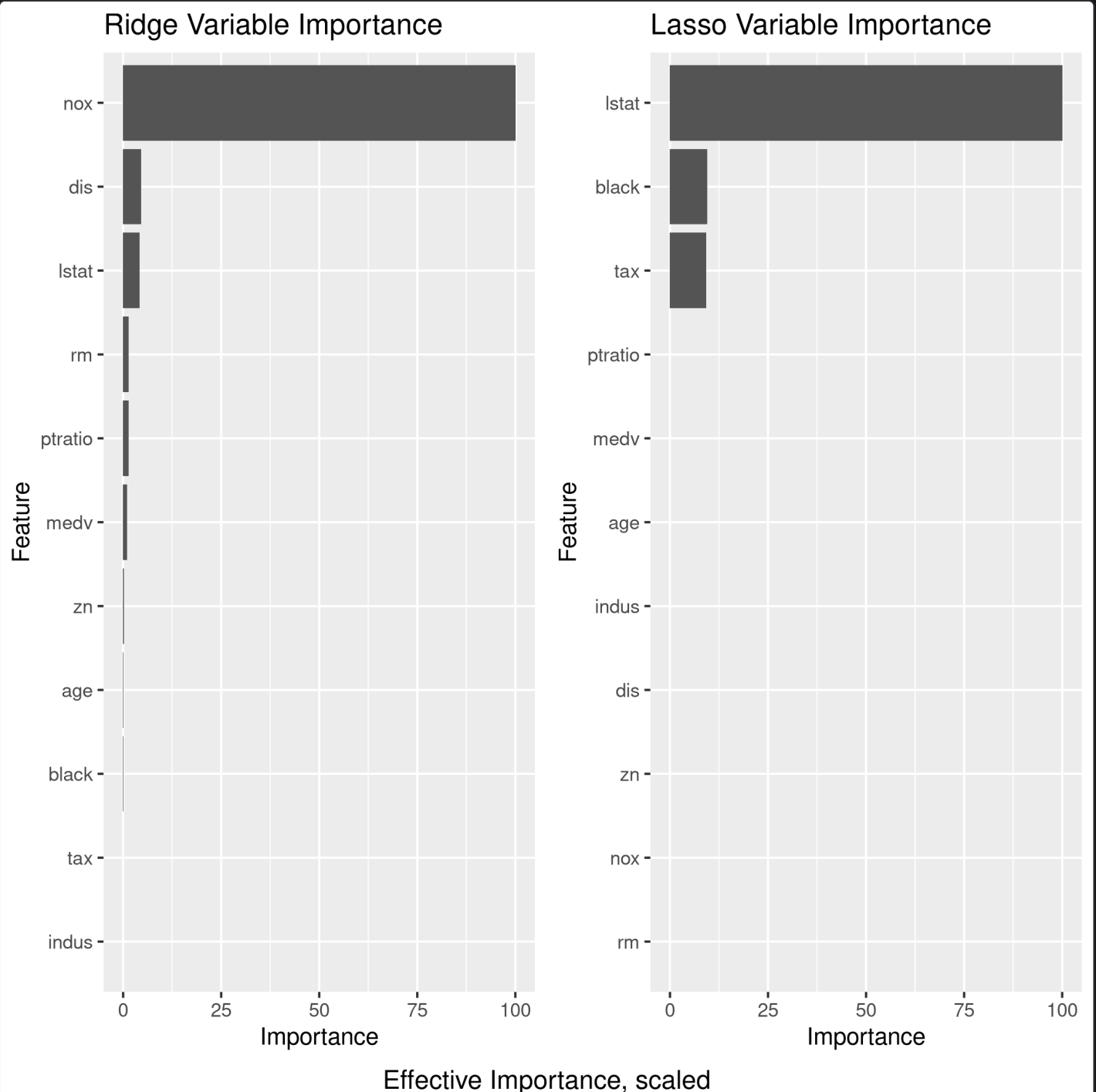
```
ggplot(lassoCol)
```



```
lassoPred<-predict(lassoCol,test_set)
lassoPred %>% postResample(obs = test_set$crim)
```

##	RMSE	Rsquared	MAE
##	6.9630616	0.3693364	2.6767742

```
rgp<-ridgCol %>% varImp %>% ggplot + ggtitle("Ridge Variable Importance")
lsgp<-lassoCol %>% varImp %>% ggplot + ggtitle("Lasso Variable Importance")
grid.arrange(rgp,lsgp,ncol=2,bottom="Effective Importance, scaled")
```



None of these models are actually any good apparently, given that we have an R^2 of 0.3619974 for the L2 regularization and 0.3693364 for the L1.

Introduction to Statistical Learning: with Applications in R.
 Berlin, Germany: Springer Science & Business Media. [^fn:2]: Lang et al., (2019). mlr3: A modern object-oriented machine learning framework in R. Journal of Open Source Software, 4(44), 1903, <<https://doi.org/10.21105/joss.01903>>